Architecture & Infrastructure

Polycarpic — a virtual Single Family Office

Owner: `architect` agent **Status:** Approved **Version:** 1.15.12 **Last updated:** 2026-11-11

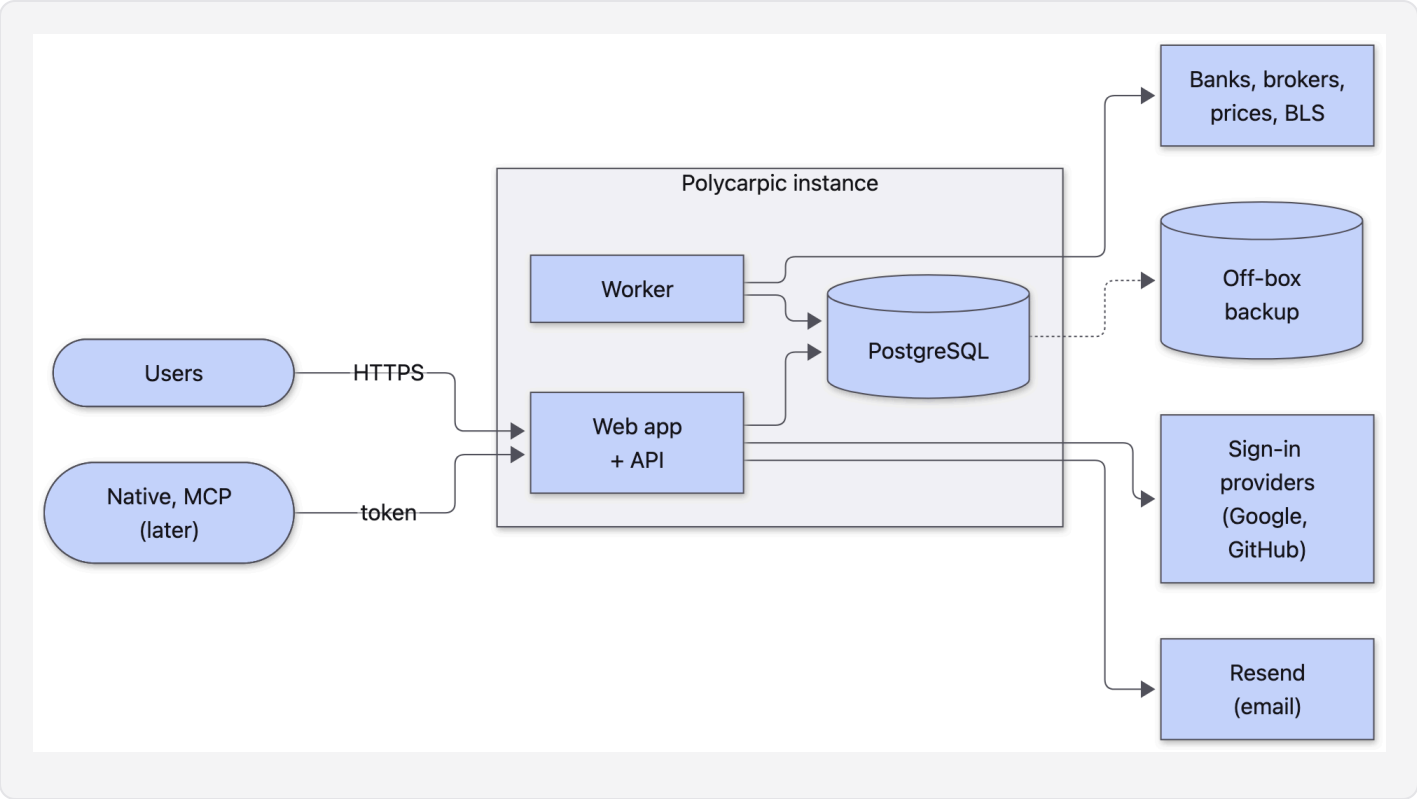
CONTENTS

1. System Context
2. Components & Layer Model
3. Data Model & Database Invariants
4. Data Flow
5. Tech Stack & Rationale
6. Plan-phase Research & Decisions: GL Build-vs-Adopt, Migration Tool
7. Deployment Topology
8. CI/CD Pipeline
9. Integration Points
10. Trade-offs & Alternatives
11. Milestone Map
12. Open Questions

1. System Context

The system and the actors around it. C4 context level.

Polycarpic is a single self-hosted web application backed by one PostgreSQL database. Users (the Principal, household members holding a membership grant, later native iOS/macOS clients and a user-scoped MCP server) reach it over HTTPS. Inside, a genuine double-entry general ledger lives in Postgres under row-level security keyed on the *membership* join. Outbound, the system pulls transactions from bank/brokerage aggregators through a provider-agnostic adapter interface (a deterministic fake first, then CSV/OFX files, direct broker APIs, Plaid and SimpleFIN), end-of-day prices and the CPI-U series as reference data, and sends transactional email through Resend. Identity is local (BetterAuth in our own Postgres) with sign-in through outside identity providers (IdPs): Google and GitHub confirm who a user is, so the user has no separate password with us, and BetterAuth then starts a session of our own. Sign in with Apple arrives with the native iOS clients (POLY-122). Nothing about a user's books leaves the box except through a provider call the user initiated or an owner-initiated export.



2. Components & Layer Model

Each package imports only what the layer map allows, and never upward. Each layer is one or more workspace packages, each with tests that run alone. The boundary is enforced by lint and by package resolution, not by convention.

2.1 Layer model

Kickoff §3 asks for layers defined before details, depending only downward, with narrow interfaces. The model below is the contract.

Rule. A package or app imports another workspace package only if its allow-list in the **layer map** lists it.

- **The file is the authority.** The layer map is `layers.json` at the repo root. For every package and app, it records the layer, the public subpaths, and the allow-list. Every subpath is restricted: an allow-list entry naming the package (`db`) grants its index only, and a subpath is importable only by a package whose allow-list names it (`db/auth`). The allow-lists are the Consumes column of §2.2, which carries each as a `data-consumes` attribute; CI fails if the two disagree. The architect owns the file. A change to what a package may import is an architecture change and needs the architect's sign-off in review; that sign-off is review, not a machine check.
- **Never up, never round.** No allow-list entry points at a higher layer. The allow-lists contain no cycle. Same-layer imports exist only where listed: `auth` → `tenancy` , `securities` → `ledger` , `reports` → `portfolio` , `tax` , `web` → `ui` . Type-only imports count the same as value imports.
- **Standing restrictions.** The specifier lists and the package each restriction exempts are constants in the lint generator (`scripts/layers/generate.mjs`). Each fails closed, so renaming a package applies its restriction to that package too. The one file exemption, the migrate module, is in `layers.json` :
  - Nothing under `packages/` imports SvelteKit (`@sveltejs/kit` , `$app/*` , `$env/*`); `packages/ui` may import `svelte` .
  - Only `packages/db` imports a Postgres driver, and it opens connections only through `withTenant()` / `withSystem()` (§3.3). The restriction names every specifier that yields a connection: `pg` , `pg-pool` , `postgres` , `@neondatabase/serverless` , the `drizzle-orm` Postgres driver subpaths and Kysely's `PostgresDialect` , however imported: outside `db` , `kysely` allows named imports only. A driver type another package needs is re-exported by `db` . BetterAuth's plain `poly_app` pool and `withContextFreeTx` are on the `db/auth` subpath, which only `packages/auth` imports; the routing pool takes its pool and Kysely

dialect from there (§3.1c, POLY-76). Session-level `SET` is refused everywhere except the migrate module in `db`: `SET`, `SET SESSION`, `RESET`, `set_config(…, false)`, and any role switch, `SET LOCAL ROLE` included. Other `SET LOCAL` statements, `SET TRANSACTION`, `SET CONSTRAINTS` and `set_config(…, true)` are transaction-scoped and pass. `layers.json` names that one file by its path, not a glob. Its client is a dedicated connection closed after the run, never pooled (§6.2). The exemption is on the A-39 list that an inline or blanket `eslint-disable` cannot bypass, and `web` and `worker` never import the module: it is exported on the restricted subpath `db/migrate`, which only `apps/migrate`, the migrate entry point, may import (SECURITY C-85, A-40).

- Only `apps/worker` imports `secrets/open`. Only `packages/secrets` imports the HPKE library or calls `createCipheriv`, `createDecipheriv`, `publicEncrypt` or `privateDecrypt`. Elsewhere, `crypto` and `node:crypto` allow named imports only, with `subtle` and `webcrypto` refused. The global `crypto.subtle` may `digest`, `sign` and `verify`, but not `encrypt`, `decrypt`, `wrapKey` or `unwrapKey` (SECURITY §6.3, A-39).
- Only `packages/auth` imports `db/session`, the constructor of the branded `SessionUserId` that `withTenant()` accepts. A cast to `SessionUserId` is refused outside `db/session` and test files, so the brand cannot be skipped.

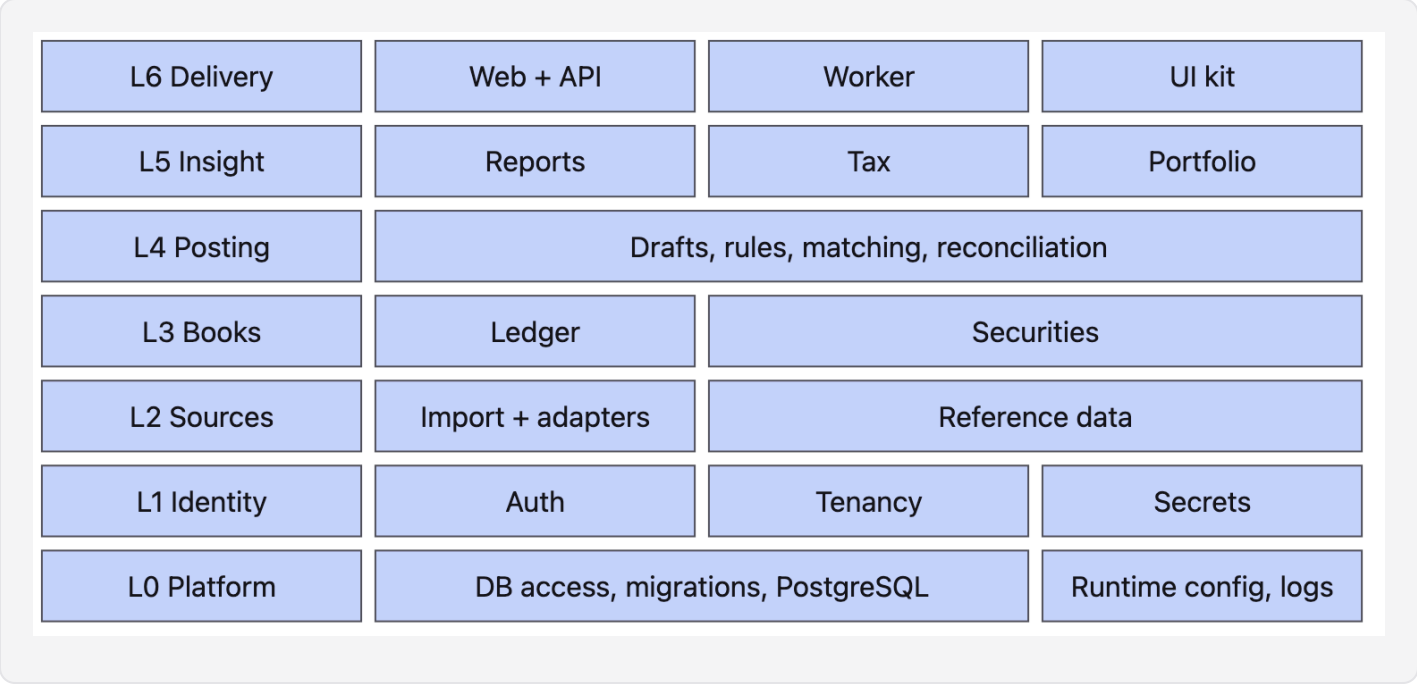
Enforced three ways (built by POLY-82; run in CI's lint job by POLY-96):

1. **Lint.** `eslint-plugin-boundaries`, with its element and allow rules generated from `layers.json`, never hand-copied. It runs in the editor and in CI. `dependency-cruiser` is a fallback only for a rule the plugin proves unable to express.
2. **Resolution.**
   - Each `package.json` has an `exports` map listing only `.` and its declared subpaths. TypeScript resolves with `moduleResolution: "bundler"`, which honours `exports`, so a deep import past the index fails in `tsc`, Vite and Vitest alike. This is also what keeps the raw pool unreachable (§3.3).
   - Each cross-package import is a declared `workspace:` dependency. Under pnpm's strict linking, an undeclared one does not resolve; `import-x/no-extraneous-dependencies` turns that into a readable lint error.
   - Resolution does not guard a restricted subpath. `web` declares `secrets`, and every `db` consumer declares `db`, so `secrets/open` and `db/session` resolve for them. Lint is the only source-side guard there. An inline `eslint-disable` of a boundaries, restricted-import, brand-cast or session-`SET` rule therefore fails lint, and so does a blanket `eslint-disable` that names no rule. Lint sees no import inside one package, so a module-graph check shows the secrets index reaches neither the open module nor the keyring reader

(SECURITY A-39). The real boundary is the keyring mount, not these rules (SECURITY §6.3).

3. **Checks on the file** (CI):
   - Every directory under `packages/` and `apps/` is in the file, and every entry in the file exists.
   - The allow-lists are acyclic and never point up a layer.
   - Each package's declared `workspace:` dependencies are on its allow-list.
   - The file matches §2.2's `data-consumes` lists.

Why the cycle check runs on the file, not in lint. Every cross-package import must be on an allow-list, so the real import graph is a subgraph of the allow-list graph, and a subgraph of an acyclic graph is acyclic. That holds only if lint sees every edge. Re-exports and `import()` with a literal specifier count as edges, and a cross-package `import()` with a computed specifier is refused. The layer order alone already rules out cycles across layers, so the file check matters only within a layer. Cycles between modules inside one package are not a boundary question and are not checked.



Why import sits below the books. Kickoff §2.6 calls import "the lowest non-infrastructure layer". That holds for the *raw* side — adapters, the raw-transaction store and de-dup depend on nothing domain-level, so they sit at L2. Confirmation, rules and transfer matching need both raw rows and the ledger, so they are split into `posting` (L4). This keeps "raw data arrives first" true while dependencies stay strictly downward. The ledger never imports the importer: a journal entry carries an opaque `source_ref`; the two-way link and the posted-status flip happen inside one database transaction owned by `posting`.

2.2 Components

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/db`	Owns the schema, migrations, Postgres roles, and the only path to a connection — a tenant- or system-scoped transaction.	backend-lead	`withTenant(userId, sessionId, fn)`, `withSystem(job, target, fn)`, Drizzle table objects; the `SessionUserId` constructor on the `./session` subpath (`auth` only, §2.1); on the `./auth` subpath (`auth` only): BetterAuth's plain pool, `withContextFreeTx`, and the BetterAuth `fields` map derived from the Drizzle tables (§6.2); the migration runner on `./migrate` (`migrate` only) and the start-up schema guard	PostgreSQL

Component	Responsibility	Owner agent	Exposes	Consumes
			on `./startup` (`web`, `worker`)	
`packages/config`	Reads runtime configuration at start. Every secret comes from the mounted file its `*_FILE` variable names; a missing or empty file refuses start, naming the variable. It also holds the structured JSON log schema and logger, which log no secret, token or raw provider payload field (INFRA §3.2, §9.3; POLY-85). It reads `process.env`, never `$env/*`. It does not read the HPKE keyring, which stays with `packages/secrets`. The status listener and the health route live in `apps/web`, not here.	backend-lead	`loadConfig()`, `readSecretFile(name)`, `logger`, the log schema	environment and mounted secret files; no workspace package
`packages/auth`	Configures BetterAuth (sessions, OAuth via Google and GitHub, MFA: TOTP and passkeys) against our Postgres; maps a request to a user id. Each provider asserts a verified email as C-05 pins it; GitHub reads `/user/emails` with `read:user` and `user:email`. Sign in with Apple is not configured on the web; it ships with native iOS (POLY-122), and its email rules, Hide My Email included (SECURITY C-05), are conditions on that issue (A-37). Keeps `user.second_step_at` in step through the triggers of §3.1c. Owns the routing pool that puts BetterAuth's statements inside the request's `withTenant` transaction (§3.1c, SECURITY A-31), the global `hooks.before`, an allowlist that classifies every `/api/auth/*` route and answers 404 to the rest (`src/route-policy.ts`, §3.1c, SECURITY A-34), and the pinned options (SECURITY A-33): `account.encryptOAuthTokens: true`; a	backend-lead	`getSession(request)`, auth route handler	db (with `./auth` and `./session`), tenancy (`provisionPerson` from the sign-up hook, §3.1),

Component	Responsibility	Owner agent	Exposes	Consumes
	`databaseHooks.account` create and update before-hook that drops `idToken`; `twoFactor({ allowPasswordless: true })`; `skipVerificationOnEnable` unset; no `otpOptions`; `user.deleteUser.enabled` left false; `emailAndPassword.revokeSessionsOnPasswordReset: true` (SECURITY A-38); the `bearer` plugin not installed (native and MCP tokens are our own hashed table, later); `session.freshAge` = 30 days, on condition of the allowlist (§3.1c). BetterAuth runs on its Kysely path over the routing pool; a lint rule refuses `drizzleAdapter` in this package. `better-auth` and `@better-auth/passkey` are pinned to an exact version; an upgrade must pass the routing-pool and route-policy tests (§8).			Resend, OAuth IdPs
`packages/tenancy`	Entities, entity types (base vs parent kind), the many-to-many roll-up junction (`entity_rollup`), memberships, the owner/manager/viewer permission matrix as data, the cap table (`equity_total`, `equity_holding`; §3.1a), co-owner approval and the founding owner (`change_request`, `change_approval`; §3.1b), and the Person-entity-at-sign-up hook (§3.1).	backend-lead	entity/membership service, `can(role, action)`, `capTable(entity, asOf)`, `stake(holder, entity, date)`, `requestChange(entity, kind, payload)`, `decideChange(id, decision)`	db
`packka`	Seals and opens aggregator credentials with HPKE (X25519 / HKDF-SHA256 / AES-256-GCM; SECURITY §6.3). The library is `@hpke/core` with `@hpke/dhkem-`	backen	`seal(plain, aad) → {enc, ciphertext,`	public keys (web);

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/secrets`	`x25519` (proposed; SecEng signs off in the `packages/secrets` design, M2). Lint refuses `hpke-js` and every `@hpke/*` specifier outside this package, so the ban does not wait on that sign-off. `web` only seals, under the public key; only `worker` holds the private keyring and can open.	d-lead (SecEng sign-off)	`key_id, alg_version}` (web); `open(sealed, aad)` (the `./open` subpath, which also builds an opener bound to one connection; only `apps/worker` may import it, §2.1); `Opener` (a type-only export of the index)	keyring file (worker); no workspace package
`packages/import` + `adapters`	Provider-agnostic adapter interface; fetches, normalizes, hashes and stores raw transactions and reported balances.	backend-lead	`ImportAdapter` interface, `ingest(connectionId, range)`	db, secrets (sealing and the type-only `Opener`; `apps/worker` passes `inges`

Component	Responsibility	Owner agent	Exposes	Consumes
				`t` an opener bound to the job's one connection, never `open()`), provider APIs
`packages/refdata`	Global reference series — EOD prices, CPI-U, per-jurisdiction per-year tax tables — behind provider interfaces.	backend-lead	`PriceProvider`, `SeriesProvider`, lookups	db, BLS, price provider
`packages/ledger`	Chart of accounts by nature, custodial accounts and journal entries; posting and reversal; balances as of a date. DB enforces balance and immutability.	backend-lead	`postEntry()`, `correctEntry()`, `balances(asOf)`	db, tenancy

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/securities`	Lots held at custodial accounts, lot matching, quantities, book value adjustments (return of capital, amortization), holdings as of a date. Writes each lot in the same transaction as its entry (§3.6).	backend-lead	lot service, `holdings(asOf)`	db, ledger, refdata
`packages/posting`	Turns raw rows into drafts, applies rules, matches transfers, confirms drafts into entries, reconciles custodial balances.	backend-lead	draft queue, `confirm(draftId)`, rule engine, matcher, reconciliation	db, import, ledger, securities
`packages/reports`	Evaluates declarative report definitions (rows by account range / tag / type, formula lines, period or entity columns).	backend-lead	`run(definition, params) → ReportTable`	db (the roll-up definers, §3.3a, §3.3c), ledger, securities,

Component	Responsibility	Owner agent	Exposes	Consumes
				portfolio, tax
`packages/tax`	Running annualized accrual, safe-harbor derivation, period-cutoff freezes, reporting-period closes, pass-through allocation by the cap table (§4.5), Form 2210 / Schedule AI export.	backend-lead	`estimate(entity, jurisdiction, asOf)`, `freeze(period)`, `close(entity, periodEnd)`, `sinceClose(entity, asOf)`	db, ledger, refdata, tenancy
`packages/portfolio`	NAV, allocation vs. target, inflation adjustment of any time series.	backend-lead	`nav()`, `allocation()`, `holdingsByClass(entity, node, asOf)`, `deflate(series, base)`	db, ledger (book-basis allocation, §2.4), securities, refdata

Component	Responsibility	Owner agent	Exposes	Consumes
`packages/ui`	Shared Svelte components and LayerChart wrappers, once a second Svelte app consumes them. Until then the shell components live in `apps/web/src/lib/shell`, on `bits-ui` primitives, and the token stylesheet in `apps/web/src/app.css`, its `:root` and `.dark` blocks copied from `docs/DESIGN/tokens.css` and held equal to it by a parity test that a change to either file runs.	frontend-lead	Svelte components (and a Swift token export later)	no workspace package
`apps/web`	SvelteKit pages (server `load` calls packages directly) and `/api/v1` thin route wrappers with Zod schemas → OpenAPI.	frontend-lead (pages) / backend-lead (routes)	HTML, `/api/v1`, `openapi.json`	config with `./log`, db (`withTenant`; `./startup`), auth, tenancy, secrets (sealing), import, refdata, ledger, securities, postin

Component	Responsibility	Owner agent	Exposes	Consumes
				g, reports, tax, portfolio, ui
`apps/worker`	Schedules the idempotent job units (import poll, EOD prices, CPI fetch, tax period freeze, tax close prompt; Cash-in-Transit aging is computed on read, not a job).	backend-lead / devops-engineer	none (outbound only)	config with `./log`, db (`./startup`; `withSystem`), secrets with `./open`, and the job units in import, refdata, securities, posting, tax
`app`	The one-off `migrate` entry point of the shared image: `check-config`, `--plan` and the apply run (§8.1).	back	a CLI with the exit codes of	config, db

Component	Responsibility	Owner agent	Exposes	Consumes
`s/migrate`	Logs in as `poly_migrator` only.	end-lead	§8.1	with `./migrate`

2.3 Interface contracts worth pinning now

- **`ImportAdapter`** — `id`; `capabilities` (transactions, balances, holdings, trades); `connect(input) → sealedCredential`; `listAccounts(cred)`; `fetchTransactions(cred, account, range) → RawTxn[]`; `fetchBalances(cred, account) → ReportedBalance[]`; `revoke(cred)` (provider-side revocation, PRD §5). The fake source implements exactly this and is registered as `fake`; switching an instance to SimpleFIN is configuration. A CSV/OFX file adapter implements the same interface with no stored credential (SecEng's preferred credential-free path).
- **`RawTxn`** — provider id, provider account id, posted date, amount (decimal string), currency, payee/memo, provider transaction id (nullable), provider payload (JSON). Before storing the payload, the adapter redacts full account numbers, PANs and any credential fields (SECURITY A-9). The de-dup hash is computed by `import`, never by the adapter: `sha256(provider, provider_account_id, provider_txn_id ?? (date, amount, normalized payee, per-day ordinal))`, enforced by a unique index.
- **`PriceProvider` / `SeriesProvider`** — `fetch(symbolOrSeries, range) → Observation[]` with a `source` and a priority; CPI-U reuses the price-series pattern (PRD §5). Fake EOD provider: seeded random walk per ticker.
- **Job unit** — `run(ctx, params): Promise<Result>`; must be idempotent, short (< 60 s per invocation, chunked otherwise), and triggerable by either scheduler (§7).
- **Screen DTOs** — each page's server `load` returns a DTO type exported by the owning package. The same types back `/api/v1`, so the web UI and native clients share one contract. The list of screens comes from DESIGN (ux-designer); DTOs get pinned per screen in each package's design file (kickoff §2.13).

2.4 Screen → DTO ownership

The sidebar has eight items (DESIGN §5.10): **Dashboard**, **Transactions**, **General Ledger**, **Custodial Accounts**, **Allocation**, **Tax**, **Reports**, **Entities**.

- **No persistent header** (DESIGN). The top of the scroll area is one row: page title and breadcrumbs, the entity selector, and a per-page "more" menu holding that page's global actions. The sidebar's collapse control sits in its brand row; the top row carries a sidebar toggle only on narrow screens, where the sidebar is off-canvas. Settings, Search and Notifications remain surfaces with *proposed* entry points (rows below). The AI/agent placeholder has no v1 affordance.
- **Sidebar foot:** the avatar opens the user popover (profile, MFA, Log out; proposed: Settings and Notifications).
- **Gone:** the earlier Securities and Import screens. Lots now live inside Custodial Accounts, and import setup lives on the provider item.

Screen layouts are in DESIGN.

A screen that spans packages gets a composite DTO assembled in `apps/web` from package DTOs. A panel whose package hasn't shipped yet renders the empty-state pattern.

Screen / surface	DTO(s) → owning package	First milestone
Dashboard	`NetWorthSeries`, `CashFlowByPeriod`, `RecentActivity` → ledger; `TaxPosition` → tax; `AllocationVsTarget` → portfolio	M2 (panels fill in at M3, M4 and 1.0)
Parent views: gross / economic toggle (Dashboard,	`RollupTotals` (gross, §3.3a) and `EconomicRollup` (the same rows and grain, each weighted by the viewer's stake, plus the viewer's economic net worth; §3.3c) → reports	POLY-1.1

Screen / surface	DTO(s) → owning package	First milestone
Reports, Entities on a parent)		
Transactions: three accordion cards (Exceptions, Pending, Posted), each with a "show N" limit	`ExceptionList` (aged CIT, link-health banners) → posting + import; `PendingInbox` (draft with locked custodial leg + suggested offset) and `ImportStatus` → posting + import; `PostedJournal` (paged whole entries, correction badges) → ledger	M2 (CIT exceptions in M3)
Journal-entry modal (header + legs), used for split, group, manual "+ New entry" and "Correct entry"	`EntryDraft` in, `GateResult` out → posting (§3.2a)	M2
General Ledger: chart-	`ChartTree` (category → nature class → custodian sub-account, balances, custodial marker) → ledger	M2

Screen / surface	DTO(s) → owning package	First milestone
of-accounts tree		
General Ledger: account register page	`AccountRegister` (lines on one account, date order, running balance, link to entry) → ledger	M2
Custodial Accounts	`CustodialAccountList` (book, cash, market and unrealized per §3.6, `ReconciliationStatus`, link-health badge) → posting + import; `LotsCard` (per security: lots, short / long / total unrealized gain, per lot and summed) → securities	M3 (lots card at M4)
Provider-item modal ("+ New" on Custodial Accounts)	`ProviderItem`: source type (aggregator / broker API / file / manual), discovered accounts with a mapping to custodial accounts, token refresh, unlink with the revocation-lag note → import	M2 (fake + file); direct, Plaid and SimpleFIN follow per §9
Sync commands (Transactions, Custodial	`SyncRequest` → a job in import; on Custodial Accounts it also pulls balances and positions	M2

Screen / surface	DTO(s) → owning package	First milestone
Accounts) and "Import file"		
Allocation: treemap, Actual vs. target card, target editor, in-place holdings popover	`AllocationVsTarget`, `AllocationTree`, `TargetEdit` (two levels; sub-targets are whole-portfolio percent and must sum to their class) and `HoldingsByClass` (one class or sub-category across every custodial account, grouped by account, with lots and a total) → portfolio. The top level reads the nature accounts across custodials (book) or lots by class (market); sub-categories come from `security_allocation` (§3.6). One `withTenant` query serves a class spanning accounts.	M4
Tax: pending liability, required payment to date, the gap, and "post your payment" (DESIGN)	`TaxPosition` (§4.5) → tax; empty state until then	POLY-1.0
Reports	`ReportTable` (Balance Sheet, Income Statement, Cash Flow; NAV and NAV delta), trade blotter and Form 8949 view (§3.6) → reports; `TaxCloseStatus` ("last closed through *date*") → tax; empty state until then	M5

Screen / surface	DTO(s) → owning package	First milestone
Entities	`EntityList`, `MembershipList` (each member's role in force, `is_founder`, and any held raise with `pending_role` and `pending_since`, §3.1c; for owners only, the pending `change_request` rows as members-list lines with invitee email, role, requester, awaited approvers and expiry, and the count, §3.1b), invite command, `ChangeDecision` and `TransferFounder` commands → tenancy	M1 (pending requests at POLY-1.0)
Entities: cap table, beside the membership list	`CapTable` (total units and holder rows as of a date, each with its access role or none, the outside-owners remainder, and the dated history), `HolderCandidates` (owner-access members first, then the editing user's other Persons, Trusts and Businesses) and `CapTableChange` in (one effective date, a new total and/or holder units; a holder with no access carries an optional viewer invite) → tenancy (§3.1a)	M1 (pass-through allocation reads it at POLY-1.0)
Settings (proposed entry: avatar popover)	`ImportSchedule` (per-entity batch schedule and options) → import; app preferences and notification preferences → tenancy; `ExportReceipt` (time, who, file name, SHA-256, and the chain head `{journal_number, entry_hash, alg_version}`, null for an entity with no posted entry; §3.2.9) → ledger; `SecondStep` (methods with added dates, backup codes left, `required` when the user holds manager or owner outside the own-Person exemption, so the last method has no Remove; `enrolment`: null once enrolled, else `{step`: `password` | `email_code` | `factor`, `sent_to` (masked current address), `code_expires_at, grant_expires_at, resend_available_at}`, the three-step set-up; §3.1c) → auth; `AccountDeletionPlan` (the three groups, transfer candidates with a second step, each purge choice's `ExportReceipt`, and `confirmation`: `second_factor` | `password_and_email` (the latter for a user with no second step, with `sent_to`, `code_expires_at`, `resend_available_at` and the plan's `reauth_at`); §3.4a) → tenancy	M2 (second step and deletion at M1)

Screen / surface	DTO(s) → owning package	First milestone
Entity selector (top row)	`EntityList` (the user's memberships only) → tenancy. The selected entity scopes every screen's DTO request, and RLS still decides what returns.	M1
Search (cross-cutting; proposed entry: keyboard shortcut and the page "more" menu)	`SearchResults` grouped by kind: accounts (name and number), payees and memos on posted entries and pending drafts, and report views → a `search` query service in `apps/web` over ledger, posting and reports. It runs inside `withTenant` over Postgres full-text indexes (`tsvector` on payee, memo and account name), so results are RLS-filtered with no separate index to secure. Viewers see results without masked numbers or raw payloads.	M3 (accounts and entries); report views at M5
Notifications (cross-cutting; proposed entry: avatar popover, with an unread badge on the avatar)	`NotificationList` plus an unread count → a `notification` table (entity-scoped, per recipient, RLS-covered). Rows are written by the producers of the alerting events in SECURITY: link health (`token_expired` , `principal_invalid` , `stale`), aged Cash-in-Transit items, an auto-post rule switched off, membership changes, a purge pending, failed import batches, tax due dates, and a tax close due. Delivery is in-app; email via Resend follows the user's preferences.	M2 (import and link health); others as their producers land
AI / agent (v2)	None in v1, and no v1 affordance. It is a placeholder for the future user-scoped MCP / agent surface (PRD §3 non-goal), which would be a client of `/api/v1` .	v2 placeholder

Model consequence: import status on Transactions comes from an `import_batch` row per ingest run, holding status and counts. Rows skipped by `ON CONFLICT DO NOTHING` are counted into the batch as duplicates instead of vanishing silently (§3.1).

Permissions (mirrors the SECURITY §4.2.1 matrix @ 6fb5385):

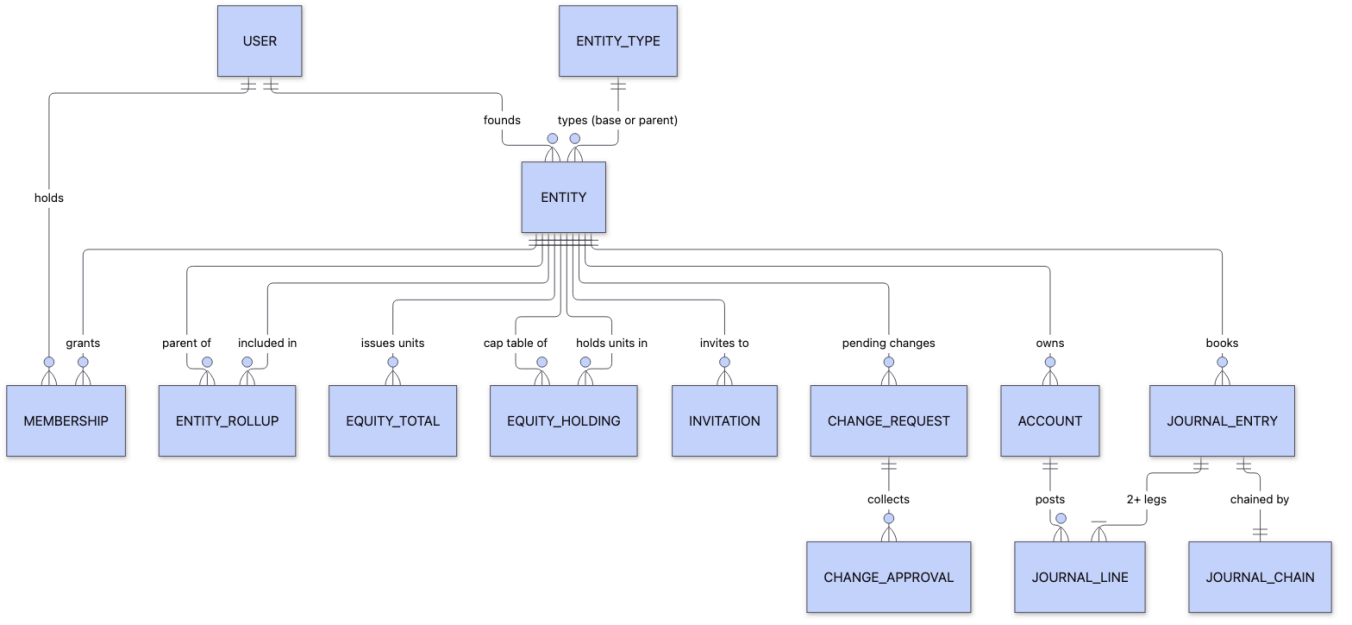
Action	Viewer	Manager	Owner
New provider item, map accounts, refresh token, unlink. Managers act on their own links only, with step-up re-auth; a mapping change writes an `audit_event`.	—	✓	✓
Sync, Import file. Sync enqueues a worker job, and the credential is never opened in web. Rate-limited to the INFRA minimum interval.	—	✓	✓
Post, Split, Group, + New entry, Correct entry	—	✓	✓
Rule auto-post opt-in. `rule.auto_post_by_user_id` is the principal.	—	✓	✓
Settings: per-entity import schedule, floored at the INFRA minimum interval	—	✓	✓
Export this entity (step-up; the receipt carries the ledger fingerprint, §3.2.9)	—	—	✓
Edit the cap table (total units, holders, effective dates). It is entity metadata, like the tax classification; every change notifies the holders it touches (§3.1a).	—	—	✓
Read every screen above, the cap table included (account numbers masked to last 4; no raw payloads)	✓	✓	✓

Viewers see none of the action buttons. The server enforces every row regardless of what the UI shows.

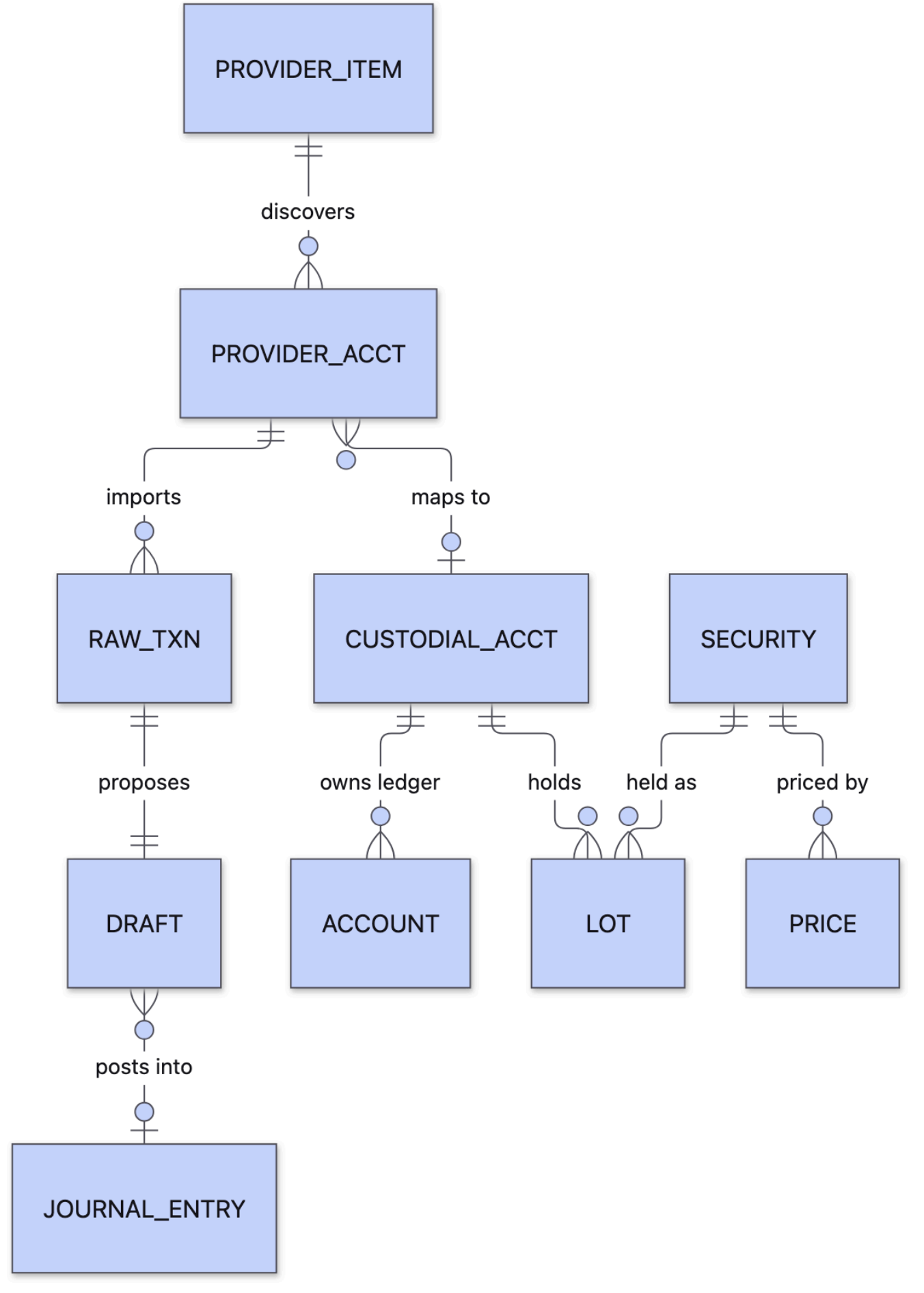
3. Data Model & Database Invariants

Logical model at ARCH grain; column-level design lives in each package's design file.

3.1 Core entities



Tenancy, the cap table and the ledger, above. Access (`MEMBERSHIP`, user to entity) and equity (`EQUITY_HOLDING`, entity to entity) are separate edges. On a multi-owner entity, changes to either pass through `CHANGE_REQUEST` (§3.1b). Import and securities, below. The table that follows lists every table group.


PROVIDER_ITEM
discovers
PROVIDER_ACCT
imports
maps to
RAW_TXN
CUSTODIAL_ACCT
SECURITY
proposes
owns ledger
holds
held as
priced by
DRAFT
ACCOUNT
LOT
PRICE
posts into
JOURNAL_ENTRY

Keys: every tenant-visible primary key is a UUID, never a sequential integer, so ids leak neither counts nor ordering across tenants (SECURITY A-6). Tenancy columns (`entity_id`) and actor columns are not updatable.

Table group	Tenant key	Mutability	Notes
`user` (BetterAuth)	self	mutable; deletion tombstones	`second_step_at` (nullable) marks a verified second-step method; it gates holding manager or owner (§3.1c). Account deletion turns the user row into a tombstone ("deleted user") and removes credentials, sessions and memberships. The row itself is never deleted, and entries are never UPDATEd. `actor_id` FKs are `ON DELETE NO ACTION`, and BetterAuth's `deleteUser` is left disabled (its endpoint answers 404, measured on 1.7.18); our own route does the deletion (§3.4a; SECURITY A-8, A-33, C-52). Session tokens are stored in the clear in `session.token`; the cookie is the token signed with the auth secret (SECURITY R-12, measured). The tombstone transaction refuses while the user is still the founding owner of any entity that has another owner (§3.1b). After the bootstrap user, an invitation is the only way to create a user, and no configuration flag reopens sign-up.
`email_challenge`, `enrolment_grant`	self	written only by `poly_governer` definers	The emailed code and the grant it opens for a user with no second step (§3.1c, SECURITY A-29, A-30). Code: purpose `first_enrolment` | `account_deletion`, hashed, bound to the session, 10 minutes, single use. Grant: one live per user, bound to the session, 10 minutes, consumed by the first factor write, recording its transaction id. The code is removed with the session; the grant's `session_id` is set null instead, so a consumed grant outlives the session BetterAuth replaces (A-32).
`entity`	entity	mutable	Five system types in two kinds. Any entity may hold accounts and books: a Household's joint checking account lives in the Household. *Base* (Person,

Table group	Tenant key	Mutability	Notes
`y`, `membership`, `entity_type`	y id; types: `owner_user_id` or null (system)	e; tenancy columns not updatable	Trust, Business) never have children. *Parent* (Household, Holding) also roll up whole base entities and other parents, never individual accounts of theirs. Holding is a generic umbrella, not a household. `entity_type.kind` is `base` | `parent`, and a user-defined type declares its kind. **Every user gets a Person entity and an owner membership at account creation**: BetterAuth's `user.create.after` hook calls `tenancy.provisionPerson(userId)`, which inserts the entity (with `self_user_id` set, marking it as that user's own Person; §3.1a), the owner membership and the seeded accounts. That hook does *not* share the user insert's transaction (measured on POLY-76, better-auth 1.7.18), so **every auth route that can insert a `user` row runs BetterAuth inside a context-free transaction of ours**, `withContextFreeTx` from `db/auth`. The user row, the Person, the owner membership and the seeded accounts then commit together or not at all; a throw anywhere in provisioning leaves no user row. Inside that transaction the hook first takes the *new-user step*, which sets `app.user_id` to the new user's id, because the tenancy tables are under FORCE RLS and a context-free transaction has no user set; only then does it call `provisionPerson`. The step is a second way to assert an identity, beside `SessionUserId`, so it is narrow (SECURITY C-84, T-90, A-40): • It is a method on the transaction handle that `withContextFreeTx` passes in, and is not exported from the `db/auth` index. • It accepts only a `user` row inserted by the current transaction, and the database checks that, not the caller. Row `xmin` cannot be used: BetterAuth's insert runs inside a savepoint (§3.1c), so the row carries a subtransaction id. Instead an `AFTER INSERT` trigger on `user` records the id in a transaction-local setting, and the step's definer function compares against it. A savepoint rollback discards the record with the row (unmeasured). `poly_app` can forge a transaction-local setting with `set_config` (SECURITY R-2), so the step also refuses an id with no `user` row, or one that already has its own Person (`entity.self_user_id` = id). Its definer function is owned by a C-10 NOLOGIN role, with `search_path = ''`, and is on the C-15 allowlist. Lint

Table group	Tenant key	Mutability	Notes

refuses the setting's literal name anywhere in the source; it appears only in the trigger's migration.

- It refuses a second call in the same transaction.
- It uses `set_config(…, true)` only, never a session `SET`.

The step is unmeasured until POLY-87. The provisioning stays idempotent, and the session loader still re-runs it for any user holding no owner membership on a Person, as a backstop only, so no signed-in user is ever without one (this is why the entity selector is always present). `membership.role` is the role in force; `pending_role`, `pending_since`, `pending_by_user_id` and `pending_change_id` hold a raise waiting for the member's second step (§3.1c). **Membership is written only by definers.** `poly_app` holds no INSERT on `membership`: a direct insert would skip the invitee's consent and the second-step gate. The invitation-accept definer seats the accepting user against a live invitation. An AFTER INSERT trigger on `entity`, `app.seat_creator()` (a `poly_govern` definer), inserts the owner membership for `app.current_user_id()` in the same statement and refuses when no user is set, so an entity never commits without an owner. **The direct path while sole owner** (§3.1b, SECURITY C-28): `app.gate_owner_changes()`, BEFORE UPDATE OR DELETE on `membership` and BEFORE INSERT on `invitation`, refuses for `poly_app` an invitation, a role raise, or a change to another owner's row unless `app.owner_count` is 1; removing or demoting a viewer or manager and lowering one's own role stay direct. `provisionPerson(tx)` acts for the transaction's user: it inserts that user's own Person under that trigger unless one exists. It uses no `ON CONFLICT`, because the arbiter makes the insert check the SELECT policy, which cannot yet see the trigger's membership. A concurrent first call fails on the unique `self_user_id` (23505), and the caller retries, and the session loader in `packages/auth` calls it. **At least one owner** (SECURITY C-20): `app.keep_an_owner()`, BEFORE UPDATE OR DELETE on `membership`, refuses removing or demoting an owner row while `app.owner_count` is 1. Both guards lock the entity row first and refuse when the lock returns no row. `app.owner_count` returns null to a caller with no membership on the entity. Tax-classification attribute on `entity`, settable on every entity (individual;

Table group	Tenant key	Mutability	Notes
			trust grantor / non-grantor; business pass-through / C-corp; for a parent, `joint filers` or `pass-through to members`, the default for a new Household and, by extension, a new Holding; §4.5). Stored values: `individual`, `trust_grantor`, `trust_non_grantor`, `business_pass_through`, `business_c_corp`, `parent_joint_filers`, `parent_pass_through`. A trigger admits only the values of the entity's kind and, for a system type, its family. A Person defaults to `individual` and a parent kind to `parent_pass_through`; a Trust or Business states its own. User-defined types readable by creator plus holders of any membership on an entity of that type — the narrow cross-tenant metadata exception SecEng flagged; the read is label-only (no column grant on `owner_user_id`, SECURITY R-9), and an entity may use only a system type or one its creator owns. **`founding_owner_user_id`** NOT NULL, set to the creator (for a Person, its own user), transferable only through `app.transfer_founder`; a deferred constraint trigger refuses any commit where the founder does not hold owner (§3.1b, SECURITY A-22). **`purged_at`**: a purged entity's row stays as an id-only tombstone, so cap-table rows on other entities still resolve (§3.4b).
`invitation`	entity id (null for an operator invite)	insert; `accepted_at` / `revoked_at` only	A live invitation: email, role, inviter, single-use token hash, expiry, and `issued_via` (`direct`, or the id of the `change_request` that applied it), not updatable (SECURITY A-24). A revoked token is refused at acceptance. On a multi-owner entity the row is written only when its approved `change_request` applies, and the email goes out after that commit; until then the invite exists only as the request's payload, visible to owners alone (§3.1b). Inserting a row on a parent-kind entity fires the ancestor-invite notice (§3.3b). The operator command `polycarpic user invite <email>` writes a row with no entity: it shares nothing, is audited, and the invitee gets only their own Person, so neither the approval path nor the notice applies.

Table group	Tenant key	Mutability	Notes
`change_request`, `change_approval`	entity id	append-only except `change_request.status` (trigger)	Co-owner approval (§3.1b, SECURITY §4.2.2, A-20). Request: entity, `kind` (`invite` | `role_raise` | `owner_remove` | `owner_demote` | `cap_table`), target user (nullable), payload, the entity version it was made against, requester, created, expiry, status (`pending` | `applied` | `declined` | `expired` | `withdrawn`), `forced` . Approval: request, approver, decision (`approve` | `decline`), time. Readable by the entity's owners only. Status flips only through `poly_govern` definers. Purged with the entity; their `audit_event` s survive.

Table group	Tenant key	Mutability	Notes
`entity_rollup` (the parent link)	parent entity id	insert / delete only; no UPDATE	Many-to-many: `(parent_entity_id, child_entity_id)`, both FK to `entity` `ON DELETE CASCADE`, primary key on the pair, `CHECK (parent_entity_id <> child_entity_id)`. A trigger requires the parent's type to be of kind `parent`; the child may be of either kind, so parents nest to any depth. A base entity never has children. Any entity may sit in any number of parents. The graph stays acyclic and within the sanity limit by the trigger in §3.3b. **Insert** requires owner on both the child and the parent (SECURITY A-11 / C-19; it is what stops T-16 injection). **Delete** requires owner on the parent or on the child. **Read** iff the user holds any membership on the *parent*, or owner on the *child*. A child's owner gets the parents' names and nothing else about them, through `app.parents_of(entity_id)`, which lists every parent above with the direct ones marked. It is owned by `poly_rollup` and reads under that role's second predicate, the ancestors of entities the caller owns, and sits on the definer allowlist (SECURITY A-17). The row never grants a row of the child (§3.3a).
`ledger_account`	entity id	mutable until first posting; then name/parent only	Number, name, parent, class (asset / liability / equity / income / expense), normal side, currency, `nature_class` (nullable; §3.6), `custodial_account_id` (nullable), default tax-treatment tag. Seeded per entity: the four nature classes under Assets, Cash in Transit, Suspense, Opening Balance Equity, realized-gain income accounts by holding period (§3.6), and the per-jurisdiction tax accounts listed in §4.5. Prepaid-tax accounts belong to a `custodial_account` for the tax authority (IRS, the state).

Table group	Tenant key	Mutability	Notes
`entity_journal_counter`	entity id	one row per entity; written only by the trigger below	Holds `last_number` for the Journal ID (§3.2.7) and the chain head, `head_number` and `head_hash` (§3.2.9). Created at number 0 with the fixed seed by a `poly_journal` AFTER INSERT trigger on `entity`, the only counter INSERT, so no caller can pre-seed a head; purged with the entity. `poly_app` and `poly_worker` hold no INSERT or UPDATE on it and read it through tenant RLS, viewer and up; `poly_journal` may UPDATE only `last_number`, `head_number` and `head_hash` (SECURITY A-23).
`journal_chain`	entity id	**append-only** (trigger)	One row per posted entry: `(entity_id, journal_number, entry_id, prev_hash, entry_hash, alg_version)`, unique on the first two. Written only by the deferred chain trigger (§3.2.9); the entry itself is never UPDATEd to hold its hash. Readable by the entity's members. Purged with the entity; the final head goes into `entity.purged` (SECURITY C-24).
`custodial_account`	entity id	mutable; tenancy columns	A real account at an institution: institution, account type, number mask, currency, provider link. It owns ledger accounts one-to-many, and lots point at it (§3.6).

Table group	Tenant key	Mutability	Notes
		not updatable	
`journal_entry`, `journal_line`	entity id (denormalized onto the line)	**append-only** (trigger)	Entry: `journal_number` (per entity, gapless; §3.2.7), posting date (exactly one), description, `actor_kind` (user / rule / system) + `actor_id`, `reverses_entry_id`, `replaces_entry_id`, `source_ref`. Line: account, `currency`, `native_amount`, `base_amount`, `fx_rate`, category, tax-treatment tag, memo. Amounts are `numeric`, signed (debit > 0); never JS floats.
`import_batch`, `raw_transaction`, `re`	entity id (via linked acco un	raw rows insert-only; `status` draft	Unique de-dup hash. Full provider payload kept for audit; `payload` is readable by manager+ only, never by viewers (SECURITY §4.2.1). Batch holds run status and new / duplicate / needs-category counts (§2.4).

Table group	Tenant key	Mutability	Notes
`ported_balance`, `reported_position`	t, denormalized)	→ posted (then frozen by trigger); batch mutable while running	
`draft`, `rule`, `transfer_match`	entity id	mutable until confirmed	One draft per raw row. Several drafts post into one entry when the user groups them (Flow B), so draft → entry is many-to-one. A split (Flow A) is one draft posting into one multi-leg entry. A posted draft carries exactly one `entry_id`. The draft carries the proposed lines (JSON), with the custodial cash leg locked (§3.2a), and an actor. Rule carries `auto_post` opt-in (default false). Opting in counts as the user's confirmation. `auto_post_by_user_id NOT NULL` records who opted in. If that user drops below manager, `auto_post` flips off and an `audit_event` is written (SECURITY §4.2.1). An auto-posted row is immutable like any other, and "Correct entry" applies to it.

Table group	Tenant key	Mutability	Notes
`provider_account`	entity id	mutable	One account discovered on a provider item (a Fidelity brokerage, a Schwab checking account). It carries a `tracked` flag and a nullable mapping to a `custodial_account`, never to a ledger account, set in the modal's mapping step. Raw transactions, reported balances and reported positions hang off it.
`provider_connection` (the UI's "provider item")	entity id + linking user	mutable	The first-class record for one provider login. It owns the discovered `provider_account` rows. Source type: `aggregator` | `broker_api` | `file` | `manual`. File and manual items hold no credential. Link health is `ok` | `token_expired` | `principal_invalid` | `stale`. Sealed credential only: `enc`, `ciphertext`, `key_id`, `alg_version`, with AAD = (connection, entity, provider, key_id, alg_version) (SECURITY §6.3). `poly_app` may INSERT and UPDATE those columns but never SELECT them; only `poly_worker` reads them. `linked_by_user_id NOT NULL` is the principal that job writes run under. A `principal_invalid` state is set when that user's membership is revoked or their account is deleted; it surfaces on the dashboard and writes an `audit_event`, so it never fails silently (SECURITY §4.4). On unlink, `web` enqueues a revoke job; `worker` opens the credential, revokes at the provider, then deletes the row.
`security`, `price_observat`	global reference data	writable by system jobs / ope	Price: unique (security, date, source) + source priority (borrowed from mosko-fintech). Tax table: jurisdiction × filing year → due dates, cutoffs, multipliers, cumulative percentages — a new year is a data row.

Table group	Tenant key	Mutability	Notes
`ion`, `cpi_observation`, `tax_table`		orator import only	

Table group	Tenant key	Mutability	Notes
`allocation_node`, `security_allocation`, `allocation_target`	entity id	mutable	The allocation hierarchy (tier, class, sub-category), seeded per entity from the DESIGN allocation classes. A tier is a label; targets sit on class and sub-category nodes as whole-portfolio percent. The four classes are the nature classes of the chart (§3.6), so they are fixed; sub-categories are editable. `security_allocation` maps a security to a sub-category for that entity, since `security` itself is global, and its parent class is the security's nature class. A reported holding with no mapping shows in Allocation as a visible gap, never silently excluded.
`lot`, `lot_match`, `lot_adjustm`	entity id	append-only	A lot references `custodial_account_id` and the security, never a ledger account. Lot-match is an append-only many-to-many junction with a `match_seq`; the latest batch is current (borrowed from mosko-fintech 032). It stores proceeds and per-lot basis as values, and a `kind` of `sale` or `transfer` (§3.6).

Table group	Tenant key	Mutability	Notes
`ent`			
`tax_period_freeze`	entity id	append-only	Annualized income, tax, cumulative required, paid — per jurisdiction per period; source of the 2210 / Schedule AI export.
`tax_close`	entity id	append-only	One row per reporting-period close: jurisdiction, filing year, `period_end`, revision, `accrued_to_date`, `entry_id`, unique on the first four. It is the record of what has been accrued for a filing year (§4.5). The cadence is `entity.tax_close_cadence`: `monthly`, the default, or `quarterly`.
`equity_total`, `equity_holding` (the cap table)	entity id	**append-only** (trigger)	`equity_total`: `(entity_id, total_units, effective_from)`. `equity_holding`: `(entity_id, holder_entity_id, units, effective_from)`. Holders are entities of kind Person, Trust or Business, never users and never parents. Independent of `membership`: any role or none may hold units. `holder_entity_id` FKs `entity` `ON DELETE NO ACTION`; a purged holder stays as a tombstone, and `equity_holder_label` carries the owners' optional label for it (§3.4b). Rules, reads and triggers in §3.1a; on a multi-owner entity every edit is a `change_request` (§3.1b).

Table group	Tenant key	Mutability	Notes
`report_definition`	owner user id (clones) / null (system templates)	system templates read-only; clones mutable	Upgrades replace system templates in place; clones are never touched.
`attachment`, `attachment_blob`,	entity id	attachment insert-only; links del	Supporting documents such as statements and receipts, stored as `bytea` in Postgres (see §3.5). They are RLS-covered, backed up, exported and purged along with everything else.

Table group	Tenant key	Mutability	Notes
`entry_attachment`		etable by manager+ (audited)	
`notification`	entity id + recipient user id	insert-only, except `read_at`	One row per recipient per alert event (§2.4): among them change requested, reminder, approved, declined, expired, forced, founder transferred, role raise waiting, held raise cancelled, placement, ancestor invite, cap-table holder, former holder, chain break. Producers write it in the same transaction as the event, through their own definers; `poly_app` holds no INSERT. A recipient reads their own rows and sets `read_at`, nothing else. Purged with the entity, and deleted with the recipient's account.
`audit_event`	entity id (nullable after pu	append-only, outside purge cas	Records purges, exports, revocations, membership changes, and `principal_invalid` transitions; survives the purge it describes. Readable by owners only, and by the actor of the event (SECURITY §4.2.1). Written only by definer triggers (membership changes: `app.audit_membership()`, `poly_govern`); `poly_app` holds no INSERT, and a BEFORE UPDATE OR DELETE trigger refuses every change, whoever runs it.

Table group	Tenant key	Mutability	Notes
	rge)	cade	

3.1a The cap table: equity, separate from access

Every entity carries a cap table; equity is independent of the access roles.

Membership answers "who may do what"; the cap table answers "who owns how much". A 0-unit owner is a bookkeeper; a 25% holder may have no access at all. The two never derive from each other.

- **Shape.** Two append-only tables, both effective-dated. `equity_total(entity_id, total_units, effective_from)` holds the unit count; `equity_holding(entity_id, holder_entity_id, units, effective_from)` holds each holder's count. Units are `numeric`, `units ≥ 0`, `total_units > 0`. The total is dated, not a column on `entity`, because a past allocation must use the total in force on its date: issuing units in June must not rewrite January's split.
- **The split on date *d*.** The latest `equity_total` row with `effective_from ≤ d`, and for each holder its latest `equity_holding` row with `effective_from ≤ d`; a holder at 0 units has left. Stake = units / total. The remainder, total − Σ units, is **outside owners**: real holders the system does not track, whose share allocates to no one.
- **Corrections are new rows.** Nothing is updated or deleted. A change is a set of rows sharing one `effective_from`; each row carries `recorded_at`, and for one holder and one date the latest recorded row wins. A change entered from the wrong date is undone by a row at that date restoring the earlier units, plus the row at the right date.
- **A Person's table is implicit:** itself at all units. No rows are stored for a Person entity, and a trigger refuses any. A parent (Household, Holding) carries a cap table like any entity, and a parent is never a holder.
- **Defaults.** A new non-Person entity is created with 100 units and the creating user's own Person holding all 100, from the creation date; the creation form shows the table so the user can change it before saving. Every user's own Person is marked by `entity.self_user_id` (unique, nullable, set by `provisionPerson`; §3.1).

- **Invariant.** Σ units ≤ total units at every date. A `DEFERRABLE INITIALLY DEFERRED` constraint trigger checks it at commit, at every change point from the earliest `effective_from` written in the transaction onward, so a new total and new holder rows commit together.
- **Who writes:** owner of `entity_id` only (it is entity metadata). While the entity has one owner the write is direct; with more than one, every edit, the entity-creation default excepted, is a `cap_table` `change_request` that applies once every other owner approves, re-validated against the Σ invariant at apply (§3.1b). **Who may be a holder:** a Person, Trust or Business that is either visible to the editing user (any membership) or the own Person of a user holding a membership on this entity. The editor lists owner-access members first, then the other members, then "add holder" over the editing user's other entities. Anyone else is reached by inviting them as a member first, or left in the outside-owners remainder. **"Invite as viewer" appears in both places** (settled for ux-designer, PR #78): in the save that adds a Person holder whose user has no access, and afterwards on any saved row whose Person holder's user still has none, so a declined or skipped offer can be taken up later. The invite is its own action, never bundled into the cap-table write: on a sole-owner entity it is an ordinary invitation insert in the same transaction; on a multi-owner entity it is a separate `invite` request raised beside the `cap_table` request, and either may apply or fail without the other. An entity holder gets no offer, since membership belongs to users.
- **Who reads:** any member of the entity, by ordinary RLS on `entity_id`. The `equity_*` SELECT policies are not widened for anyone else. Everything beyond that goes through definers owned by the NOLOGIN, NOBYPASSRLS role `poly_stake`, all on the allowlist (SECURITY A-18, C-27):
  - `app.my_stakes()` is a holder's owners' read path. It returns entity id, display name, units, total units and effective-from for every entity in which the caller's own Person, or an entity the caller owns, holds units. Never another holder's row.
  - `app.cap_table_candidates(entity_id)`, for an owner of `entity_id` only: id and display name of members' own Persons and of Person, Trust and Business entities visible to the caller.
  - `app.allocated_income` (§4.5).

  `poly_stake`'s role-targeted SELECT policies are confined to `app.held_entities()`: entities in which the caller's own Person, or an entity the caller owns, has ever held units, plus one link onward; candidates also read members' own Persons.
- **Notice.** Adding or changing a holder writes a `notification` in the same transaction: to the user whose own Person it is, or, for a Trust or Business holder, to each of its owners. The editor may not see those owners, so the write is done by the definer trigger `app.check_equity_holding()`, owned by `poly_graph` (§3.3b).

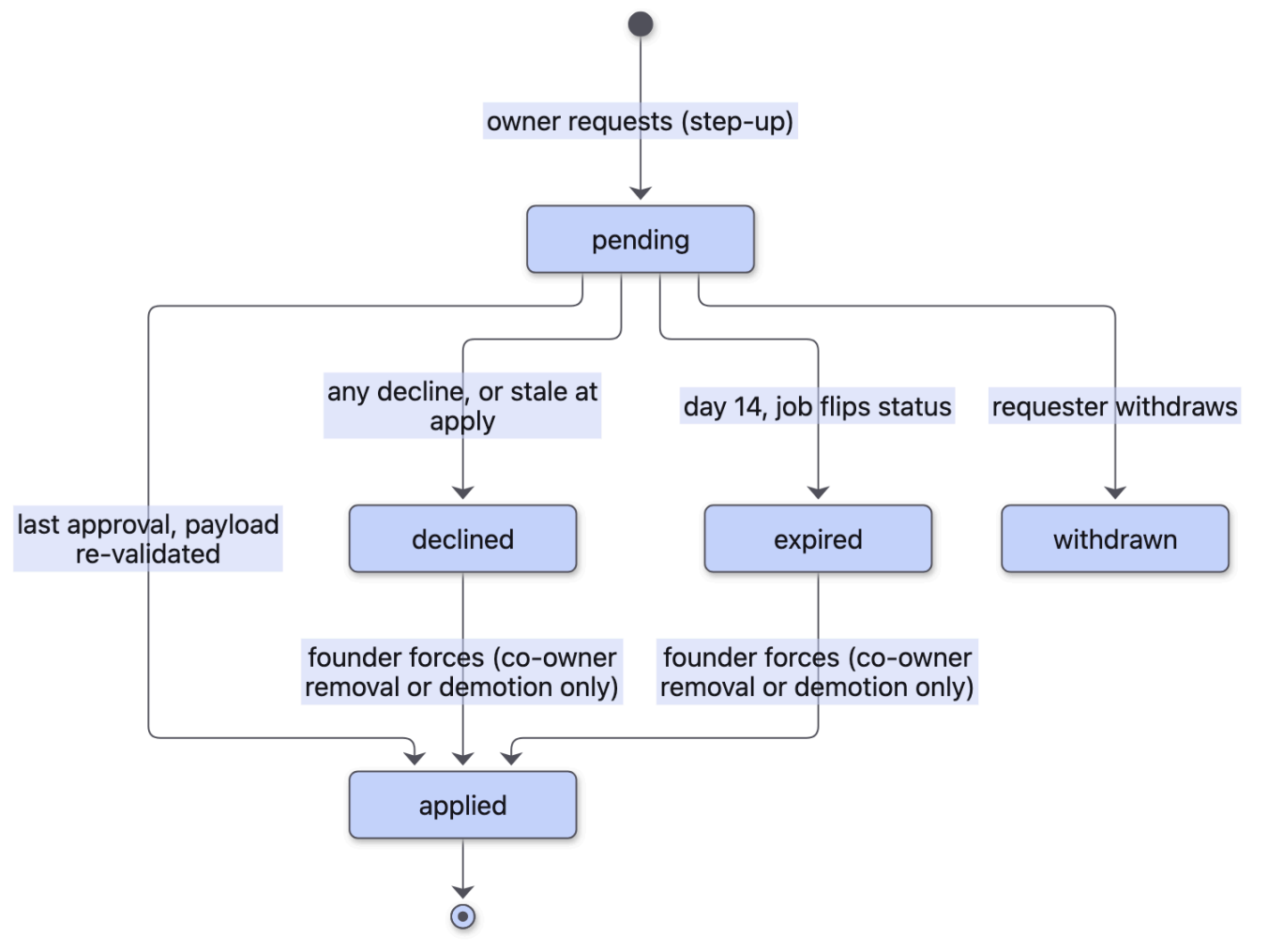
- **Cycles refused.** The holder graph (entity → holder) must stay acyclic: an entity never holds itself through any chain. `app.check_equity_holding()` runs BEFORE INSERT, `SECURITY DEFINER` owned by `poly_graph` (graph-wide read; SECURITY A-16), `search_path = ''`, under `pg_advisory_xact_lock` on its own fixed key, the same mechanism as the parent-link trigger. It also refuses a new row naming a tombstoned holder (§3.4b). An edge is any pair that has ever had a row with units > 0, whatever the date, so the check needs no reasoning about date ranges. It is stricter than needed, and the cost is that a long-exited holding still blocks the reverse one.
- **Depth: one entity link in v1, deeper chains refused.** An entity may be held by an entity whose own holders are Persons (an LLC held by a trust held by its beneficiaries), never by an entity that is itself held by an entity. The same trigger refuses a row that would make an entity-to-entity chain two links long, from either end. Refusal, rather than treating the deeper share as outside, means no income ever silently drops out of tax (§4.5). The cost is that a holding company inside a trust cannot be modelled in v1 (unmeasured; no structure described so far needs it). Raising it is one migration and a composition change in §3.3c and §4.5. Errors name no hidden entity, as in §3.3b.
- **A holder that is purged** becomes a former holder: its rows stay, shown as "Former holder (purged <date>)" with an optional label the entity's owners type themselves (a cap-table edit), and its units count as outside owners from the purge date (§3.4b).
- **Tests (pgTAP):** Σ over total refused at a later change point; a Person row refused; a direct and an indirect cycle refused, including through an entity the editor cannot see; a two-link chain refused; a non-owner write refused; `app.my_stakes` returns only the caller's holder rows, and a holder's owner reads 0 rows from `equity_holding` directly; a non-owner gets 0 candidates; a notification lands for an owner invisible to the editor; with two owners, a direct `equity_holding` INSERT is refused.

3.1b Co-owner approval and the founding owner

R-19; SECURITY §4.2.2, C-20, C-28; A-20, A-22. Co-trustees and partners govern as a group, and owner is an access role, not a stake.

What needs approval. On an entity with more than one owner, these take effect only once every other current owner approves: (a) inviting a member or raising a role; (b) removing or demoting a co-owner; (c) any cap-table edit. Removing a viewer or manager, and lowering or giving up one's own role, stay single-owner actions. A sole-owner entity is unaffected.

Action	Sole owner	Multi-owner	Founder extra
Invite a member, any role	direct	`invite` request; email after apply	none
Raise a role	direct	`role_raise` request	none
Add the first co-owner (invite or raise)	direct: nobody to approve	n/a	none
Remove or demote a co-owner	n/a	`owner_remove` / `owner_demote` request; the target votes	force after decline or expiry
Remove a viewer or manager; lower or give up one's own role	direct	direct	must transfer founder before giving up owner
Cap-table edit	direct	`cap_table` request	none
Transfer the founder role	n/a (a Person's founder is its user)	founder only, direct, step-up, every owner notified	the founder's own act



A forced apply sets `forced = true` and writes `change.forced`. The day-7 reminder is a notification, not a state.

- **The direct path is gated in the database.** A BEFORE INSERT / UPDATE / DELETE trigger on `membership`, `invitation`, `equity_total`, `equity_holding` and `equity_holder_label` raises when `current_user = 'poly_app'`, the action is one of (a) to (c), and `app.owner_count(entity_id) > 1`. A trigger, not a policy, because telling "remove a viewer" from "remove a co-owner", or "lower my own role" from "demote someone", needs `OLD`. `app.owner_count` is a `poly_authz` definer, and the trigger takes the entity row `FOR SHARE` so the count cannot change under it. A `can()`-only gate in the app fails the §4.2 design test.
- **The multi-owner path.** `poly_app` INSERTs a `change_request` (owner of the entity, step-up within 5 minutes) and `change_approval` rows (an approver must be a current owner other than the requester). Every status change and every resulting write happens in a `poly_govern` definer:

  - `app.apply_change(change_id)`, called after each decision. One decline flips the request to `declined`. Otherwise it recomputes the approvers as every current owner other than the requester, counts only approvals by current owners, and applies when none is missing. Before applying it re-validates the payload: the role still exists, the cap-table Σ invariant holds, and the entity version is unchanged or no conflicting change has applied since. A failed check declines the request as stale and nothing applies. It runs in the last approver's transaction. An invitation row is written only here, and the route sends the email after commit.
  - `app.withdraw_change(change_id)`, requester only.
  - `app.expire_changes()`, executable by `poly_worker` only, run daily: flips `pending` past 14 days to `expired` and writes the day-7 reminder notification once per request. It changes nothing but status and notifications.
  - `app.force_change(change_id, reauth_at)`: caller is the founder; kind is `owner_remove` or `owner_demote`; status is `declined` or `expired`; the target is not the founder; second-factor step-up within 5 minutes. Re-validates as apply does, writes `change.forced`, notifies every owner, the target included. The founder may have raised the request; the force still waits for the decline or expiry, so the target always has warning.
  - `app.transfer_founder(entity_id, to_user_id, reauth_at)`: caller is the founder; target a current owner; step-up; audit and notice to every owner. Stepping down is this call.
- **`poly_govern`** is NOLOGIN, NOBYPASSRLS, owns no tables, and holds role-targeted INSERT / UPDATE / DELETE on `membership` and `invitation`, INSERT on `equity_total` and `equity_holding`, UPDATE of `status` and `forced` on `change_request`, UPDATE of `founding_owner_user_id` on `entity`, INSERT on `notification` and `audit_event`, and SELECT on the rows those need. Its policies are keyed to the request's entity through a transaction-local GUC the definer sets itself, as `poly_purge` does (§3.4).
- **A raise waits for the second step.** An approved `role_raise` for a member without one is `applied` here and held in `membership` until they enrol (§3.1c).
- **Limits on requests.** One open request per kind and target. An expired request may be re-raised at most once per 7 days. A new owner becomes a required approver of every pending request; a departed owner drops out.
- **Invitations issued while sole owner.** An invitation the sole owner sent directly may still be outstanding when a second owner joins. When the effective owner count goes from 1 to 2, every other outstanding invitation on that entity with `issued_via = 'direct'`, any role, is revoked and its inviter notified to re-raise it as a request; otherwise a sole owner could pre-issue owner invites that land after the approval rule applies. An invitation applied through `apply_change` is not revoked. The revocation runs in the transaction of the membership write that makes the second owner, an invite acceptance, a direct raise or a held role's promotion

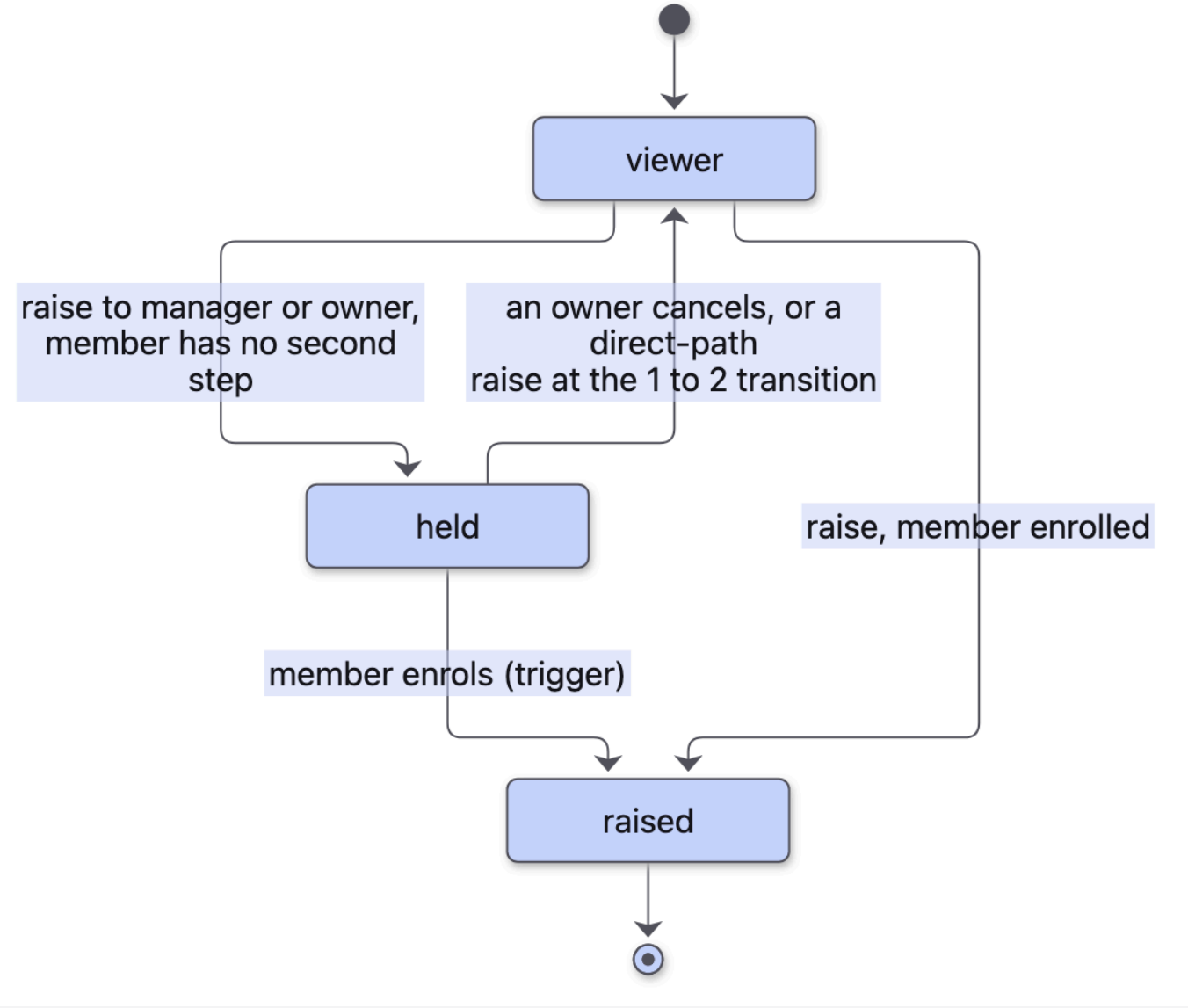
(§3.1c), after a `FOR UPDATE` lock on the entity row taken before `app.owner_count` is read, so two direct owner invitations accepted concurrently end with exactly two owners and the other revoked. It counts effective owners only: a held raise does not trigger it (SECURITY A-24, C-28).

- **The founding owner.** `entity.founding_owner_user_id` is set to the creator; a deferred constraint trigger refuses any commit where the founder does not hold owner, which is also what forces a transfer before the founder gives up owner. The founder has no extra power over invites, role raises or cap-table edits, and is never a `force_change` target. Others cannot remove a hostile founder in-app (accepted risk). A founder who vanishes is handled by the operator's audited CLI (`polycarpic entity set-founder`), which notifies every owner.
- **An owner's tombstone** (§3.4a) re-evaluates every pending request on their entities in the same transaction. Their approvals stop counting and they stop being required. A request that is now fully approved is not auto-applied: its requester is notified to confirm, with step-up, by calling `apply_change` . Requests they raised are withdrawn.
- **Audit and notice.** Every step writes `change.requested` , `.approved` , `.declined` , `.withdrawn` , `.expired` , `.applied` or `.forced` . Every owner is notified on request and on outcome. The invitee sees nothing until apply.
- **Tests (pgTAP, SECURITY C-28):** with two owners, a direct membership INSERT, a role raise or an `equity_holding` INSERT by one owner is refused; removing a viewer stays direct; `apply_change` with one approval missing does nothing; an ex-owner's approval does not count; a decline and an expiry block; a stale cap-table payload is declined; a sole owner's invite applies at once; the first co-owner is added directly; outstanding direct invites are revoked at the 1 → 2 transition and one applied through `apply_change` is not; a revoked token is refused at acceptance; two direct owner invitations accepted concurrently end with two owners and the other revoked; after dropping back to one owner, a new direct invitation is revoked at the next transition; a held owner raise does not trigger it and its promotion does; every `force_change` misuse (non-founder, pending, invite or cap-table kind, founder as target) is refused; a forced removal notifies the target; the founder cannot drop owner without a transfer; the tombstone is refused while the user founds a multi-owner entity. Integration: no invite email before apply; each step without a recent step-up returns 403.
- **Scope:** the gate and the founder column ship with tenancy in POLY-M1, since any entity can gain a second owner; the request flow, screens and expiry job ship with sharing in POLY-1.0 (POLY-70). Between the two, a multi-owner entity's gated actions are simply refused, which is safe on staging, where no real books exist.

3.1c A raised role waits for the second step

SECURITY §4.1, C-03; PRD §5 ("a viewer promoted to manager or owner enrols before the new role takes effect"); DESIGN second-step flow (the member keeps viewer access, a `Waiting for second step` badge, the "Role raise waiting" notice). Meets SECURITY A-25 / C-03 / T-81: the stored value every check reads stays at the previous role until enrolment.

Choice: a pending role on `membership`. `membership.role` is always the role in force; a raise the member cannot yet hold is parked in a separate column and promoted by a trigger when they enrol. Every existing reader of `role` (`app.has_role`, `app.visible_entities`, `app.owner_count`, the approver set, `poly_graph`'s owner rows, `app.held_entities`, notification recipients) stays correct unchanged and fails safe, because none of them reads the new column. The rejected alternative, storing the new role and masking it until enrolment, would have to be applied at every one of those readers, and any reader that forgot would grant the role early. Refusal at apply was rejected because it throws away an approval the co-owners already gave and makes the owner chase the member before asking.



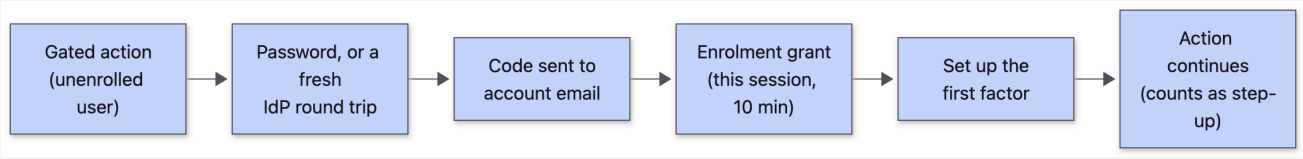
- **Schema.** `membership` gains `pending_role` (nullable: `manager` | `owner`), `pending_since`, `pending_by_user_id` (who raised it) and `pending_change_id` (FK `change_request`, null on the direct path). `CHECK (pending_role IS NULL OR role = 'viewer')`, and the four columns are null together. `user` gains `second_step_at` (nullable), the time the user's first second-step method was verified: set when BetterAuth marks TOTP verified or the first passkey row is inserted, cleared when the last method goes. It is written only by `app.sync_second_step()`, an AFTER trigger on BetterAuth's `twoFactor` and `passkey` tables, `SECURITY DEFINER` owned by `poly_govern`, which recomputes it from those tables; `poly_app` holds no UPDATE grant on the column, so a caller cannot mark itself enrolled without a factor row. Authorization reads one column of `user`, never the auth plugin's tables. The database cannot check a WebAuthn ceremony; that a factor row is genuine rests on BetterAuth, as every sign-in does.
- **The hold.** A BEFORE INSERT / UPDATE trigger on `membership`, `app.hold_unenrolled_role()`, owned by `poly_authz`, compares the new role with the member's `second_step_at`:
  - *UPDATE* from viewer to manager or owner, member not enrolled: rewrites the row to `role = 'viewer'` with `pending_role` set, and writes the "Role raise waiting" `notification` to the member and a `role.held` `audit_event` in the same transaction. The caller sees the held row in `RETURNING`. This covers the sole owner's direct raise and `app.apply_change` on a `role_raise` request alike, since both are the same UPDATE.
  - *INSERT* of manager or owner for a member not enrolled: refused (`second_step_required`). An invitee joining as owner or manager enrols on Sign up before the invitation is accepted, so the membership row is written only after `second_step_at` is set; PRD story 1 requires that they cannot reach the entity at all until then, which a held viewer row would breach. Creating a Person, Trust, Business, Household or Holding writes an owner row for the creator, so it too needs the second step first. A newly created Person is a new owned entity like the rest; only the sign-up Person is exempt.
  - *Exempt:* the owner row on the user's own Person written by `provisionPerson` (`entity.self_user_id = user_id`), while that Person is unshared. *Unshared* means no other member **and** no `entity_rollup` row with the Person as `child_entity_id` (SECURITY A-28). Without the edge clause, a user who rolled their Person into a Household and then gave up their role on the Household would regain the exemption and could drop their last method while the Household's members still see the Person. The predicate is one function, `app.own_person_exempt(user_id, entity_id)`, owned by `poly_authz`, which reads `entity_rollup` through a role-targeted `SELECT … TO poly_authz` policy (as it reads `membership`); the hold trigger, the last-method trigger, the

sign-in gate and `SecondStep.required` all call it, so they cannot disagree. Without the exemption every user, viewers included, would have to enrol at account creation, which contradicts "optional for viewers". The exemption ends at the user's first real action. Enrolment is forced before each of these:

  - creating a Person, Trust, Business, Household or Holding, because the founder's owner row is an INSERT and is refused;
  - sharing the own Person, because the first invite on it is refused while the inviter has no `second_step_at`; the BEFORE INSERT trigger on `invitation` requires an enrolled inviter on every entity;
  - linking the own Person under a parent: `app.check_rollup_edge()` refuses an edge whose child is a Person whose `self_user_id` has no `second_step_at` (`second_step_required`), since the parent's members would then see it (A-28, C-25);
  - accepting an owner or manager invite (INSERT refused, above), or having a raise to manager or owner take effect (held, then promoted);
  - any step-up action on the own Person: export, purge, granting access, linking a bank, since step-up needs a second factor (the sign-in gate below).

  Viewing shared entities, as a viewer, never triggers it.
- *A manager raised to owner* is already enrolled (else they could not hold manager), so the raise applies at once. A manager whose methods the operator reset is the one exception; see the sign-in gate below.

- **First enrolment is anchored on the account email** (SECURITY §4.1, A-29, T-82). A gate that asks for the password and then accepts whatever first factor is offered stops someone at an unlocked laptop, not a stolen password: with no existing factor there is nothing to compare against. So while `second_step_at` is null, setting up a factor needs the password (for an OAuth-only user, a fresh round trip to their IdP) **and** a code sent to the account's current, verified email address, which SECURITY already treats as the recovery root. This covers every own-Person gate, re-enrolment after the last method is removed, and the sign-in gate after the operator's `reset-mfa`. Once a factor exists, adding another method and every step-up use it as before.



  - *The code:* `email_challenge`. Columns: `id`, `user_id`, `session_id` (FK BetterAuth `session`, `ON DELETE CASCADE`), `purpose` (`first_enrolment` | `account_deletion`),

`code_hash` (six digits from a CSPRNG), `link_hash` (the same code offered as a link), `created_at`, `expires_at` (created + 10 minutes), `attempts`, `used_at`. Hashed as a T1 verification token and never logged (SECURITY §4.1). Our table, not BetterAuth's `verification`: that keys on an identifier string, carries no session, and its plugin deletes the row on use, so no trigger could read it. RLS by `user_id = app.current_user_id()`; `poly_app` may SELECT the state columns only, never the hashes, and has no INSERT or UPDATE.

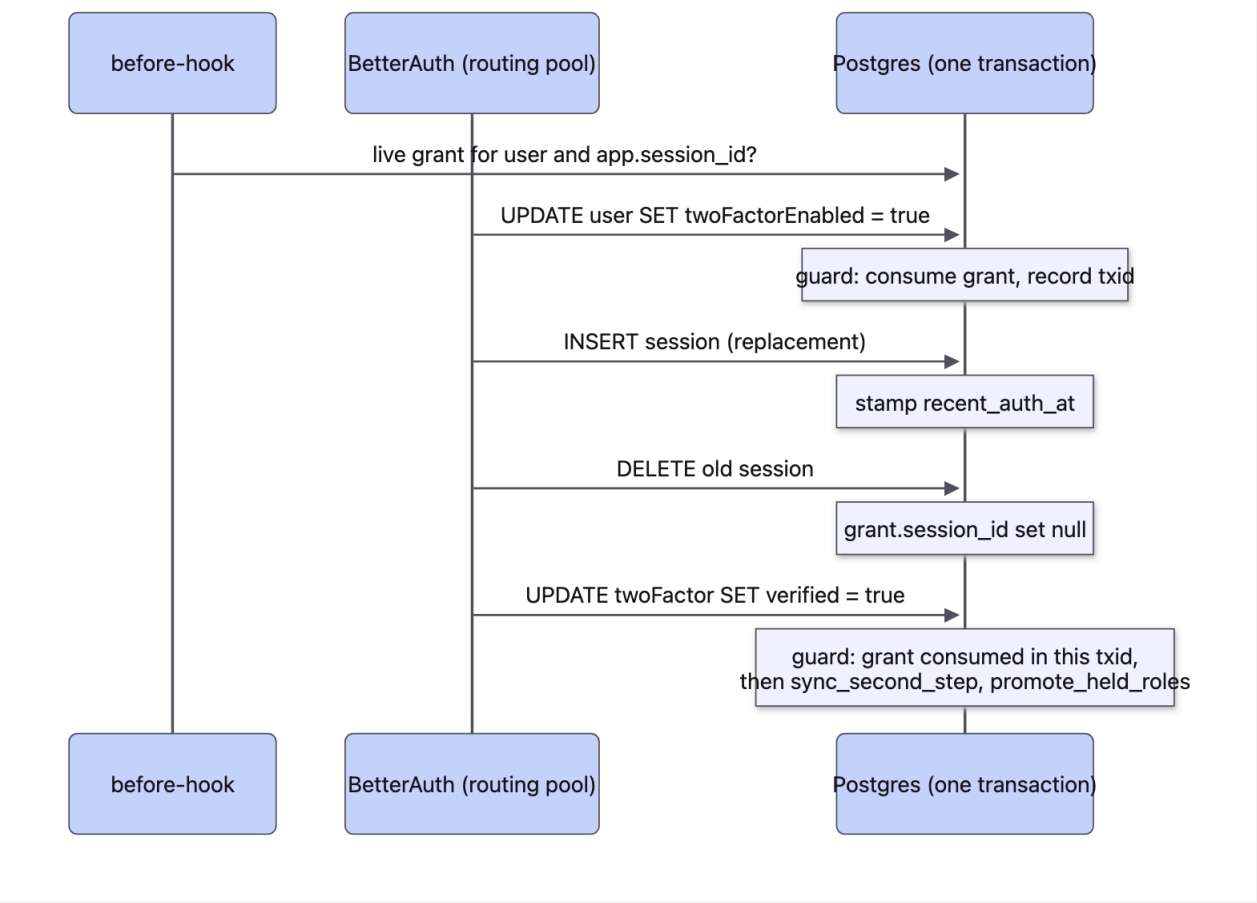
- *Send.* `app.issue_email_challenge(purpose, reauth_at)`, a `poly_govern` definer, requires the password check (or IdP round trip) in this session within 5 minutes, generates the code with `gen_random_bytes`, stores only its hash, voids any earlier open challenge of that user and purpose, and returns the plaintext once for the route to send through Resend after commit, to the current verified address only, never a pending new one (changing email is itself step-up). At most 3 sends per user per 15 minutes (C-04); the response is the same whether or not a send was suppressed.
- *Confirm, bound to the session that asked.* `app.confirm_email_challenge(code)` (`poly_govern`) matches the hash, requires `session_id` to equal the caller's session, the challenge unexpired and unused, and voids it on the fifth wrong entry. The request's session reaches the database as the transaction-local GUC `app.session_id`, set by `withTenant` beside `app.user_id` (§3.3). A code entered in another session, or after a new sign-in, is refused; a link opened in another browser says the request came from another device and, if that was not the user, to change the password. Without the binding the victim clicking the thief's link would complete the thief's enrolment. On success it marks the challenge used and, for `first_enrolment`, writes the grant.
- *The grant:* `enrolment_grant` (SECURITY A-29, A-32): `user_id`, `session_id` (FK BetterAuth `session`, `ON DELETE SET NULL`), `expires_at` (10 minutes), `consumed_at`, `consumed_txid` (`txid_current()` of the consuming transaction), with a unique partial index allowing one live grant per user. *Live* means unconsumed, unexpired and `session_id` not null. Written and consumed only by `poly_govern` definers. Not `CASCADE`: the first TOTP verify deletes the requesting session in the same transaction that consumed the grant, and the later `twoFactor.verified` guard must still find the consumed row (measured order, below).
- *BetterAuth's writes run in our transaction* (SECURITY A-31, measured on 1.7.18). On its own pool, BetterAuth's `twoFactor` INSERT ran on another backend and transaction, and the trigger read `app.session_id` as null; a refusal at the last write of the first TOTP verify then left the user half-enrolled. So BetterAuth's `database` is a routing pool, about 30 lines in `packages/auth`: an object with `connect()` that, inside `withTenant` (found through `AsyncLocalStorage`), returns the request's transaction client wrapped so that

BetterAuth's `BEGIN`, `COMMIT` and `ROLLBACK` become `SAVEPOINT`, `RELEASE SAVEPOINT` and `ROLLBACK TO SAVEPOINT`, and `release()` is a no-op; outside `withTenant` it returns a client from a plain pool, with no GUC set. Measured: the trigger saw the GUC on our backend and transaction, rolling ours back removed BetterAuth's rows, nested transactions became savepoints, and a guard that needs the GUC refused a write made outside `withTenant`. Consequences:

- *Which requests run inside.* The `/api/auth/*` handler loads the session first (on the plain pool), then, for a signed-in request, runs BetterAuth inside `withTenant(userId, sessionId, …)`, so every write of an enrolment, step-up or sign-out commits or rolls back with ours. Sign-in, sign-up by invitation, magic link and OAuth callbacks have no session yet and run on the plain pool (except that every route that can insert a `user` row, which includes invitation sign-up, OAuth first sign-in and magic link, runs inside `withContextFreeTx`, so the hook's `app.user_id` step and `provisionPerson` share BetterAuth's client, §3.1; with no such transaction open, the hook throws and writes nothing, and never opens a transaction of its own. That is a test-time catch only: in production BetterAuth has already committed the user row by then, so the per-route tests are the real guard (SECURITY A-40); the invitation is checked and consumed in that same transaction, so two sign-ups racing on one invitation yield one user, SECURITY C-84); none of them writes a factor row, and the enrolment guards refuse there because `app.session_id` is unset.
- *The role BetterAuth runs as:* `poly_app`, the `withTenant` login role, on both paths (the plain pool uses the same credential). Its grants on BetterAuth's tables: `user` SELECT, INSERT, and UPDATE on BetterAuth's own columns only (never `second_step_at`, the tombstone columns or the id); `session`, `account`, `verification`, `twoFactor`, `passkey` SELECT, INSERT, UPDATE, DELETE. No grant on `email_challenge` or `enrolment_grant` beyond the state-column SELECT above. BetterAuth's tables are keyed by user, not entity, and are read before a user is known, so they carry no entity RLS; the guards are the triggers below, and `poly_app` already sees every password hash, the same trust as today (SECURITY §4.1). The role checks in the POLY-62 spike ran as a superuser; POLY-76 re-ran the grant list as `poly_app` and it held (DELETE on `user`, UPDATE of `id` and TRUNCATE refused with 42501). BetterAuth's columns are snake_case, mapped through the `fields` map that `db` derives from the Drizzle tables, plugins included; BetterAuth refuses to start on a mismatch.
- *One request, one client.* The wrapper serialises `connect()` calls inside a request; a second concurrent `connect()` while a savepoint is open waits rather than interleaving savepoints (unmeasured; covered by the integration test).

- *Pinned.* The shim depends on how BetterAuth's Kysely layer calls `pool.connect()` and issues transaction statements, so `better-auth` and `@better-auth/passkey` are pinned to an exact version, and an integration test (§8) fails if a release changes that: the GUC is visible to a trigger on `twoFactor` and `passkey`, the backend pid and `txid_current()` match ours, a rollback removes BetterAuth's rows, a nested transaction becomes a savepoint, and a factor write outside `withTenant` is refused.

- *The order BetterAuth writes in* (measured). `/two-factor/enable` needs the password, writes a `twoFactor` row with `verified = false` and encrypted secret and backup codes, leaves `user.twoFactorEnabled` false, and returns the TOTP URI and the backup codes at once. The first `/two-factor/verify-totp` then sets `user.twoFactorEnabled`, creates a new session, deletes the requesting one, and only then sets `twoFactor.verified`. Passkey registration (`/passkey/generate-register-options`, then `/passkey/verify-registration`) needs only a session and inserts the `passkey` row; no session is replaced, since `createSession` is not set.



- *Not completed until confirmed: two guards.* (1) *App:* the global BetterAuth `hooks.before` (it sees the path and the session; when it throws nothing is written, measured), on its enrolment class (the allowlist below: `/two-factor/enable`, `/two-factor/verify-totp`

with a session, `/passkey/generate-register-options` and `/passkey/verify-registration`), refuses a user with no `second_step_at` unless a live grant exists for that user *and the requesting session*, so no TOTP secret or backup code is shown and no passkey ceremony begins without it. For an OAuth-only user (`allowPasswordless`, so `/enable` asks no password) it also requires the fresh IdP round trip in this session within 5 minutes. `databaseHooks` cannot be the gate: they never run for `twoFactor` or `passkey` (measured). (2) *Database*, all `poly_govern` definers that read `app.session_id`, refusing with `email_confirmation_required` for a user whose `second_step_at` is null:

- *TOTP, at the flip* (A-32): BEFORE UPDATE on `user` when `twoFactorEnabled` goes false → true requires a live grant for the user and `app.session_id`, and consumes it, setting `consumed_at` and `consumed_txid = txid_current()`. It fires first in the sequence, while the session and the grant still exist. A refusal rolls back the whole request: `twoFactorEnabled` stays false and the old session stays in place.
- *TOTP, at verified*: BEFORE UPDATE on `twoFactor` that sets `verified` passes only if a grant for that user has `consumed_txid = txid_current()`.
- *Passkey*: BEFORE INSERT on `passkey` requires a live grant for the user and `app.session_id` and consumes it the same way (unchanged from A-29).

`app.sync_second_step()` then sets `second_step_at` and `app.promote_held_roles` runs. A forgotten route, or a factor write outside `withTenant`, fails closed.

- *Email is not a factor.* BetterAuth's `otp` two-factor method is not configured (a `method: "otp"` call is refused, measured); the code anchors the first enrolment and never stands in for the second step. `skipVerificationOnEnable` stays unset (false). Backup codes are stored encrypted under the auth secret, not hashed (BetterAuth decrypts to verify), and one redeems at `/two-factor/verify-backup-code` while TOTP is still unverified, flipping nothing (measured). So `requireRecentAuth()` accepts a backup code as step-up only for a user whose `second_step_at` is set (C-03).
- *Set-up counts as the step-up* (SECURITY §4.1). A completed first enrolment has checked the password or IdP, the emailed code and the new factor's own proof in this session, more than Confirm it's you asks, so the factor-write transaction sets `recent_auth_at`, and the gated action continues without a second prompt within the same 5 minutes. Where it lands (A-32): on the passkey path, the guard stamps the requesting session. On the TOTP path the requesting session is deleted, and BetterAuth builds the replacement from the old row as loaded at the start of the request, so a stamp on the old row would not carry over; a BEFORE INSERT on `session` (`poly_govern`) sets `recent_auth_at = now()` on the new row when a grant for that user has `consumed_txid = txid_current()`. The client receives the replacement session's cookie in the same response. A failed or abandoned

set-up sets nothing. The interstitial is therefore three steps: password, emailed code, factor set-up. The user is emailed when a factor is enrolled (SECURITY §9).

  - *What this protects.* A thief with the password and not the mailbox cannot finish first enrolment, so cannot reach any gated action. The database guard stops a route that skips the email step, not a compromised `web` process, which already sees passwords, the same trust as today. A thief holding both the password and the mailbox can enrol; email is already the recovery root, so this adds none (SECURITY residual).

- **Every auth route is classified** (SECURITY A-34, T-86). The one global `hooks.before` is an allowlist over `/api/auth/*`, so a session alone reaches nothing we have not looked at. It lives in `packages/auth/src/route-policy.ts`: a constant map `AUTH_ROUTES`, path → class, and the hook built from it with `createAuthMiddleware`. Each path is in exactly one class, and a path not in the map answers 404 (`APIError("NOT_FOUND")`), the same answer a blocked path gives, so nothing tells a caller which routes exist. The list is every HTTP route better-auth 1.7.18 and @better-auth/passkey 1.7.18 expose with the plugins we enable (core, `twoFactor`, `magicLink`, passkey), read from the published tarballs (code-read, 47 paths):

Class	Paths	What the hook checks
Open (23)	`/ok`, `/error`; `/sign-in/email`, `/sign-in/social`, `/callback/:id`, `/sign-in/magic-link`, `/magic-link/verify`, `/sign-up/email`, `/sign-out`; sign-in's second step: `/two-factor/verify-totp` and `/two-factor/verify-backup-code` with no session, `/passkey/generate-authenticate-options`, `/passkey/verify-authentication`; `/verify-email`, `/send-verification-`	Nothing beyond BetterAuth's own checks and the rate limits (C-04), with four narrowings: `/sign-up/email` needs a live invitation, as there is no other sign-up (SECURITY S-1); `/update-user` accepts only `name` and `image` in the body, and `image` is null or a path on our own origin (an external URL is refused: every co-member's browser would fetch it; SECURITY A-38); `/passkey/update-passkey` only the name; `/change-password` already needs the current password and is sent with `revokeOtherSessions: true`. `/verify-backup-code` with a session is refused: a backup code as step-up goes through Confirm it's you. `/list-accounts` returns provider ids and scopes, no tokens (code-read; the test asserts it). Three conditions (SECURITY A-38): `/request-password-reset`, `/reset-password` and `/send-verification-email` answer identically for known and unknown emails (C-04), and a completed reset revokes every session of the user; `/sign-in/social` and `/callback/:id` answer 404 in private-CA mode (C-76, §7.8), a gate the route policy carries; and

Class	Paths	What the hook checks
	`email`, `/request-password-reset`, `/reset-password/:token`, `/reset-password`, `/change-password`; `/update-user`; `/revoke-sessions` (it also ends the caller's session); `/list-accounts`, `/passkey/list-user-passkeys`, `/passkey/update-passkey`	`/magic-link/verify` and `/callback/:id` get no exception from P1 — a session they create for a user with `second_step_at` set reaches no `withTenant` until the second step is verified. Whether BetterAuth's `twoFactor` plugin already holds these two paths, and not only `/sign-in/email`, is (unmeasured) on 1.7.18; POLY-86's route-policy test measures it. If it does not, `packages/auth` adds the hold: such a session is marked pending, `withTenant` refuses it, and only the second step clears the mark.
Enrolment (4)	`/two-factor/enable`, `/two-factor/verify-totp` with a session, `/passkey/generate-register-options`, `/passkey/verify-registration`	With no `second_step_at`: the live grant for this user and session, plus the IdP round trip for an OAuth-only user (above). With it: the step-up check below, which takes the factor the user already has. A stolen session therefore cannot add a passkey that would outlive a password reset.
Step-up (7)	`/passkey/delete-passkey`, `/two-factor/disable`, `/two-factor/generate-backup-codes`, `/unlink-account`, `/link-social`, `/change-email`, `/revoke-other-sessions` (SECURITY A-38)	The step-up check. The database's last-method trigger (below) stays the floor under the first two. `/revoke-other-sessions` is here because, open, a stolen cookie could sign the owner out everywhere and keep the thief's own session.

Class	Paths	What the hook checks
Blocked (13)	`/list-sessions` and `/revoke-session` (raw tokens out and in); `/get-session` over HTTP (its body carries the token); `/two-factor/get-totp-uri` (the secret is shown once, at enable); `/two-factor/send-otp`, `/two-factor/verify-otp` (`otp` is off); `/delete-user`, `/delete-user/callback` (C-52's route deletes); `/update-session`; `/get-access-token`, `/refresh-token`, `/account-info` (IdP tokens never leave the server); `/verify-password`	404 whenever the call arrives over HTTP, refused twice: in the SvelteKit `handle` (`hooks.server.ts`), which answers 404 before calling `auth.handler`, and again in the hook (SECURITY A-38). The app's own Settings → Sessions reads the `session` table through `withTenant` and returns id, device, created and last-seen, never the token; it revokes by id.

- *The step-up check* is `requireRecentAuth()` (§3.3), run inside the hook: it loads the session from the request, reads `recent_auth_at` and passes only if it is within the last 5 minutes. Otherwise it throws a 403 with code `step_up_required`; the client opens Confirm it's you, which sets the stamp after the password and second factor (or a passkey), and retries. `recent_auth_at` is a BetterAuth session field declared `input: false`, so no endpoint body can set it, and `/update-session` is blocked as well. Only a step-up that took a second factor or a completed first enrolment sets it (above), so a backup code from an unverified enrolment never opens this class.
- *HTTP versus in-process.* BetterAuth runs before-hooks on both paths (code-read, `api/dispatch.mjs`). The blocked class applies only when the call carries a request (`ctx.request` set), so our server code can still call `auth.api.getSession` in the SvelteKit loader. The enrolment and step-up classes apply on both paths. That the

discriminator holds is (unmeasured); the route-policy test measures it, and the `handle` check means the HTTP block does not depend on it. No load function returns the session token: the root layout returns a projection (user id, name, image, expiry), and when the in-process call refreshes the session its `Set-Cookie` is forwarded on the response (SECURITY A-38).

  - *The route-policy test* (`packages/auth/src/route-policy.test.ts` , §8) builds BetterAuth from the exported app configuration, the same object the app uses (SECURITY A-34), and collects the path of every HTTP endpoint on that instance. It fails if a path is missing from `AUTH_ROUTES` , naming it, and if `AUTH_ROUTES` lists a path the instance no longer has. So a version bump or a new plugin fails CI until each route is classified. Per class, over `auth.handler(new Request(…))` : every blocked path and an invented one answer 404; a step-up path passes with `recent_auth_at` 4 minutes old and answers 403 at 6 minutes and with none; an enrolled user's `/passkey/generate-register-options` on a fresh session without step-up answers 403; an OAuth-only user's `/two-factor/generate-backup-codes` and `/two-factor/disable` without step-up answer 403; `/update-user` with any field besides name and image is refused; `/sign-up/email` without a live invitation is refused; `/list-accounts` and the app's session list carry no token field; the in-process `auth.api.getSession` still answers while the HTTP `/get-session` is 404. SECURITY A-38 adds: the serialized page data carries no token; an HTTP `/get-session` is 404 from the `handle` even with the hook disabled; `/revoke-other-sessions` without step-up answers 403; an enrolled user's magic-link and OAuth sign-ins reach no tenant data before the second step (the measurement above); a reset ends every other session; an external `image` is refused; the OAuth paths are 404 in private-CA mode, so the test runs in both modes.

- **The promotion: the only write that makes the role effective** (A-25). When `second_step_at` goes from null to set, `app.sync_second_step()` calls `app.promote_held_roles(user_id)` , a `poly_govern` definer keyed by a transaction-local GUC on the user id (as `poly_purge` keys on the entity). It re-checks that the user has a factor row, sets `role = pending_role` and clears the four columns on each of that user's held rows, changes nothing else, and writes one `role.promoted` `audit_event` per row, in the enrolment's own transaction. No other path clears `pending_role` upward: the hold trigger refuses an UPDATE that sets `role` to manager or owner on a row with `pending_role` set unless `current_user = 'poly_govern'` inside this function. A raise to owner counts toward `app.owner_count` only from this moment, so the approver set, founder eligibility and the 1 → 2 transition see the new owner exactly when they can act; the promotion's membership write is what fires the transition (§3.1b), and an approved co-owner raise is not re-approved (A-25).

- **Cancelling a held raise.** Any owner may cancel one directly, as a lowering: `pending_role` and its three companions cleared, `role.cancelled` written, a notice to `pending_by_user_id` and one to the member, so their "Role raise waiting" notice is replaced rather than left stale; no approval, on any entity (A-25). A held raise does not expire (unmeasured need; nothing so far asks for one).
- **Direct-path held raises at the 1 → 2 transition.** A held raise made by a sole owner (`pending_change_id` null) is the same exposure as a direct invitation outstanding when a second owner arrives: it would land later without that owner's vote. So the 1 → 2 revocation (§3.1b) cancels it with the direct invitations, in the same transaction and under the same entity-row lock, notifying both the raiser and the member. A held raise applied through `apply_change` stays, as an invitation applied that way does (C-28).
- **The approval flow.** The `role_raise` request reaches `applied` when the last approval lands, exactly as today; the waiting lives in `membership`, not in `change_request`, so the request's state machine is unchanged and `pending_change_id` links the two for the audit trail. One open request per kind and target still holds: a held raise counts as open for its target, so a second `role_raise` for the same member is refused until it is promoted or cancelled.
- **Direct invites.** The invite path never holds: an owner or manager invitee enrols before the row exists (above). A viewer invitee may skip; raising them later follows the hold.
- **The founder.** `app.transfer_founder` requires a target holding owner in `role`, so a held owner is never eligible, and an eligible target is already enrolled. Account-deletion transfer (§3.4a) makes a member owner and founder in one transaction; a member without a second step would be held as viewer and the founder trigger would refuse the commit, so the deletion flow offers only enrolled members as transfer targets and shows the rest as needing a second step.
- **Keeping the second step.** A BEFORE DELETE trigger on `passkey`, and on the TOTP disable (BEFORE UPDATE on `user` when `twoFactorEnabled` goes true → false, and BEFORE DELETE on `twoFactor`), refuses removing a user's last method while they hold manager or owner on any entity outside the own-Person exemption (DESIGN: no Remove on the last method). The operator's audited `polycarpic user reset-mfa` (SECURITY §4.1) is the one path past it: it leaves memberships as they are and revokes every session (C-02).
- **The sign-in gate.** The session loader sends any user who holds manager or owner outside the exemption and has no `second_step_at` to Set up a second step, with no other route reachable. In normal operation that state cannot arise; it exists for the operator reset and for a row predating enrolment. Every step-up (SECURITY §4.1: password and code, or a passkey) needs a second factor, so a user with none who starts a risky action, an export, purge, grant or bank link on their own Person included, meets the same set-up first, password then the emailed code then the factor (above); the completed set-up counts as the step-up, so the

action then continues without Confirm it's you (SECURITY §4.1). Viewing shared entities never reaches it. BetterAuth's trusted-device window is set no longer than the 30-day absolute session (C-02), so "a known device asks nothing until its session ends" holds literally. `session.freshAge` is set to the same 30 days: BetterAuth's passkey routes refuse a session older than `freshAge` (`SESSION_NOT_FRESH`, measured at 25 hours against the 24-hour default), and "fresh" there means the session's age, not a re-authentication. Left at 24 hours, a user on a two-day-old session would pass the password and the emailed code and then fail the passkey step. The control is ours: the before-hook requires the grant (first factor) or `recent_auth_at` within 5 minutes (any later method), which is stricter than `freshAge`. It holds only with the allowlist below (SECURITY A-33, A-34), because `freshAge` also guards `/list-sessions` and `/unlink-account`, which the allowlist blocks or puts behind step-up.

- **Reads.** `MembershipList` (§2.4) carries `pending_role` and `pending_since`; members of the entity see the badge, since the row is their own entity's membership data. The member's own notice and Settings read their row and `second_step_at`.
- **Tests (pgTAP, with SECURITY C-03 and C-28):** a direct raise of an unenrolled viewer leaves `role = 'viewer'` and `has_role(…, 'manager')` false; the same through `apply_change`, with the request `applied`; enrolling (TOTP verified, or first passkey) promotes in the same transaction and `has_role` turns true; an unenrolled INSERT as manager or owner is refused and the own-Person row is not; a held owner does not count in `owner_count`, is not a required approver, and does not trigger the 1 → 2 transition, while its promotion does; `poly_app` cannot set `second_step_at` or promote a held row directly; the promotion writes an `audit_event` and touches no other column; an owner's cancel needs no approval; at the 1 → 2 transition a direct-path held raise is cancelled and its raiser notified, and an approved one is kept; `transfer_founder` to a held owner is refused; removing the last passkey while manager is refused, while viewer is allowed; a second `role_raise` for a held member is refused. Own Person: an unenrolled user's own-Person owner row is written at account creation; the same user's INSERT of a Trust (founder owner row) and their first `invitation` on the own Person are both refused, and both succeed after enrolment. Integration: the gated sign-in reaches only the set-up screen. Email anchoring (SECURITY A-29, A-30): with no live `enrolment_grant`, marking TOTP verified or inserting a first passkey raises `email_confirmation_required` and leaves `second_step_at` null; a code confirmed from another session, or after a new sign-in, is refused; an expired, used, superseded or fifth-wrong-entry code is refused; a fourth send within 15 minutes is suppressed with the same response; one grant enrols once; `poly_app` cannot write either table or read the hashes, and cannot UPDATE `user.second_step_at`; deleting the session that consumed a grant leaves the grant row with `session_id` null; a completed first enrolment sets `recent_auth_at` and a failed one does not. Routing pool (A-31, against the pinned BetterAuth version): see §3.1c above and §8. Integration: a password-only session cannot complete first enrolment (each of

`/two-factor/enable`, `/two-factor/verify-totp`, `/passkey/generate-register-options` and `/passkey/verify-registration` refused without a grant); a first TOTP verify the database guard refuses leaves `twoFactorEnabled` false, no new session and the old session in place; a completed first TOTP verify leaves the grant consumed with the request's txid, `second_step_at` set and `recent_auth_at` set on the replacement session, so the gated action passes step-up once and a 403 follows after 5 minutes; a `twoFactor.verified` update in a transaction that consumed no grant is refused; a backup code from an unverified enrolment does not pass step-up; passkey enrolment succeeds on a 25-hour-old session that has the grant; a gated action, the unenrolled user's Trust INSERT and their first own-Person invitation, is not performed until both confirmations are in, and then runs without a second Confirm it's you; an OAuth-only user enrols after an IdP round trip and the code; the unenrolled deletion stamp does not satisfy any other step-up action. A-28: an own Person with an `entity_rollup` parent and no other member is not exempt; a user who linked their Person into a Household and then left the Household as owner cannot remove their last method while the edge remains; an unenrolled user's edge with their own Person as child is refused. Held raises and new Persons: an owner's cancel and the 1 → 2 cancel each notify the raiser and the member; an unenrolled user's INSERT of a new Person (founder owner row, `self_user_id` null) is refused.

- **Scope:** POLY-M1, with tenancy and the second step (C-03), since a sole owner can raise a role from the first milestone.

3.2 Invariants the database enforces (not the app)

1. **Balanced entries.** A `DEFERRABLE INITIALLY DEFERRED` constraint trigger on `journal_line` checks `Σ base_amount = 0` per entry at commit, and at least two lines. The app computes balance too, for good error messages; the trigger is the backstop.
2. **Append-only posted rows.** `BEFORE UPDATE OR DELETE` triggers on `journal_entry`, `journal_line`, `journal_chain`, `change_approval`, `lot`, `lot_match`, `tax_period_freeze`, `tax_close` and on `raw_transaction` where `status = 'posted'` raise, and so do `BEFORE TRUNCATE` statement triggers on the same tables. No login role holds `TRUNCATE`. The one exception is DELETE by `current_user = 'poly_purge'` for the entity named in `app.purge_entity` (§3.4). Corrections go through `correctEntry(original, newLines)`. In one transaction it writes a reversal (a mirror entry with `reverses_entry_id` set) and a replacement entry with `replaces_entry_id` set. The UI calls this action "Correct entry". The UI never offers "edit" or "reverse" on a posted row; it badges the replacement and shows the struck-through reversal on click-through (DESIGN). `reverseEntry()` on its own stays internal, for a correction whose replacement is empty.

3. **Two-way link.** Confirming one or more drafts inserts the entry and, in the same transaction, sets each draft's `entry_id` and flips each raw row to `posted`. The link is held on the draft side (many drafts → one entry). The entry's `source_ref` only marks it as import-sourced; it doesn't point back to a single row. A check ensures a posted raw row always has an entry (gate G4, §3.2a).
4. **Single posting date.** One date column per entry; a `posting_date_allowed(entity, date)` function hook (always true in v1) is the seam where period locking plugs in later without a schema change.
5. **Same-entity legs.** Every line's account belongs to the entry's entity (composite FK `(entity_id, account_id)`). Cross-entity transfers are two entries, one per entity's books (§4.3).
6. **Actor recorded.** `actor_kind` + `actor_id` are NOT NULL on entries and drafts, and are *forced* by a BEFORE INSERT trigger from the transaction's context rather than merely defaulted. `actor_kind = 'user'` requires `actor_id = app.current_user_id()`, so a caller cannot attribute an entry to someone else (SECURITY A-5).
7. **Gapless Journal ID.** Every entry carries `journal_number`, unique on `(entity_id, journal_number)` and shown as `JE-000214`; the UUID stays the key. A BEFORE INSERT trigger assigns it, so a caller cannot supply one: `UPDATE entity_journal_counter SET last_number = last_number + 1 … RETURNING`. Since `poly_app` has no write grant on the counter, the trigger function is `SECURITY DEFINER`, owned by the NOLOGIN, NOBYPASSRLS role `poly_journal`, which also owns the chain trigger below (SECURITY A-23). A plain sequence would burn a number on rollback. The counter row is locked until commit, so a rolled-back posting, including one failing the deferred balance check, returns its number, and postings in one entity serialize. Different entities never contend. The append-only trigger is unaffected, because the number is set before the row exists and the counter is a separate table. A correction takes two numbers, reversal then replacement, in its one transaction. A transaction touching several entities takes the counters in `entity_id` order, which rules out deadlock. Auto-post batches commit per entry or in short groups, so a batch never holds the lock against a user posting by hand. The cost is one row lock per posting (unmeasured; household write rates make contention unlikely).
8. **Lots tie to the ledger.** For each custodial account and securities nature class, the ledger balance equals the summed adjusted basis of open lots there (§3.6). Lot write and entry commit in one transaction; a periodic assert job is the tripwire, and a mismatch alerts rather than repairs.
9. **Hash-chained ledger** (SECURITY C-24; POLY-M2). Each entity's posted entries form a SHA-256 chain in `journal_number` order: `entry_hash = sha256(prev_hash ‖ canonical(entry, its lines))`, the first entry chaining from a fixed per-entity seed. The

canonical encoding is pinned and versioned (`alg_version`): field order, numeric text form, UTF-8 NFC.

- **Written at commit.** The hash covers the lines, which do not exist when the entry is inserted, so it is computed by a `DEFERRABLE INITIALLY DEFERRED` constraint trigger on `journal_entry`, `app.chain_entry()`. It reads the entry's lines, reads the head from `entity_journal_counter`, inserts the `journal_chain` row and moves the head. The chain row follows the balance check in effect, not in firing order: deferred triggers on two tables fire in queue order, so the order is not guaranteed, and it does not need to be: a failed balance check aborts the transaction, chain row included, so no chain row ever commits for an unbalanced entry.
- **No new lock.** The counter row is already locked by the numbering trigger until commit, so the head moves under the same serialisation as the gapless number, and a rolled-back posting leaves neither a number nor a chain row. Two entries in one transaction (a correction) chain in number order, which is their queue order.
- **The trigger is the only writer** (SECURITY A-23, C-24). `poly_app` and `poly_worker` hold no write grant on `journal_chain` or the counter, and read both through ordinary tenant RLS, viewer and up, for the export head and the verifier. `poly_journal`'s grants, each through a role-targeted policy `TO poly_journal`: SELECT, plus UPDATE of `last_number`, `head_number` and `head_hash` only, on the counter; INSERT on the counter, used only by the AFTER INSERT trigger on `entity`; INSERT on `journal_chain`, never UPDATE or DELETE (append-only under C-21, the purge excepted); SELECT on `journal_entry`, `journal_line` and `journal_chain`. The wide read is acceptable for the reason SecEng gives for `poly_graph`: a function returning `trigger` cannot be called to return data. Posted entries are never UPDATEd to hold their hash.
- **No line after the chain row.** `SET CONSTRAINTS … IMMEDIATE` is open to any role, so `poly_app` could fire the deferred chain trigger before an entry's lines exist. A `poly_journal` BEFORE INSERT trigger on `journal_line` refuses a line whose entry already has a `journal_chain` row, so the chain always covers every line and an early fire fails the posting instead of raising a false SEV1 at the verifier.
- **Verifier.** A weekly job unit per entity, running under the entity's earliest owner through `app.job_targets` (§3.3), recomputes the chain from the entity's lines and compares every link and the head. A break is SEV1 ledger tampering: an `audit_event`, a notification and an email to every owner, and an email to the operator (§7.4).
- **Head leaves the box.** Every export carries the head `(journal_number, entry_hash)`, recorded in its `export.completed` event, and the `ExportReceipt` DTO (§2.4) carries both, with `alg_version`: the receipt shows `JE-000214` and a shortened hash, the full

hash behind Copy. An entity with no posted entry has no head; the receipt says so and the field is null. A purge deletes the chain and records the final head in `entity.purged` (§3.4).

- **Limit.** The operator's superuser can disable the triggers and recompute the whole chain. Detection then rests on a head held off the box, an export the owner kept (R-1).
- **Cost.** One SHA-256 over an entry of a few lines and one row insert per posting, inside a lock already held (unmeasured; expected negligible against the 2 s dashboard NFR, which is a read path).
- **Tests (C-24):** a superuser edit (triggers disabled) to an entry, a line, or the order is caught by the verifier; a rolled-back posting leaves no chain row and no gap; `poly_app` cannot insert into or update `journal_chain`, or insert or update a counter row; a new entity gets its counter at number 0 with the seed, and a caller-supplied `journal_number` is overwritten; under `SET CONSTRAINTS ALL IMMEDIATE`, an entry then its lines is refused and no chain row commits; catalog: each `poly_journal` function is attached as a trigger only to its intended table (`pg_trigger.tgfoid` to `tgrelid`), and a direct call fails; a correction's two entries take two links in number order; the export's head equals the verifier's.

3.2a Posting gates

A draft or group of drafts leaves Pending only by passing every gate below. An item whose entry is not correct stays in the Pending queue. `posting.confirm()` evaluates the gates and returns a `GateResult` that the modal shows live; the Post button stays disabled until all gates pass. The database re-checks gates G1, G2, G4 and G5 at commit, so a bug in the app cannot bypass them.

#	Gate	Enforced by
G1	**Balanced:** Σ base_amount = 0 over the entry's lines, with at least two lines.	App (live) + deferred constraint trigger (§3.2.1)
G2	**Accounting equation per entity:** every line's account belongs to the entry's entity. With G1, every entity's books then keep Assets = Liabilities + Equity (+ Income − Expense) after every posting. No separate entity-wide sum runs per posting; a property-based test asserts the trial balance is zero after random postings.	Composite FK (§3.2.5) + G1; test
G3	**Dates:** an entry has exactly one posting date, equal to the bank date of every raw row it consumes. Legs with different dates never share an entry: a transfer whose legs land on different dates posts as two entries	App + a check in `confirm()`

#	Gate	Enforced by
	through Cash in Transit (PRD §5). A group (Flow B) may only combine drafts with the same bank date, so a daily summary is allowed and a weekly one is not.	
G4	**Linked:** every draft being posted gets exactly one `entry_id`, and its raw row flips to `posted` in the same transaction. A posted draft without an entry, or an entry claiming a draft that is still pending, is rejected.	Check constraint + trigger (§3.2.3)
G5	**Custodial leg locked:** for each custodial account touched by the consumed raw rows, the entry carries lines on that custodial's cash ledger account (its liability account for a credit card) summing exactly to those rows' amounts. The user edits only the offsetting legs; the bank's reality is never editable. A split keeps one locked leg and divides the offset; a group aggregates the locked legs into one line per custodial account.	App (the locked leg is not editable in the modal) + trigger comparing against the raw rows
G6	**Allowed:** the actor holds manager+ on the entity, or the post comes from a rule whose `auto_post_by_user_id` still holds manager+ on the entity. `posting_date_allowed()` passes; it always does in v1, and is the hook for period locking later.	RLS + trigger (§3.2.6)

G7, securities lines carry lots: a line on a securities nature account needs its lot write in the same transaction, and the security needs a class mapping. An unmapped trade stays in Pending until it is mapped, in keeping with the rule above. This applies to manual entries and corrections too, and `posting.confirm()` enforces it; the §3.2.8 assert is the backstop.

A manual entry ("+ New entry") consumes no raw rows, so G3–G5 do not apply to it; G1, G2, G6 and G7 do. A "Correct entry" replacement runs G1, G2 and G6, and must keep the original's custodial lines unless the user explicitly re-links the raw rows.

3.3 RLS mechanism and connection-pooling safety

SECURITY §4.3 / §12 govern this section. The permission matrix is SECURITY §4.2.1: roles are uniform across entity types.

- **Roles.** `poly_owner` (NOLOGIN; owns every object); `poly_migrator` (LOGIN; used only by the migrate step; runs migrations after `SET ROLE poly_owner` on its own client, closed after the run and never `RESET ROLE`, §6.2); `poly_app` (LOGIN; DML grants only; `NOBYPASSRLS`; not the table owner, so RLS applies); `poly_worker` (LOGIN; `NOBYPASSRLS`; reference-data

writes plus tenant DML limited to the import path, SECURITY §4.4); `poly_authz` (NOLOGIN; owns the authorization functions and nothing else); `poly_purge` (NOLOGIN; `NOBYPASSRLS`; owns the purge function; DELETE on purgeable tables only, plus the tombstone UPDATE on `entity`, §3.4b); `poly_backup` (LOGIN, `REPLICATION`, `BYPASSRLS`, member of `pg_read_all_data`, no write grants; `NOSUPERUSER NOCREATEDB NOCREATEROLE`, `CONNECTION LIMIT 3`, `default_transaction_read_only = on`; used only by the backup job, from the backup network, §7.2; SECURITY C-79, A-36). The remaining NOLOGIN, NOBYPASSRLS roles own definer functions and no tables:

  - `poly_rollup`: `app.rollup_totals`, `app.rollup_economic`, `app.rollup_tree`, `app.rollup_descendants`, `app.parents_of`; SELECT only, confined to descendants of the caller's parents and to ancestors of entities the caller owns (§3.3a).
  - `poly_graph`: the trigger functions `app.check_rollup_edge()`, `app.check_equity_holding()`, `app.notify_ancestor_invite()` and `app.notify_former_holder()`; graph-wide SELECT on `entity_rollup` and `equity_holding`, SELECT on owner `membership` rows and `entity.kind`, INSERT on `notification` (§3.3b, SECURITY A-16, A-21).
  - `poly_stake`: `app.cap_table_candidates`, `app.my_stakes`, `app.allocated_income`, `app.held_entities`; SELECT only, confined to `app.held_entities()` (§3.1a, SECURITY A-18).
  - `poly_govern`: `app.apply_change`, `app.withdraw_change`, `app.expire_changes`, `app.force_change`, `app.transfer_founder`; the writes listed in §3.1b (SECURITY A-20, A-22); the email-challenge definers, the enrolment guards on `user`, `twoFactor`, `passkey` and `session`, `app.sync_second_step()` and `app.promote_held_roles()` (§3.1c; SECURITY A-29, A-32).
  - `poly_journal`: the counter-creation trigger on `entity`, the numbering, chain and late-line triggers of §3.2.7 and §3.2.9; grants as listed in §3.2.9 (SECURITY A-23).

  Every one of these functions is on the definer allowlist (C-15) and every role is in the role-hygiene set (C-10). Calling a trigger function directly fails. Role hygiene follows SECURITY §4.3(k). The operator's superuser bypasses RLS by definition — documented as an accepted risk (PRD §6). BetterAuth's own statements run as `poly_app` too, through the routing pool; its grants on BetterAuth's tables are listed in §3.1c (SECURITY A-31).
- **Who creates the roles**. A superuser-run, idempotent file, `packages/db/sql/roles.sql`, creates every role above and resets its attributes on each run, grants `poly_migrator` its memberships, makes `poly_owner` the database owner, and revokes EXECUTE on new functions from PUBLIC by default. `provision.sh` runs it on the box, and the test harness runs it before the battery. Roles live outside the chain because `poly_backup`'s attributes need a superuser and `poly_owner` is given no `CREATEROLE`. A file that creates a function

owned by a definer role grants that role CREATE on `app` and revokes it before the file ends. Detail: `process/cairn/reviews/POLY-83/ruling.md` R1–R3.

- **Context per transaction, never per session.** `withTenant(userId, sessionId, fn)` opens a transaction and runs `select set_config('app.user_id', $1, true)` (`is_local = true`) before any query, and sets `app.session_id` the same way from the request's session, which only the email-challenge definers and the enrolment guards read (§3.1c); system jobs leave it unset. BetterAuth's statements in a signed-in auth request run inside the same transaction through the routing pool, so they see both settings and commit or roll back with the request (SECURITY A-31). The step-up window is per session: `requireRecentAuth()` reads `recent_auth_at` on the server-side session record, set by a successful step-up or a completed first enrolment, never the session's creation time (SECURITY §4.1). The setting dies at COMMIT/ROLLBACK, so a pooled connection cannot carry a previous request's identity. `withTenant`, `withSystem` and `withContextFreeTx` (§3.1) refuse to nest inside one another, and the context-free one asserts with its first statement that `app.user_id` is empty (SECURITY C-84). This works with the in-process `node-postgres` pool on the VPS and with Neon's transaction-mode pooler. Session-level `SET` is banned by lint.
- **Default deny.** Policies read context only through `app.current_user_id()` = `nullif(current_setting('app.user_id', true), '')::uuid`. The `nullif` matters: once a backend has set the GUC and the transaction has ended, `current_setting` returns `''`, not NULL. Without `nullif`, a reused backend would compare against an empty string instead of denying cleanly (SECURITY A-3). Every tenant table has RLS `ENABLE`d and `FORCE`d, including drafts, raw rows, lots and staging tables.
- **Policy shape.** `using (entity_id = any (app.visible_entities()))` for reads; writes add `app.has_role(entity_id, 'manager')` or `'owner'` per the permission matrix. Both functions are `STABLE SECURITY DEFINER`, owned by `poly_authz` (not `poly_owner`, because under FORCE RLS the owner is itself subject to policies). They read `membership` through a role-targeted `SELECT … TO poly_authz` policy, with `search_path = ''`. Every definer function is on a CI-checked allowlist (SECURITY A-1, §4.3(h)). Tenant policies never consult parent/child structure — structure grants no rows. The single place `entity_rollup` is read for authority is the consolidation function in §3.3a, and it returns totals, never rows.
- **System jobs.** `withSystem(job, target, fn)` takes no principal from its caller: it looks the target up in `app.job_targets(job)` inside its own transaction, then sets `app.user_id` to the target's principal and `app.actor = system:<job>`. It never runs context-free; if no principal resolves, the job errors. Targets come from one allowlisted definer function, `app.job_targets(job)`, which returns `(entity_id, principal_user_id, connection_id)` and is executable only by `poly_worker`. Entity-bound jobs with no connection (the tax-period freeze) run as the entity's earliest owner membership. Reference-

data policies are role-targeted (`TO poly_worker`), and `poly_app` gets SELECT only. Cash-in-Transit aging is computed on read by the exceptions report, not stored by a job (SECURITY A-12). Tenant writes by a job (import polling) run under the linking user's membership (`provider_connection.linked_by_user_id`) with actor = `system:<job>` , so authority and attribution stay separate. There is no system bypass clause in tenant policies (SECURITY §4.4).

- **SecEng additions.** (a) The RLS coverage probe is derived from `pg_catalog` , never from a hand-maintained list. (b) `poly_app` also loses `CREATE` on `public` . (c) Policies are per-command with an explicit `WITH CHECK` on every write; `FOR ALL` is banned. (d) The inversion canary stays in the battery. The review added (e) to (l) (SECURITY §4.3):
  - (e) Authorization functions are owned by `poly_authz` .
  - (f) Context is read in the `nullif` form.
  - (g) TRUNCATE triggers.
  - (h) `search_path = ''` , plus a definer allowlist.
  - (i) Tenancy columns are not updatable.
  - (j) Views are `security_invoker = true` , and there are no materialized views over tenant data (a matview has no RLS). A report needing a cache materializes into a tenant table under RLS.
  - (k) Role hygiene.
  - (l) UUID keys.
- **Proof.** Every migration run in CI executes an RLS battery: an *inversion self-test* (an unprotected canary table must leak, proving the probe has teeth — borrowed from mosko-fintech), a per-table cross-tenant probe generated from the catalog over every relation of kind `r` or `p` outside `pg_catalog` , `information_schema` and `pg_toast` , partitions included. A table with RLS disabled or not FORCEd fails the build. A materialized view or foreign table (`m` , `f`) fails unless exempted, since RLS never covers them. The only exemptions, BetterAuth's tables and `drizzle.__drizzle_migrations` , are named in the test, and each is asserted to still exist (SECURITY C-13, A-40). Drizzle Kit cannot express FORCE, so it is written in custom SQL and this probe is its only guard (POLY-76), and a pool-reuse test. That test must run on a backend that has *already* served a request: tenant A's transaction, then tenant B on the same physical connection sees zero A rows, and a context-free query on that dirty backend sees zero rows. The full battery list is SECURITY §12.3 (qa-engineer). *Shape.* The per-table probe is a catalog probe, one query shared by the real chain and its canaries. It skips temporary relations (session-private; no poly_* role holds TEMP). It fails a table that has RLS off, is not FORCEd, has no policy, has a `FOR ALL` policy or has no statement-level `BEFORE TRUNCATE` trigger, and it fails any view without `security_invoker` and any materialized view. A cross-

tenant SELECT is proven on canary tables, and each tenant table's own cross-tenant assertion lands with the feature that creates it. Detail: `process/cairn/reviews/POLY-84/ruling.md`.

3.3a Parent roll-up under RLS: totals, not rows

Access stays membership-only and is never inherited (PRD §5): membership on a parent grants nothing on its children, and the reverse. Yet a parent member must see the parent's consolidated figures, including children they hold no membership on, and drilling into such a child stops at the boundary panel (DESIGN §5.7). Plain RLS cannot serve that: under `poly_app` the child's lines are invisible, and §3.3(j) bans materialized views over tenant data.

Mechanism: one allowlisted definer function computed on read.
`app.rollup_totals(parent_entity_id, as_of, period_from, period_to)` is `STABLE SECURITY DEFINER`, `search_path = ''`, owned by a new NOLOGIN, NOBYPASSRLS role `poly_rollup`, with `EXECUTE` granted to `poly_app`. It sits on the definer allowlist.

1. It returns nothing unless `app.has_role(parent_entity_id, 'viewer')` holds for `app.current_user_id()`.
2. `poly_rollup` owns no tables. It reads `entity_rollup`, `entity` (name and type only), `ledger_account`, `journal_line`, `lot` and the reference prices through role-targeted `FOR SELECT TO poly_rollup` policies whose predicate is `entity_id = any (app.rollup_descendants())`, the recursive walk of §3.3b from the parents the current user belongs to. Even a bug in the function body can read only entities beneath those parents.
3. It returns aggregates only: one row per (entity, report line, currency) for the parent's own accounts and each entity beneath it, each entity once however many paths reach it (§3.3b), carrying the entity's display name and a `viewer_has_membership` flag, which tells the UI whether a click opens the register or the boundary panel. Report lines are account class as of `as_of`: assets, of which each of the four nature classes (§3.6; cash at its balance, the three securities classes at market) and other assets (real estate, prepaid tax and the like, at book); liabilities; net worth; plus income and expense by top-level category for the period. Never an account, entry, line, payee, memo, lot or position. **The period is any range**: the function takes whatever `period_from` / `period_to` it is given, with no snapping to whole months and no period guard. For a small entity, one-day ranges effectively reveal individual amounts and their dates; that exposure is consented, and the consent wording on the inclusion step and on the placement and ancestor-invite notices is a security requirement (SECURITY C-25, T-72, R-17), changed only through a security-gate review.
4. A companion, `app.rollup_tree(parent_entity_id)`, returns the edges beneath the parent with display names and no figures. The UI uses it to draw subtotals for nested parents, including ones the viewer holds no membership on, from the rows above.

5. The function's result shape is fixed by its `RETURNS TABLE`. A pgTAP test asserts that a parent viewer without child membership gets totals equal to the child owner's own figures, and zero rows from every tenant table for that child.

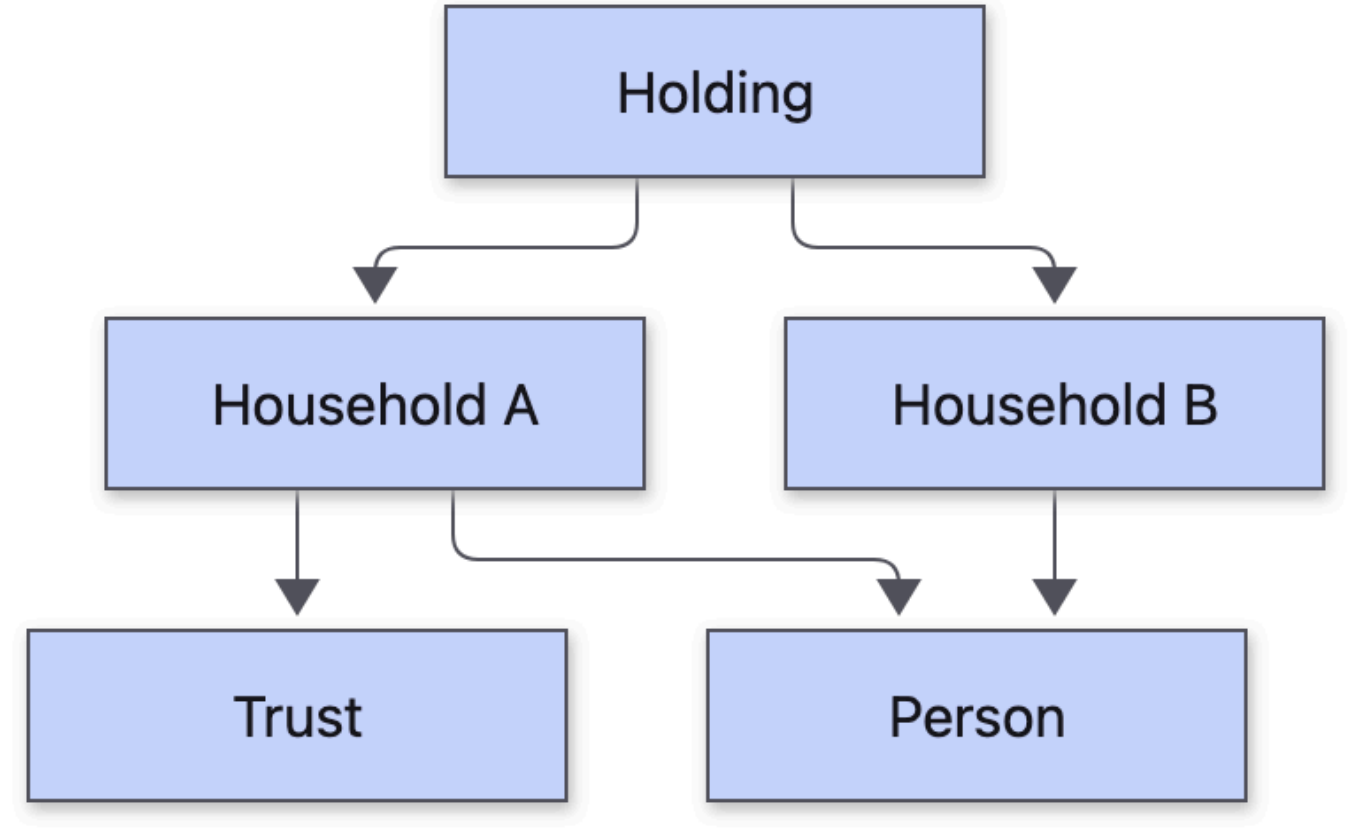
Alternative rejected: worker-materialized per-child aggregates in a tenant table keyed by the parent. A job must run under a principal holding membership (§3.3), and the child's principal is the wrong one to write the parent's rows. The cache goes stale on every posting and needs invalidation triggers. And it buys no extra isolation: the function above already confines its reads by role and by policy. Revisit only if the dashboard's <2 s NFR fails measured at parent scale (unmeasured; a household has a handful of children).

Holdings popover on a parent (DESIGN). The popover lists positions, which this function never returns. So on a parent it lists the parent's own accounts and those of entities the viewer holds a membership on, read through the ordinary RLS path, and any other included entity shows its total and the boundary panel. It belongs to the roll-up milestone; The gating meets SECURITY C-25: no definer returns a position.

Scope: the junction and its trigger (§3.3b) ship in M1 with tenancy. The consolidated view, `app.rollup_totals`, its economic sibling (§3.3c) and the popover land in POLY-1.1, the quick follow-up right after GA (§11). The function and `poly_rollup` meet SECURITY C-19 and C-25; the graph triggers belong to `poly_graph`, not `poly_rollup` (§3.3b).

3.3b Nesting: an acyclic graph, each base entity counted once

A parent may include base entities and other parents to any depth, and a base entity may sit in several parents (PRD §5). So the structure is a directed acyclic graph, not a tree, and two paths can reach one entity:



Person is reached twice from Holding and is summed once.

- **Traversal: a recursive CTE on read, no closure table.** `WITH RECURSIVE` over `entity_rollup`, using `UNION` rather than `UNION ALL`, yields each descendant once, which is the count-once rule, and it would terminate even on bad data. A closure table would read faster, but it is a second copy of the structure, and on a graph with shared entities a delete needs path counting to avoid removing a pair another path still justifies. Structures here are small (unmeasured; a household has a handful of entities) and the only reader is the roll-up, so that cost buys nothing. Revisit only if the dashboard's <2 s NFR fails measured at parent scale.
- **Membership and RLS do not depend on depth.** Access stays per entity: tenant policies call `app.visible_entities()`, which reads `membership` alone and never walks the graph. The only traversal is `app.rollup_descendants()` and the two functions above, all on the definer allowlist. Depth therefore adds no cost and no risk to any ordinary read.
- **Cycle prevention: a trigger, under a lock.** `app.check_rollup_edge()` runs BEFORE INSERT and rejects a child that is the parent or any ancestor of it. It must be `SECURITY DEFINER`, because a cycle can pass through entities the inserting user cannot see under RLS. It is owned by `poly_graph`, with `search_path = ''` and a graph-wide read of `entity_rollup`: `poly_rollup`'s reads are confined to the caller's descendants, so a check run as `poly_rollup` would miss a cycle outside them (SECURITY A-16, T-77). Two concurrent

inserts could each pass while blind to the other, so the trigger first takes `pg_advisory_xact_lock` on one fixed key. That serializes every parent-link insert on the instance; these are rare, so the cost is nil. A delete cannot create a cycle and takes no lock. UPDATE is already refused.

- **Sanity limit: 8 levels.** The same trigger rejects an insert that would make the longest path from any top parent to any entity beneath it exceed 8 links. The product sets no cap. This one is a judgement, not a measurement (unmeasured): it bounds the work of a definer function, and it sits well above the deepest structure anyone has described (Holding over Household over base entity, 2 links). It is a constant in the function; raising it is one migration.
- **Validation errors name no hidden entity.** The user sees "this would make an entity its own ancestor" or "this structure is too deep (limit 8 levels)". The refusal does disclose that some link exists, one bit about structure the user may not see; it is an accepted risk (R-15).
- **Consent travels upward, with notice.** Agreeing to a parent accepts that parent's later placement under another parent. So linking Household into a Holding shows every entity beneath Household to the Holding's members as totals. In the same transaction, `app.check_rollup_edge()` writes a `notification` to every owner beneath, which a definer must do because the inserter may not see them. Each can see the parents above (`app.parents_of`) and remove their own entity's link. That link is the direct one: a Person's owner leaves a Holding by leaving the Household inside it. The notice names the new top parent and states what becomes visible, in the consent wording (C-25), with an `audit_event`.
- **Notice when a parent gains a member (SECURITY A-21).** A second `poly_graph` trigger, `app.notify_ancestor_invite()`, runs AFTER INSERT on `invitation` on a parent-kind entity, and AFTER INSERT on `membership` on a parent-kind entity when no invitation produced the row (the operator CLI). An invitation row exists only once it is live, after any §3.1b approval, so the trigger fires at exactly the moment the audience grows. In the same transaction it walks every descendant at any depth and writes one `notification` per owner per person, deduplicated across paths and entities, naming the parent and the new member, with the consent wording and the pull-out link. Any role triggers it, since viewers see totals too; a later role raise (an UPDATE) does not; an accepted invitation's membership does not fire again. Nothing waits on the recipients.
- **Tests (pgTAP):** a diamond sums once; a direct and an indirect cycle are refused, including one through an entity invisible to the inserter, and one through an entity invisible to `poly_rollup`'s predicate; two concurrent opposing inserts leave at most one row; a ninth link is refused with the validation error; a parent viewer gets totals and zero rows from every tenant table at every depth; an invitation going live on a Holding notifies the owners of a Person two

levels down whom the inviter cannot see, once each, and a role raise does not; calling a `poly_graph` function directly fails.

3.3c The economic view: the gross roll-up, weighted by the viewer's stake

Ships with POLY-1.1 beside the gross roll-up, and reveals nothing the gross view does not.

The gross roll-up answers "what does this Household hold"; the economic view answers "what of it is mine". In a marriage with separate property, a trust one partner inherited counts fully in the gross view and not at all in the other partner's economic view.

- **Mechanism: a sibling definer, same grain.** `app.rollup_economic(parent_entity_id, as_of, period_from, period_to)`, owned by `poly_rollup`, on the allowlist, with the same guard and the same `RETURNS TABLE` as `app.rollup_totals` plus a `stake` column. It takes no Person argument: the viewer's Person is the entity whose `self_user_id = app.current_user_id()`, so nobody can request another member's economic view. The gross function, its rows and its grain (§3.3a) are the same.
- **Same set, never wider.** It weights exactly the rows `app.rollup_totals` returns for that parent and adds none, so an entity the viewer holds units in but which sits outside this parent contributes nothing here. A cross-parent "my net worth" is a later option that needs SecEng.
- **Weight of a row of entity *H***, for the viewer's Person *P*:
  - *H* = *P*: 1. Another Person: 0, since a Person's table is implicit (§3.1a).
  - Otherwise *P*'s stake in *H*, plus, for each entity holder *B* of *H*, *B*'s stake in *H* × *P*'s stake in *B*. One link only, which §3.1a guarantees is the deepest chain there is. *B*'s cap table is read whether or not *B* is beneath the parent.
  - Balances use the split in force on `as_of`. Period income and expense use the split in force on each entry's posting date, the same rule as tax allocation (§4.5), so a mid-year change splits the year.
- **Economic net worth** = Σ over the selected parent's roll-up set (the returned rows) of net worth × weight, each entity once (§3.3b). It is not the sum over every entity the Person holds units in. It is a figure of the economic view only; the gross view's net worth is unchanged.
- **Disclosure.** Dividing an economic figure by its gross figure gives the viewer's own stake in that entity, which the cap-table notice already tells them. It gives nothing about any other holder (SECURITY C-25). Like the gross view, it takes any date range, with no period guard.
- **Former holders.** A purged holder's units weight as outside owners from the purge date, and nothing looks through it on any date, since its own cap table is gone (§3.4b).
- **Tests (pgTAP):** on every row, Σ over all holders' economic figures + the outside-owners share = the gross figure; a spouse's separate trust weights 0 for the other spouse; a stake changed

mid-period splits the period's income at the change date; a viewer gets no economic row for an entity absent from the gross rows.

3.4 The privileged purge path

A sole owner's whole-entity purge (PRD §5) runs through one `SECURITY DEFINER` function, `app.purge_entity(entity_id, export_id, reauth_at)`. The function is owned by the dedicated NOLOGIN, NOBYPASSRLS role `poly_purge`, with `search_path = ''`, and `EXECUTE` is granted to `poly_app` (SECURITY §4.5).

Why not `poly_owner`: under FORCE RLS the owner is itself subject to policies. Append-only tables have no DELETE policy, so an owner-run purge would delete zero rows and still report success (T-51).

How the deletes are allowed: each purgeable table carries `CREATE POLICY purge … FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), '')::uuid)`. The append-only triggers allow DELETE only when `current_user = 'poly_purge'` and the row's entity matches. `poly_app` can set the GUC but can never become `poly_purge`.

The roll-up junction has two entity columns, so its purge policy matches either: `parent_entity_id = … or child_entity_id = …`. Purging a base entity removes it from every parent it sat in, and each parent's owners get a notification. Purging a parent removes its own books and its junction rows, upward and downward; everything beneath it stays intact, and a nested parent simply loses that link. **The cap table is the opposite case:** `equity_*`'s purge policy matches `entity_id` only, never `holder_entity_id`, because the rows naming the purged entity as a holder are the other entity's history. Those rows stay, and the purged entity's row stays as a tombstone so they resolve (§3.4b). Pending `change_request`s on the entity are deleted with it (a purge needs a sole owner, so none can be awaiting another owner); their audit events survive.

The guards run in this order, in one transaction:

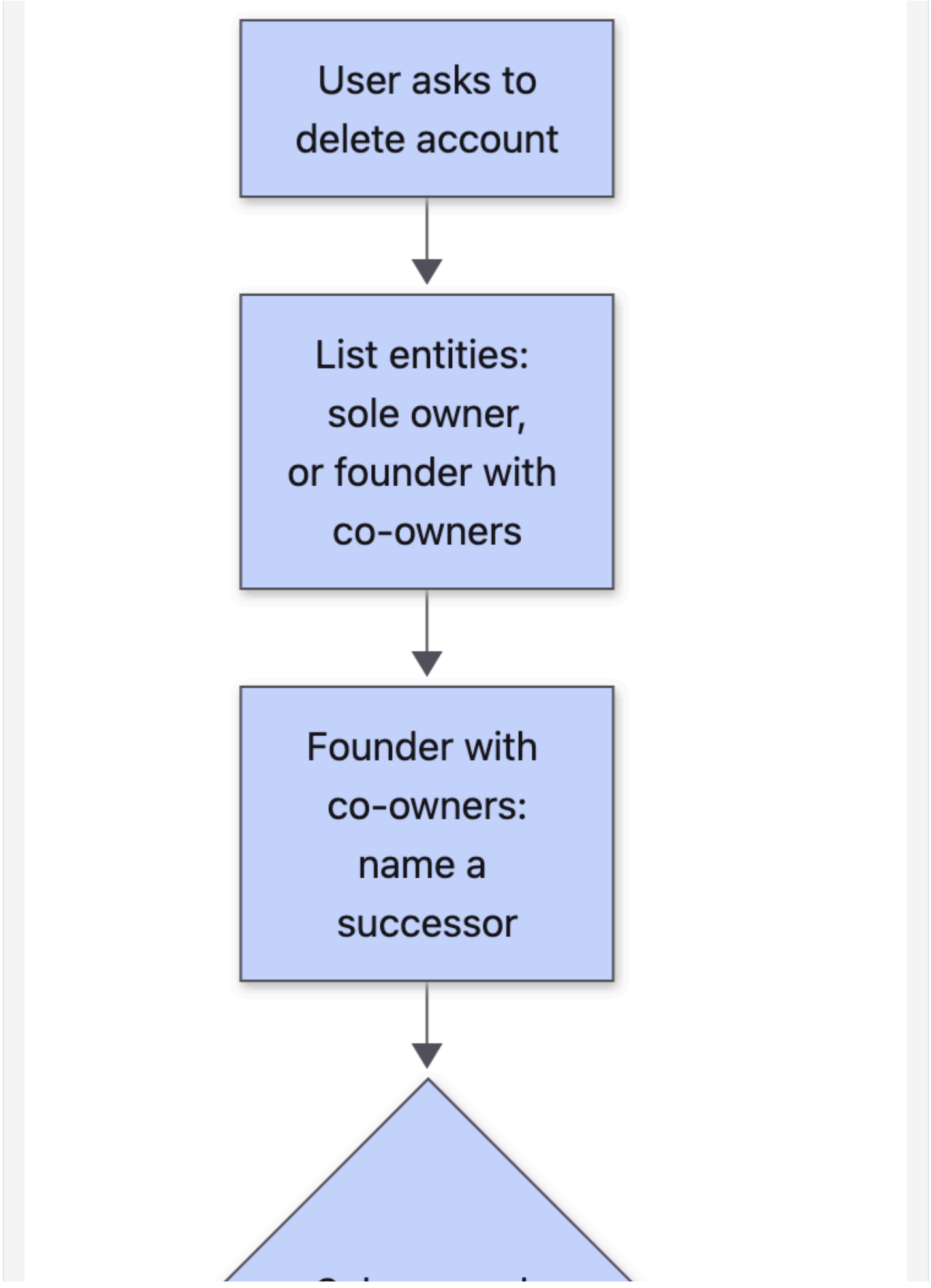
1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
2. Sole-owner check against `app.current_user_id()`.
3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
4. No tenant write to the entity since the export's snapshot marker. The entity row is locked `FOR UPDATE`.

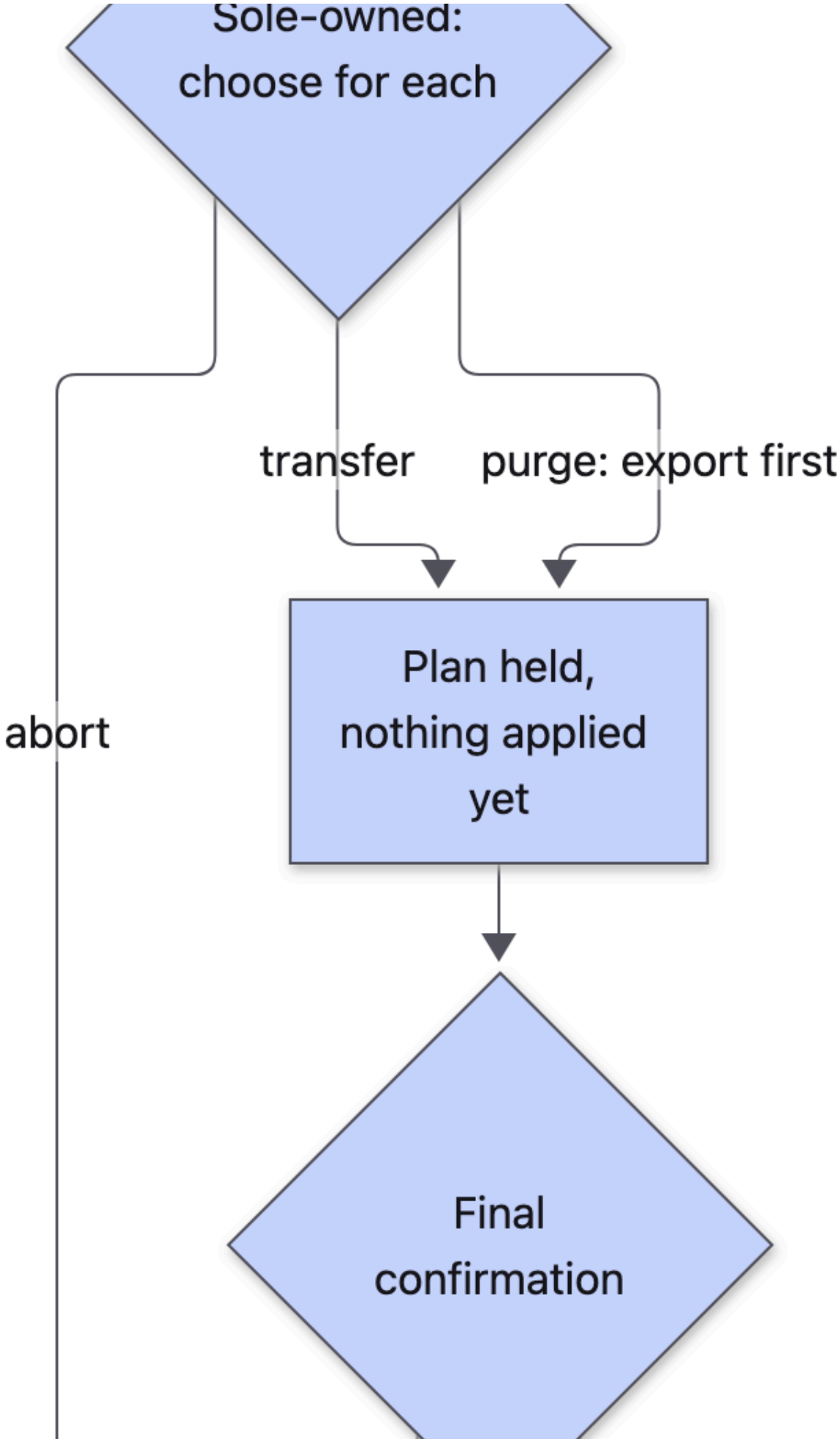
5. An `entity.purged` `audit_event` is written before the first delete. It records the actor, entity id and name, the export event, per-table row counts, the final chain head (§3.2.9) and the ids of the entities on whose cap tables it was a holder (§3.4b). `audit_event` has no FK to `entity` and no purge policy.
6. Every connection has been revoked at the provider by the worker revoke job. A failed revocation blocks the purge.
7. Set `app.purge_entity` transaction-locally and delete in dependency order, attachments, `journal_chain` and the counter included. Then reduce the `entity` row to its tombstone (§3.4b).
8. Assert zero rows remain for the entity in every purgeable table before commit, and that the `entity` row is in tombstone shape (fail loud). `entity` is not a purgeable table: it has no purge DELETE policy, so the catalog-derived list of purgeable tables never contains it, and the tombstone assert covers it instead.

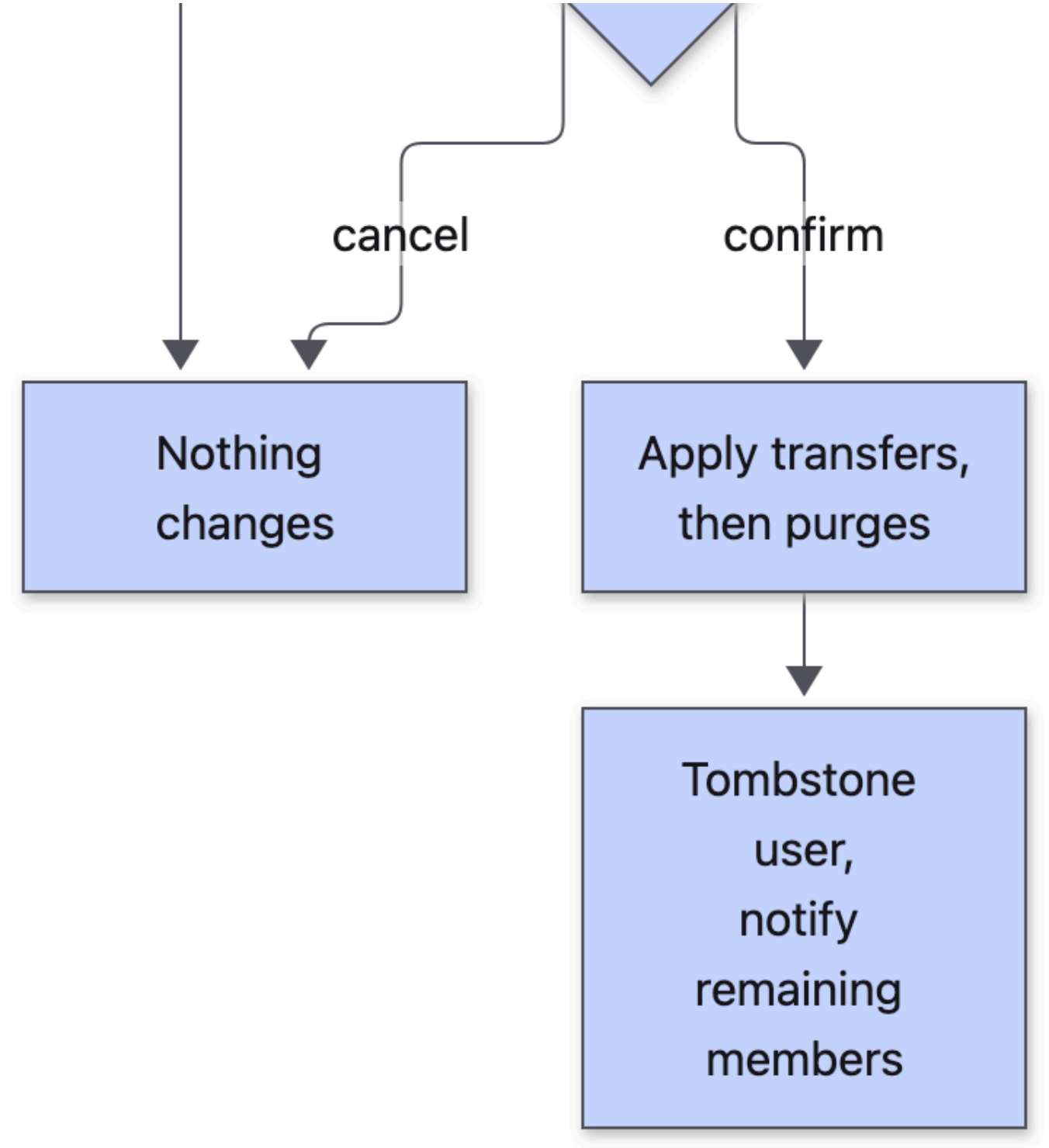
Not permitted: purge by a job; purge through the API without typed confirmation of the entity name; purge of a partial range; purge while a second owner exists. **What the confirmation says about backups** (SECURITY A-35, R-5): purged data stays "up to 35 days in backups", the bucket's fixed retention (§7.2); the store cannot delete it sooner, and there is no long-term archive tier in v1. The same words appear on each purge choice in account deletion (§3.4a). **Rejected:** `DISABLE TRIGGER` (DDL, affects every tenant), tombstoning the entity's data in place of deleting it (breaks PRD §5; the id-only tombstone of §3.4b holds no data), `BYPASSRLS` (broader than needed). **No execution delay**: the guards above stand, operator backups remain, and every member is emailed immediately, before the entity's own rows go.

3.4a Account deletion in-flow

PRD §5. Deleting an account resolves each entity it would orphan in the same flow. The invoker of every step is the account being deleted, signed in with a fresh re-authentication.


User asks to delete account
List entities: sole owner, or founder with co-owners
Founder with co-owners: name a successor


Sole-owned:
choose for each
transfer
purge: export first
abort
Plan held,
nothing applied
yet
Final
confirmation



1. **Choices come first, effects last.** A purge cannot be undone, and "abort the whole deletion" must mean what it says. So the flow collects one choice per sole-owned entity without applying any; exports for the purge choices are produced here. Abort at any point before the final confirmation changes nothing. Transfer is never preselected.
2. **Founder succession.** For each entity the user founded that has other owners, the flow requires a successor named among those owners; it is applied with the transfers, through `app.transfer_founder`, and every owner is notified (§3.1b, SECURITY §4.6). An entity they co-own but did not found needs no step: they simply leave it.

3. **Transfer** makes a named existing member an owner, in one transaction, with an `audit_event`. It names a current member who is not mid-deletion themselves and holds a second step (`second_step_at` set), so no invitation is pending at deletion time and the new owner role is in force at once; a member without one is listed as needing it, since a held role would leave the entity with no owner and the founder trigger would refuse the commit (§3.1c). The entity is sole-owned, so the transfer is a sole-owner action needing no approval, and it sets the recipient as founding owner in the same transaction.
4. **Purge** runs `app.purge_entity` unchanged, per entity, one transaction each, with every §3.4 guard. The invoker still holds owner at that moment, so the sole-owner check passes on its own terms. The five-minute re-authentication window restarts the prompt if the run outlasts it. No job purges on the user's behalf.
5. **A failed provider revocation stops the run** (guard 6). Entities not yet reached stay as they were, entities already purged stay purged, and the account is not deleted. The user sees which and retries. Full atomicity is not available, since revocation is an outside call made by the worker.
6. **Tombstone last.** One transaction locks the user's `membership` rows and re-checks that no sole-owned entity remains, so a change made during the flow cannot orphan an entity. It also refuses while any entity with another owner still names the user as founder. It then applies §3.1: tombstone, credentials, sessions and memberships removed, connections `principal_invalid`, and every pending request on their entities re-evaluated (§3.1b). While the flow runs they remain an owner and may still approve or decline. The deletion's `audit_event` and the `entity.purged` events keep the tombstoned user id, which still resolves because the row is never deleted.
7. **Departing owner on a shared entity.** The same transaction writes a `notification` for every remaining member of each entity that keeps another owner, plus an `audit_event`. Nothing waits on them. Any other removal of an owner writes the same notification (PRD §5).

An account with no second step (SECURITY A-30, §4.6). A departing user gains nothing from a factor they would use once, so the flow does not make them set one up. They confirm with the password (or a fresh IdP round trip) plus an `email_challenge` of purpose `account_deletion` (§3.1c: the same send and confirm definers, 10 minutes, single use, 5 tries, 3 sends per 15 minutes, bound to the session). Confirming writes no `enrolment_grant` and never sets the session's `recent_auth_at`; it stamps `reauth_at` on the deletion plan itself, which is already held server-side and bound to its session. That stamp authorises only this deletion run, including the own Person's export and purge inside it, and no other step-up action; it opens the same five-minute window, and a run that outlasts it asks for the password and a new code. The tombstone transaction refuses an unenrolled user unless the plan carries a live stamp. Everything else in the flow is unchanged, and enrolled users use their factor as before. Such a user holds owner only on

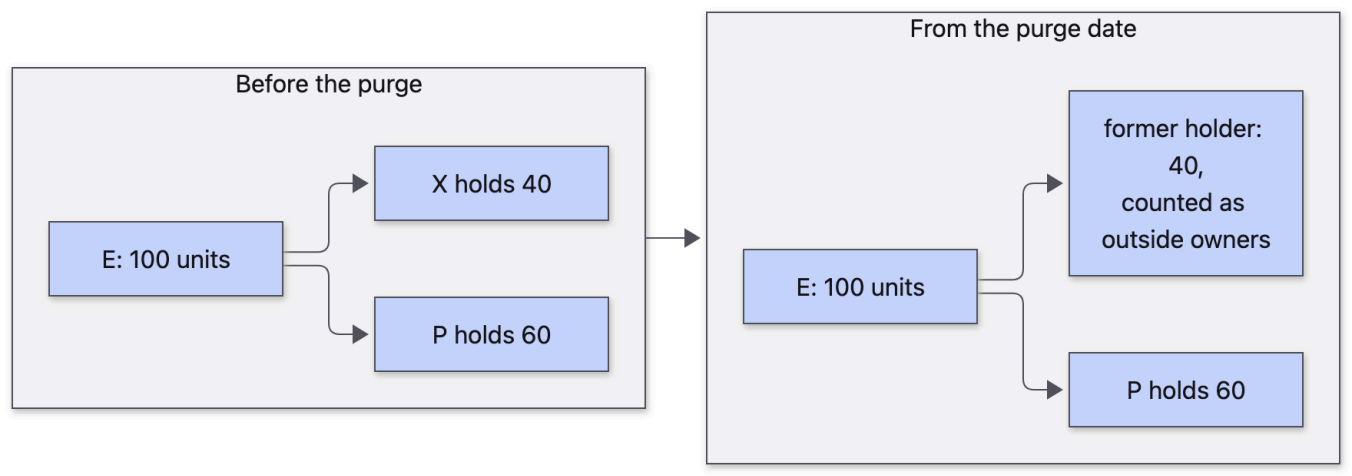
an unshared own Person (§3.1c), so the transfer and founder steps do not arise; a user the operator reset who still holds owner elsewhere meets the sign-in gate first.

Who invokes: the account being deleted, and only it, for every step: signed in, with a fresh step-up including the second factor for an owner (password or IdP plus the emailed code for an unenrolled user, above); never a job, another member or the operator CLI. The plan is held server-side and bound to the session that made it; another session starts again from the list (SECURITY §4.6, C-52).

3.4b Purging an entity that holds units on other cap tables

SECURITY A-19; before POLY-1.0.

Entity *X* holds units in *E*: *E*'s `equity_holding` rows name `holder_entity_id = X`. Those rows are keyed to *E*, append-only, and part of *E*'s history, so purging *X* must not touch them. It must not leave them dangling either, and *X*'s share must go somewhere.



- **A tombstone, always.** Every purge reduces *X*'s `entity` row to `id` and `purged_at`, every other column null (name, type, self_user_id, founder, tax attributes), instead of deleting it; a `CHECK` admits that shape only when `purged_at` is set. Always, not only when *X* is referenced: one code path, one deterministic assert, and a referencing row written concurrently cannot slip between a "referenced?" check and the delete. The cost is one id-only row per purged entity, which says only that some entity existed and when it was purged. The PRD §5 promise holds: every row of *X*'s data is gone. It is the same move as the user tombstone (§3.1).
- **How the tombstone is written.** `poly_purge` gains one role-targeted policy, `FOR UPDATE TO poly_purge USING (id = purge GUC)`, with UPDATE granted only on the columns it nulls

and `purged_at`. No DELETE policy exists on `entity`. `holder_entity_id` FKs `entity` `ON DELETE NO ACTION`, so a bug that tried to delete the row instead fails loudly.

- **The append-only trigger is untouched.** *E*'s rows are neither updated nor deleted, and *E* gains no rows. So no exception is added to the `equity_*` trigger, the Σ invariant still holds as written, and no `cap_table` request on *E* is needed: *X*'s owner purging their own entity is not an edit of *E*'s cap table.
- **The share moves by a read rule, not a write.** The split on date *d* (§3.1a) adds one clause: a holder whose entity has `purged_at::date ≤ d` contributes 0, so its units fall into outside owners = total − Σ live units. One SQL function computes the split, and the economic view (§3.3c), allocated income and tax allocation (§4.5) all call it. Before the purge date the rows stand as recorded and the share remains the former holder's, so *E*'s past closes and reports still agree with themselves. Closes are never reopened, so a close already posted is unchanged.
- **Look-through ends on every date.** If *X* was an entity holder with its own holders, *X*'s cap table is purged with it, so nobody's economic weight in *E* through *X* can be computed for any date, even before the purge. The economic view therefore shows *X*'s units as outside owners for a past `as_of` too. That is a deliberate inconsistency with *E*'s direct split, bounded to the look-through path, and the view says so when it applies. Keeping *X*'s cap table past the purge would break the PRD §5 promise.
- **No new rows naming a former holder.** `app.check_equity_holding()` refuses any new row whose holder is tombstoned. *E*'s owners reassign the units by an ordinary edit, a new row for another holder (through a `cap_table` request if *E* has several owners); a correction of *X*'s units before the purge date is not possible after it. A pending `cap_table` request on *E* whose payload names *X* fails re-validation at apply and is declined as stale.
- **The cycle and depth checks** keep treating the former edges as edges, which is harmless: a tombstoned entity can never hold or be held again.
- **Notice.** The tombstone UPDATE fires a `poly_graph` trigger, `app.notify_former_holder()`, AFTER UPDATE OF `purged_at`, which writes one `notification` to each owner of each entity whose cap table named *X*, in the purge's transaction. `poly_graph` already holds the graph-wide `equity_holding` read, owner `membership` SELECT and `notification` INSERT this needs, so `poly_purge` gains no read of other entities' rows. *X*'s own members are emailed as every purge begins; *E*'s owners are the other side.
- **What *E*'s owners see in place of the name.** The system shows "Former holder (purged <date>)" and copies nothing of *X*: the tombstone holds no name, and no snapshot is taken at purge. *E*'s owners may add an optional label they type themselves, recording what they already know. The label is *E*'s own data:

    - **Shape.** `equity_holder_label(entity_id, holder_entity_id, label, recorded_at)`, keyed to *E*, append-only like the rest of the cap table; the latest row recorded for a holder wins, and clearing writes an empty label. A trigger admits a row only for a tombstoned holder, so a live holder's name always comes from its entity.
    - **Who writes.** It is a cap-table edit: owner of *E* only, direct while *E* has one owner, a `cap_table change_request` otherwise (§3.1b). The multi-owner gate covers `equity_holder_label` along with `equity_*`, and `poly_govern` gets INSERT on it.
    - **Who reads.** Members of *E*, by ordinary RLS, like *E*'s other cap-table rows.
    - **Purge.** Purged only with *E*: its purge policy matches `entity_id`, never `holder_entity_id`, so purging *X* never touches it.
- **Tests (pgTAP):** purging *X* leaves *E*'s `equity_holding` rows unchanged and resolvable; the zero-rows assert passes with the tombstone present and fails if any other column is non-null; *E*'s split on the day before the purge includes *X* and on the purge date counts its units as outside; Σ economic + outside = gross still holds for *E* after the purge; a new row naming *X* is refused; a pending `cap_table` request naming *X* is declined as stale; each owner of *E*, including one the purging user cannot see, gets one notification; deleting the tombstone row fails; the row shows "Former holder (purged <date>)" with no trace of *X*'s name; a label for a live holder is refused; with two owners, a direct label INSERT is refused; purging *X* leaves *E*'s labels intact.

3.5 Attachments

PRD §5 includes attachments (statements, receipts, closing documents) in export and purge, but v1 had no storage design (SECURITY A-14). The design:

- **Location:** Postgres `bytea` in an entity-scoped `attachment` table. It is RLS-covered like every other tenant table, and it automatically falls inside backup, export and purge. It works unchanged on every hosting target, including Vercel + Neon.
    - *Rejected for v1:* a filesystem or S3-compatible object store. That would be a second access-control system to keep in lockstep with RLS, and a second thing to back up, export and purge.
    - *Cost:* database size. At household scale, a few hundred MB per year is expected (unmeasured).
    - *Revisit* if the attachment table exceeds a size set in INFRA.
- **Limits:** 10 MB per file. Type allowlist: PDF, PNG, JPEG, and CSV/OFX (kept as import evidence). The type is sniffed server-side, and the client-declared type is never trusted. A SHA-256 is stored for de-dup and for the export manifest.

- **Serving:** only through an app route inside `withTenant`, with `Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`, the sniffed `Content-Type`, and a restrictive CSP. Nothing is rendered inline.
- **Linking:** `entry_attachment` links an attachment to a journal entry, draft or account. A link doesn't change the entry, so attaching to a posted entry is allowed. Unlinking is manager+ and writes an `audit_event`. Attachment bytes themselves are insert-only.
- **Boundary:** uploads cross TB-11 (untrusted file ↔ parser). Attachments are stored, never parsed. CSV/OFX import files are parsed by the import adapter under TB-11's controls.

SecEng conditions (approved on security, all binding):

1. *Filename:* `Content-Disposition` uses the RFC 6266 `filename*=UTF-8''` form, with CR/LF and quotes stripped. The CSP is `default-src 'none'; sandbox`.
2. *Entity binding:* `attachment` and `entry_attachment` both carry `entity_id`, with composite FKs, so a link can never cross entities. Viewers read; manager+ uploads.
3. *Purge:* both tables get the `poly_purge` DELETE policy and the append-only trigger exception, and are included in the zero-rows-remain assertion (C-50).
4. *Quota:* a per-entity storage quota and an upload rate limit protect the disk. The numbers are set in INFRA.
5. *Isolation:* the bytes live in their own table (`attachment_blob`, 1:1 with `attachment`), so list and DTO queries never select `bytea`. Content is never parsed, and no AV scanning is an accepted consequence of that.

3.6 Custodial accounts and the chart by nature

The ledger is organised by nature, with a custodian sub-account under each nature account.

- **Chart.** Under Assets sit the four nature classes that are also Allocation's top level: Cash and cash equivalents, Fixed income securities, Marketable securities, Alternative investment assets. Each has a sub-account per custodian ("Marketable securities, Schwab brokerage").
- **Sub-account naming.** A custodian sub-account is named institution + account type ("Schwab brokerage"). When one institution holds two custodials of the same type, the number mask is appended ("Schwab brokerage ··4821"); `posting` adds the mask to the existing sibling in the same transaction that creates the second, so neither name is ambiguous. The `custodial_account` keeps its own name, which the user may edit freely; the ledger name is derived once and is mutable after posting like any account name (§3.1).
- **Real estate and other no-custodian assets** sit under Assets beside the four nature classes, not inside them: a `ledger_account` with `nature_class` null and no

`custodial_account_id`, one per property, like Prepaid tax but with no authority behind it. It has no lots, no provider link and no reconciliation, and is carried at the value the user books (an opening balance, then manual entries). It counts in the balance sheet, net worth and the roll-up's "other assets" line (§3.3a), and never in Allocation's classes, targets or percentages.

- **Mapping is one-to-many.** A ledger `account` carries `custodial_account_id`; a custodial owns one ledger account per nature class it holds, enforced by a partial unique index on `(custodial_account_id, nature_class)`. A checking account owns exactly one, and a credit card one liability account with a null nature class. There is no junction: a ledger sub-account never spans two custodials. `posting` creates a missing sub-account in the transaction that first needs it.
- **Tax authorities** are custodial accounts with no provider link. Their prepaid-by-year and refund accounts (§4.5) have a null nature class, so the index does not limit them.
- **Lots** reference the custodial and the security. The security's class picks which of the custodial's ledger accounts its basis posts to.
- **Reclassifying a held security** moves its basis between nature accounts, so the mapping change posts that entry in the same transaction. The rulings leave this case open; this keeps §3.2.8 true.
- **Money-market funds** in the Cash class are carried as cash with no lots, at their stable value. §3.2.8 therefore covers the three securities natures only, and the cash figure below stays one balance.
- **The money-market sub-category** under Cash in Allocation therefore has no lot source. It reads the latest `reported_position` on that custodial of a security mapped to the sub-category, at the provider's reported value; the rest of the cash balance is plain cash. The class total is still the ledger cash balance, so the two always sum to it. A reported figure above the cash balance shows as a reconciliation gap, never as negative cash, and a custodial with no reported positions (a file or manual source) shows its whole balance as cash.

Roll-up per custodial:

- Book = the sum of its ledger account balances.
- Cash = its cash account balance.
- Market = cash + the sum of lot quantity × latest price over lots held there.
- Unrealized = market − book.

Reconciliation compares each ledger account against the reported balances and positions of the mapped provider account. Allocation reads the nature accounts across custodials (book) or lots by the security's class (market), and each ties to its own total by construction. Sub-categories

come from `security_allocation` on lots (and, for money-market, on reported positions), never from the ledger. Accounts with no nature class are outside Allocation altogether.

Entries:

Event	Journal entry	Sub-ledger
Purchase	Dr securities nature / custodian; Cr cash / custodian	One lot
Sale	Dr cash / custodian; Cr securities nature / custodian for matched basis; Cr or Dr realized gain or loss	One `lot_match` batch, kind `sale`
Dividend	Dr cash; Cr dividend income	None
Return of capital	Dr cash; Cr securities nature	Lot adjustment
Bond premium amortization	Dr interest income; Cr fixed income	Lot adjustment
Inter-custodial transfer	Moves basis between the two sub-accounts, no gain	Re-homes the lots

Re-homing keeps lots append-only. A transfer never UPDATEs a lot. It closes the old lot with a `lot_match` of kind `transfer`, carrying no proceeds, and writes a successor at the new custodial with the original acquisition date and basis and a `transferred_from_lot_id`. Holding period survives. A "Correct entry" on an entry that wrote a lot compensates through `lot_adjustment` or a reversing match in the same transaction.

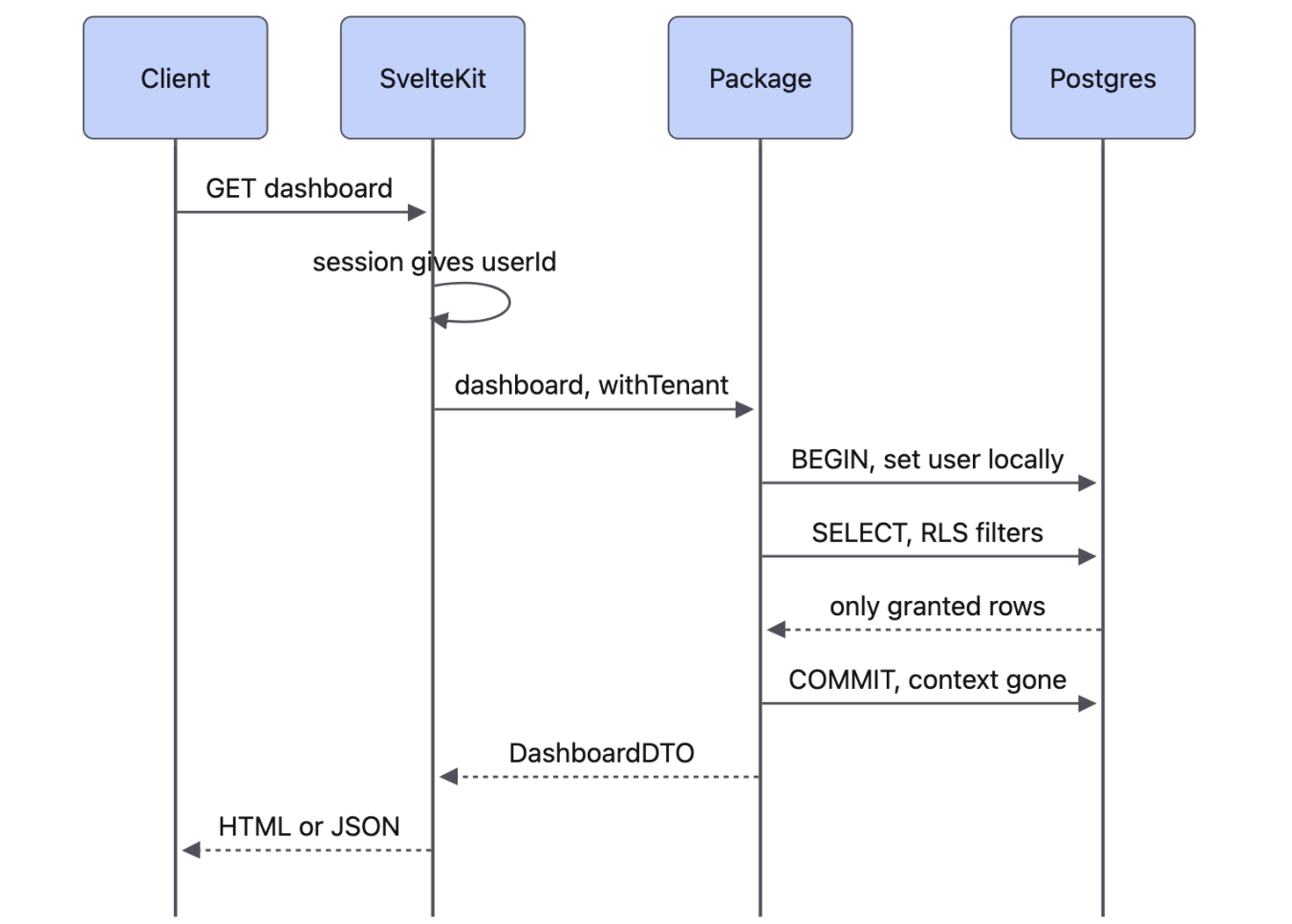
Imports from a brokerage split by nature before posting, using the security mapping. The custodial cash leg stays locked on the draft (G5).

Reporting facts are stored, not derived. `lot_match` holds proceeds and per-lot basis as values, so a later price or FX correction never changes a filed year. Form 8949 = lot-match rows of kind `sale` grouped by holding period, with adjustments from `lot_adjustment`. The trade blotter = lots (buys) + matches (sells), joined to their entries. Realized-gain income accounts split by holding period only, short-term and long-term, so ledger totals tie to the 8949 sections without reclassification. Jurisdiction lives on the tax side: one gain posts once, and the close charges tax on it per jurisdiction (§4.5).

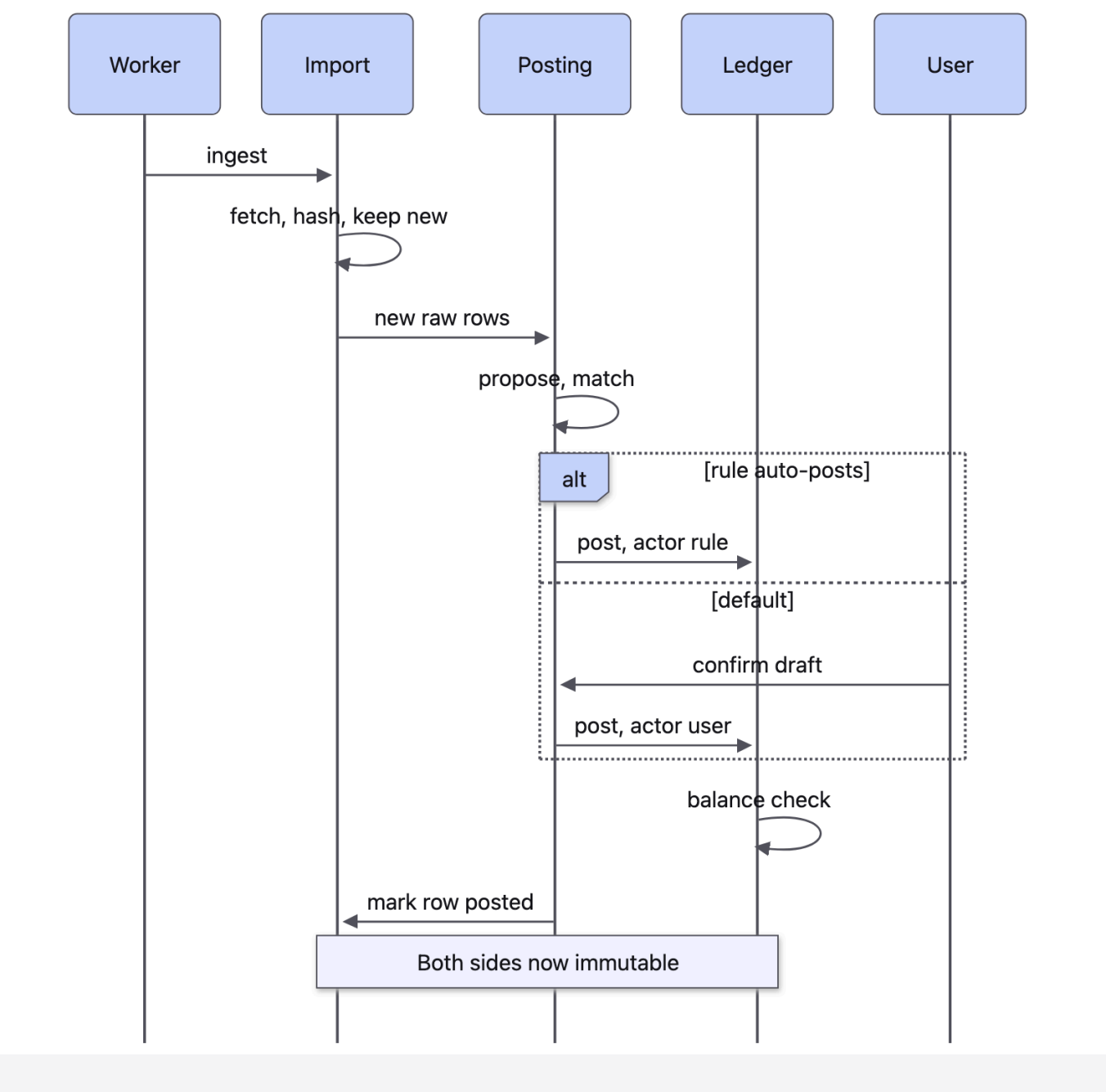
4. Data Flow

Only flows that surface a non-trivial design choice.

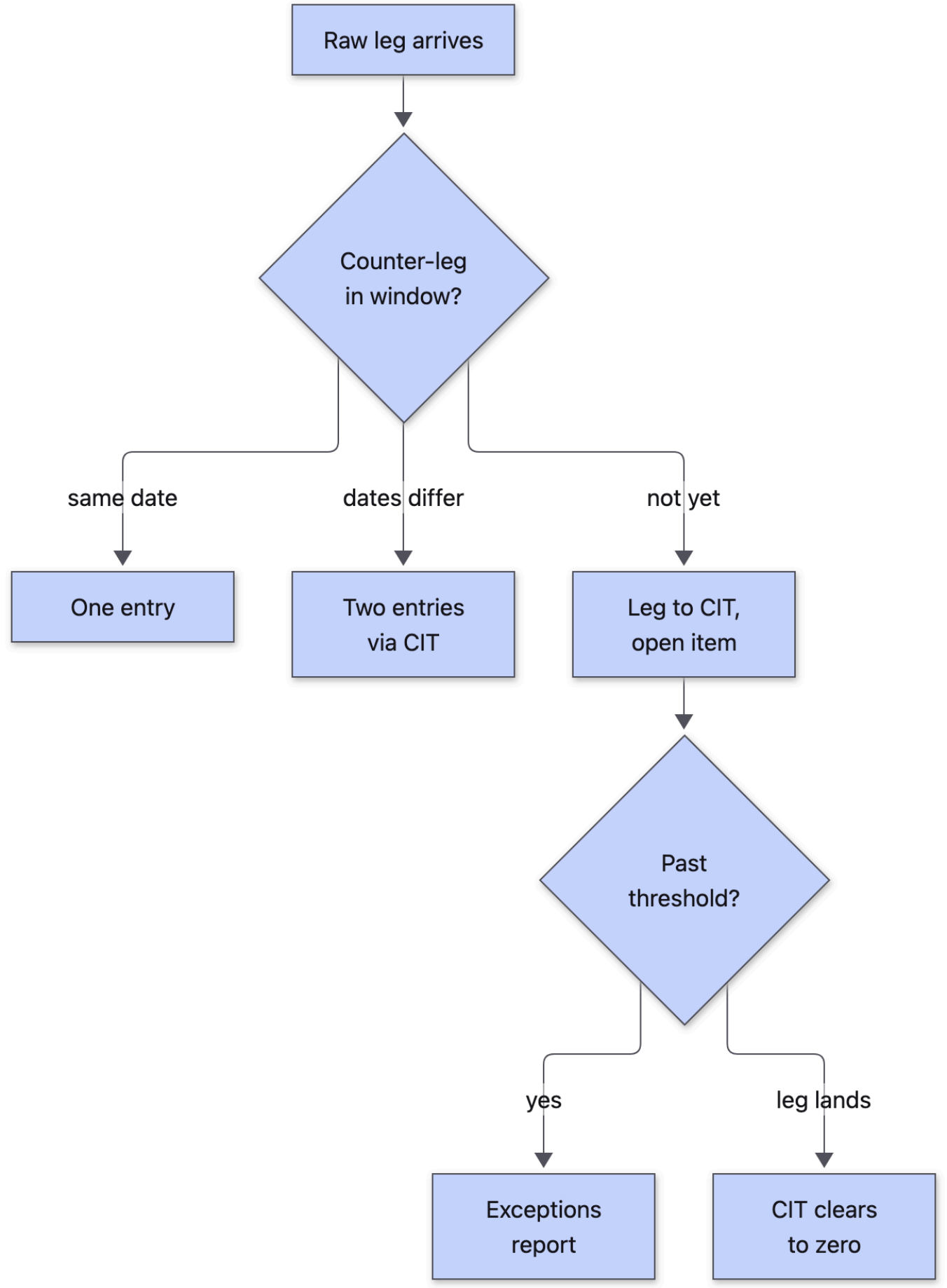
4.1 Any authenticated read (PRD Story 1, 8)



4.2 Import → draft → confirm (PRD Story 2, 3)



4.3 Transfer matching and Cash in Transit (PRD §5)

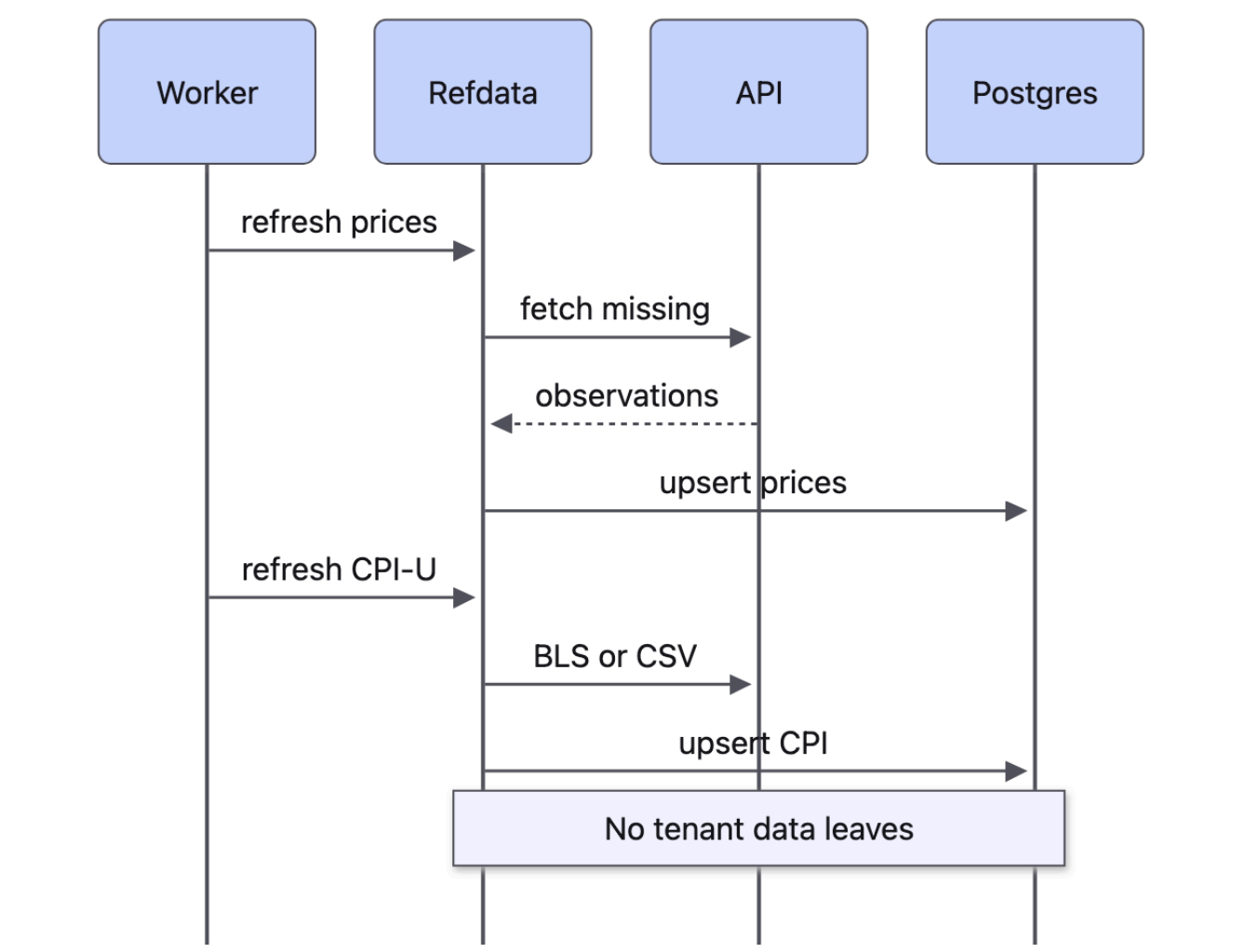

Raw leg arrives
Counter-leg in window?
same date
dates differ
not yet
One entry
Two entries via CIT
Leg to CIT, open item
Past threshold?
yes
leg lands
Exceptions report
CIT clears to zero

Only legs between two of the entity's own linked accounts are transfer candidates. A payment to an outside party has no counter-leg and never enters matching or Cash in Transit. That includes an estimated-tax payment made at the bank: it is categorized and posted like any other draft (§4.5). Matching lives in `posting` (the import side of posting), never inside `ledger.postEntry()`.

Between two entities, the same transfer posts one entry per entity, and the user picks the treatment per transfer on the draft: *contribution / distribution*, preselected, or *due-to / due-from*, which expects repayment. `transfer_match` stores the choice. Pairing and posting both sides needs manager on both entities. The choice is explained in plain language at that point (DESIGN).

Cash-in-Transit aging is computed on read by the exceptions report, not by a job (SECURITY §4.4).

4.4 Reference data: EOD prices and CPI-U (PRD Stories 5, 9)



4.5 Estimated-tax accrual (PRD Story 4)

No sequence diagram. `tax.estimate(entity, jurisdiction, asOf)` is a pure function over (a) year-to-date income by tax-treatment tag from the ledger, (b) the tax table row for that jurisdiction and filing year, (c) prior-year filed tax, typed per entity, per jurisdiction, per year, with the ledger's settled-filing figure shown beside it as a check once it exists, and (d) the balance of the custodial prepaid-tax sub-account for that jurisdiction and filing year, written *P* below. It returns:

- **Pending liability** = annualized tax to date − *P*.
- **Both safe-harbor values** and the **required payment to date** (the lesser of the two cumulative safe-harbor amounts).
- **Gap** = required payment to date − *P*, which is what is still owed by the next due date.

A job unit calls `freeze()` the day after each period cutoff and appends a `tax_period_freeze` row. The estimate is recomputed on read and never stored, except the frozen period figures. These freezes follow the authority's installment cutoffs and post nothing; the reporting-period closes below are separate.

The app never pays tax. v1 initiates no payment. The user pays the tax authority at their bank, and the payment arrives as an ordinary imported transaction with no counterparty leg. Nothing is accrued between closes: the pending liability is a computed figure, and the tax liability account is touched only at a period close and at filing.

Accounts, seeded per entity for each jurisdiction *J*:

- *Prepaid tax — J*: an asset, held at the authority's `custodial_account`. It has one sub-account per filing year *Y*.
- *Income tax payable — J*: a liability.
- *Tax refund receivable — J*: an asset, held at the same custodial account.
- *Income tax expense — J*: an expense.

Grantor trusts and pass-throughs use the entity's existing *Distributions / owner draws* equity account in place of the expense account (PRD §5 tax-classification routing).

Who accrues: a taxpayer entity accrues its own tax at each close. That means an individual, a non-grantor trust, a C-corp, or a parent classified `joint filers`. A grantor trust, a pass-through business or a parent classified `pass-through to members` accrues nothing: its tagged income allocates to its holders by the cap table (§3.1a), and they accrue. A parent classified `joint`

`filers` ignores its cap table for tax: it accrues its whole income itself. A filing group combining a Household and its Persons on one joint return is not in v1.

Allocation by the cap table.

- **The split in force on each entry's date.** Each tagged income line of a pass-through entity *E* allocates to each holder by that holder's stake in *E* on the entry's posting date (latest rows with `effective_from ≤ posting_date`). A holder who joins in July takes nothing from June's income.
- **One level onward.** A holder that is an entity *B* is treated by its own classification. A taxpayer *B* (C-corp, non-grantor trust) adds the allocation to its own estimate and accrues it. A pass-through or grantor *B* allocates it onward to its holders by their stakes in *B* on the same posting date. Those holders are Persons, because §3.1a refuses a deeper chain, so nothing is left unallocated except the outside owners' share.
- **Outside owners** take their share off-system: it allocates to no tracked entity. With the old share table that was an error (shares had to total 100%); now it is the normal case of an untracked partner.
- **The read.** `tax.estimate` for any taxpayer entity, the Person-estimate definer included, adds allocated income through one allowlisted definer, `app.allocated_income(taxpayer_entity_id, from, to)`: income-by-tag totals × stake, never rows, because a holder's owner may hold no membership on the entity. It is owned by `poly_stake`, guarded by a membership on the taxpayer entity, and its reads are confined by role-targeted policies to `app.held_entities()` (SECURITY C-27, A-18). It reads `equity_holding` and `equity_total` for the two entities in the chain and nothing more. Any date range is accepted, with no period guard. A purged holder's share is outside owners from the purge date (§3.4b).
- **A cap-table change after a close** is late data: closes are never reopened, and the next close's delta carries it, as for a back-dated import.

Why a sub-account per filing year: a January Q4 payment belongs to the prior year. When a payment draft is created, the rule proposes the filing year from the due-date table (the year whose next unpaid due date the payment falls on or before), and the user can change it before confirming.

Reporting-period close. The accrual is trued up at each close, monthly by default, per the entity's cadence. Tax is a real liability, so net assets should not show it as one lump a year; and tax is nonlinear, so accruing per entry would mean correcting earlier entries.

- **The close.** At period end a job unit raises a notification; it never posts. The user confirms one entry per jurisdiction, dated the period end. `tax.close()` writes the entry and its `tax_close` row in one transaction.
- **The amount** is *A* = the annualized tax to date from `tax.estimate` at the period end, before prepaid is subtracted, because Prepaid tax stays a separate asset on the balance sheet. Using the pending liability, which already nets out *P*, would count prepaid twice.
- **The delta** is *A* − the last `accrued_to_date` for that filing year, so the first close of a year posts the full figure and later ones the change. It is taken from `tax_close`, not the payable balance, because in the months before filing that one account also holds the prior year's accrual.
- **Late data.** A back-dated import after a close is picked up by the next close's delta. Closes are never reopened. A skipped period needs no catch-up, and a cadence change applies from the next period end.
- **A wrong close** is fixed by closing that period again: a new revision posts the difference. "Correct entry" is not offered, so the record and the ledger cannot part.
- **Between closes,** the balance sheet and NAV reports add one computed line, "estimated tax since last close": `tax.sinceClose()`, the estimate at the report date minus the last `accrued_to_date` on or before it, across unfiled years. Net assets are right on any day with no posting on read. The line is labelled as an estimate, is not a ledger balance, and stays out of the trial balance.
- **Year end** is simply the last close of the year. The line and the status "last closed through *date*" arrive with tax in POLY-1.0; the POLY-M5 statements render without them until then.

Event	Source	Journal entry
Between closes	none	No accrual entry. The pending liability and the since-close line are computed on read.
Estimated payment	Imported bank debit, confirmed by the user (or a rule the user opted in)	Dr Prepaid tax — J — Y / Cr Bank
Period close in Y (to-date tax *A*, last accrued *E*), one entry per jurisdiction	User-confirmed; taxpayer entities only	Dr Income tax expense — J / Cr Income tax payable — J (*A* − *E*), reversed if the estimate fell. A pass-through or grantor entity posts nothing.

Event	Source	Journal entry
Filing (actual tax *T* for year Y, accrued through the last close *E*, prepaid balance *P*)	Entered by the user from the filed return; one multi-leg entry dated the filing date	True-up: Dr Income tax expense — J (*T* − *E*) / Cr Income tax payable — J, reversed if *E* > *T*. Settle: Dr Income tax payable — J / Cr Prepaid tax — J — Y min(*T*, *P*), leaving *T* − *P* payable if *T* > *P*. A pass-through or grantor entity that prepaid has no payable: Dr Distributions / Cr Prepaid tax — J — Y. If *P* > *T* and the user elects a refund: Dr Tax refund receivable — J (*P* − *T*) / Cr Prepaid tax — J — Y (*P* − *T*). If *P* > *T* and the user elects to apply it to next year: Dr Prepaid tax — J — Y+1 / Cr Prepaid tax — J — Y (*P* − *T*). Penalty or interest, if any: Dr Tax penalties expense / Cr Income tax payable.
Balance due paid	Imported bank debit, confirmed	Dr Income tax payable — J / Cr Bank
Refund received	Imported bank credit, confirmed	Dr Bank / Cr Tax refund receivable — J

After filing and settlement, Prepaid tax — J — Y and Income tax payable — J both read zero. The liability is recognized at each close, trued up to the return at filing, and settled from the prepaid account, so any remainder is the true balance due. Custodial reconciliation (POLY-72) can compare the prepaid sub-account against the authority's account transcript. A filing entry made in error is fixed with "Correct entry", like any other entry.

5. Tech Stack & Rationale

Justify every row. "We know it" is a valid reason — write it down.

Layer	Choice	Why	Alternatives considered
Language	TypeScript (strict) end to end, Node LTS	One language across UI, API, jobs and schema; the Principal's prior project (mosko-fintech) is TS/SvelteKit.	Go / Python backend (a second language for no gain at this scale)
Frontend framework	SvelteKit 3 + Tailwind 4 + `bits-ui` (the headless primitives under shadcn-svelte; components are owned source in `apps/web`); LayerChart. Kit 3 has no `svelte.config.js`: the kit options, `kit.csp` among them (SECURITY C-07), are passed to `sveltekit()` in `apps/web/vite.config.ts`. Imports inside `apps/web` are relative, not `$lib`, so the layer resolver (§2.1) sees every path.	Kickoff §2.9: owned component source, headless primitives; shadcn-svelte charts are LayerChart. Monarch/Origin feel from dashboard-01.	Flowbite Svelte (rejected: fixed look, ApexCharts)
API	SvelteKit `/api/v1` routes, Zod schemas, OpenAPI generated and committed (CI fails if stale)	Thin wrappers over packages; one contract for native clients and a future MCP server.	Separate API server (tRPC / Fastify): a second deployable for no gain
Data store	PostgreSQL, latest GA major (18 at Plan time; bump deliberately)	RLS, deferred constraint triggers, `numeric`, SECURITY DEFINER — the whole integrity story lives here.	SQLite (no RLS), external ledger services (excluded by kickoff §2.5)
ORM /	Drizzle ORM	TS schema as the single source; RLS policies and	Kysely + generated

Layer	Choice	Why	Alternatives considered
query layer		roles declared in schema; BetterAuth's tables are declared in the Drizzle schema, so auth tables share one migration chain. Decision, see §6.2.	types; Prisma (weak RLS / raw-SQL story, heavy engine)
Migrations	drizzle-kit `generate` (SQL files, committed) + hand-written custom SQL migrations for triggers / functions / roles, applied by a one-off migrate container	Plain SQL in the repo, a tracking table, never hand-run (kickoff §2.13). drizzle-kit 0.31.11 / drizzle-orm 0.45.4, exact. Decision — see §6.2.	dbmate + Kysely codegen (fallback), Atlas, graphile-migrate, Supabase CLI (mosko-fintech incumbent)
General ledger	Home-grown schema in our Postgres (build)	Decision, see §6.1.	pgledger (adopt or fork), TS ledger libraries (Medici), mosko-fintech's journal model; external services ruled out at kickoff §2.5
Money arithmetic	Postgres `numeric`; decimal strings on the wire; a decimal library (e.g. `big.js`) in TS	No binary floats anywhere near money.	Integer minor units (awkward for share quantities and FX rates)

Layer	Choice	Why	Alternatives considered
Auth	BetterAuth, pinned to an exact version (measured on 1.7.18), on its built-in Kysely Postgres path with our routing pool (§3.1c); auth tables declared in the Drizzle schema. OAuth: Google / GitHub (Apple with native iOS, POLY-122); MFA: TOTP and passkeys; no `bearer` plugin; email via Resend	Kickoff §2.8: identity in our DB, no leased tier. The routing pool, which puts BetterAuth's writes in our transaction, was measured only on the `pg`-pool path; the Drizzle adapter takes a fixed `db` and the same routing there is unmeasured.	Clerk (rejected: external identity store), Lucia (maintenance mode)
Jobs	Plain idempotent job units; VPS: `apps/worker` with a Postgres-backed scheduler (pg-boss); Vercel: Vercel Cron → an authenticated internal route that runs the same unit	No Redis, no extra service; Postgres is already the system of record. Dual-target rule (kickoff §2.8).	BullMQ + Redis (extra service), system cron in the container (no retries or visibility)
Monorepo	pnpm workspaces + Turborepo (affected-only)	Kickoff §2.10; per-package tests and versions.	Nx (heavier), plain pnpm filters (no caching)
Tests	Vitest per package; Postgres via a disposable container (Testcontainers or CI service); pgTAP for the RLS / trigger battery; Playwright smoke suite (login, import a batch, confirm a draft, view a statement)	Kickoff §2.13. DB invariants are tested in SQL where they live.	Jest (slower ESM story)
Hosting / runtime	Primary: Hetzner VPS, Docker Compose (Caddy, web, worker, Postgres). Secondary: a user-owned Linux box on a local network, same Compose, no inbound from the internet (§7.8). Tertiary: Vercel + Neon, CI-verified	The two Compose paths share one deployment shape; the Vercel path is the zero-ops last resort, an adapter-only difference (kickoff §2.8).	Fly.io, Railway (leased), Kubernetes (overkill)

Layer	Choice	Why	Alternatives considered
Reverse proxy / TLS	Caddy	Automatic HTTPS in one short config file, which suits the novice-operator runbook.	Traefik, nginx + certbot
Observability	Structured JSON logs (pino) with a redaction list (credentials, tokens, account numbers); Postgres slow-query log; uptime ping. OTel export is optional	Cheap stack. Logs must be safe by construction because PII lives in memos.	Hosted APM (subscription)

6. Plan-phase Research & Decisions

The two research items the PRD (§10) assigns to the architect; each cites its `process/DECISIONS.md` entry.

6.1 General ledger: build vs. adopt

Scope (kickoff §2.5): the ledger lives in our Postgres under our RLS. External services (Formance, TigerBeetle, Blnk), plain-text engines and ERPs are already out. The question is only: a home-grown schema, or a library that manages tables inside our own Postgres.

Candidate	What it is	Fit against our requirements
pgledger (pgr0ss, MIT)	A single SQL file of tables, functions and views: accounts with denormalized balance + version, two-legged transfers, entries carrying previous/current balance.	✕ No tenant column, no RLS. ✕ The primitive is a two-legged transfer; our entries are multi-leg with one posting date. Batched transfers approximate this, but reports would have to re-group them. ✕ Single currency per account and no `native / base / fx_rate` per line. ✕ No hierarchical CoA, normal side, custodial flag, or reversal link. ✕ No versioned migrations yet (its own TODO). Upgrades would mean diffing a vendored file.
TS/Node double-entry libraries (e.g. Medici)	App-level ledger libraries.	✕ Medici is MongoDB. We found no mature TS library that manages Postgres tables under host-app RLS (web survey, 2026-09-27).
moskō-fintech journal model	Single-entry `account_trans` legs grouped post-hoc by `journal`; GL derived by an imputation function (`fn_gl_entries`); `users_id` as sole tenant anchor.	✕ Not a stored double-entry GL, and direct-owner tenancy contradicts our membership model. Its *patterns* port well, though (below).

Candidate	What it is	Fit against our requirements
Home-grown	~6 tables (§3.1) + 2 triggers + a posting service.	✓ Fits every requirement directly. Cost: we own correctness. We mitigate that with DB-enforced invariants (§3.2), property-based tests (random balanced / unbalanced entries), and the RLS battery.

Decision: build (`process/DECISIONS.md` → *2026-11-09 — The general ledger is built in our own schema, not adopted*). A GL core is small. The hard parts here are membership tenancy under RLS, multi-leg / multi-currency-ready lines, the securities sub-ledger, and declarative reports, and no candidate library provides any of them. Adopting pgledger would mean forking it to add all of them, which is building with extra steps. Alternatives considered: pgledger adopted or forked, TS libraries such as Medici, and the external services kickoff §2.5 already ruled out.

What we take and what we leave, each traced to the issue that builds or guards it:

Item	Source	Built or guarded by
Append-only entries as the source of truth	pgledger	POLY-107 (posted rows reject UPDATE, DELETE, TRUNCATE)
Per-line running balance: **deferred** until a measured need; at household scale, `SUM` over an indexed `(entity_id, account_id, posting_date)` is expected to be fast (unmeasured)	pgledger	Guard: POLY-63's under-2 s dashboard criterion; a miss there reopens this row
Immutability-trigger fence	mosko-fintech 004	POLY-107 (append-only and TRUNCATE triggers); POLY-84 checks TRUNCATE triggers on every table
Append-only lot-match junction with `match_seq`, latest batch current	mosko-fintech 032	POLY-67 (POLY-M4)
Price observations, unique (security, date, source), with source priority	mosko-fintech 019	POLY-67 (POLY-M4)

Item	Source	Built or guarded by
Structural de-dup: unique index + `ON CONFLICT DO NOTHING`, skips counted as duplicates on `import_batch`	mosko-fintech 021	POLY-64
RLS inversion self-test	mosko-fintech tests/00	POLY-84
Account-number masking (last 4 only)	mosko-fintech 002	POLY-108
NOLOGIN owner; migrations apply as `poly_migrator` via `set role poly_owner`	mosko-fintech	POLY-76 (c), POLY-83
Leave behind: direct-owner `users_id` tenancy	mosko-fintech	POLY-62 (membership tenancy); POLY-84's catalog-derived cross-tenant probe
Leave behind: post-hoc journal grouping	mosko-fintech	POLY-107 (stored, balanced entries); POLY-64 groups drafts before posting (G3)
Leave behind: Supabase `auth.uid()` / JWT policy clauses	mosko-fintech	POLY-83 (`app.current_user_id()` over a transaction-local GUC, A-3)

6.2 Migration tool

Constraints (kickoff §2.13): migrations are files committed with the code and never hand-run; CI applies them to a fresh Postgres on every run; the VPS runs a one-off migrate step before the app starts; Vercel runs it as a build step against Neon. On top of that, our schema is heavy on triggers, SECURITY DEFINER functions, roles and RLS policies.

Incumbent check (mosko-fintech): 120 hand-written plain-SQL files applied with `supabase db push`, tracked in `supabase_migrations.schema_migrations`. Lessons worth keeping: a tracking table is non-negotiable (a bare `psql -f` loop was ruled out there); migrations must be tested *through the real apply path*, not psql (their `set local role` was a measured no-op under

`db push`); objects need a NOLOGIN owner distinct from the applying login. The tool itself doesn't carry over: Supabase CLI assumes a Supabase stack we don't run.

Tool	For	Against
Drizzle Kit (decision)	TS schema → generated SQL; `pgPolicy` / `pgRole` / `enableRLS` in schema; custom SQL migrations in the same chain for triggers and functions; BetterAuth's tables are declared in the Drizzle schema (from its CLI generator), so they share one chain, while BetterAuth itself queries through its Kysely path on our routing pool (§3.1c); the query layer and the migration tool share one schema, so types never drift.	Triggers, functions, grants and FORCE RLS aren't modelled, so they live in hand-written files. The runner takes no lock and orders by timestamp, not hash (§6.2).
dbmate	Plain SQL, tiny, language-agnostic, boring; tracking table; trivially run in a container.	No schema-as-code. Types would come from a codegen step (Kysely), and BetterAuth tables go through its own CLI generator.
Atlas	Declarative diffing, strong linting.	Some Postgres object types sit in the paid tier (unmeasured); a second schema language (HCL) alongside TS.
graphile-migrate	Excellent SQL-first dev loop (`current.sql`).	Postgres-only SQL workflow with no TS schema; smaller community.

Decision: Drizzle Kit (spike ruling at `process/cairn/reviews/POLY-76/ruling.md`). Measured on Postgres 17 with drizzle-kit 0.31.11, drizzle-orm 0.45.4 and better-auth 1.7.18:

- **(a), (b), (c) pass.** A `pgPolicy` round-trips with no spurious diff. BetterAuth's tables migrate in the same chain. A trigger and a definer function apply through the real runner as `poly_migrator`, owned by `poly_owner`, with one tracking row per journal entry.
- **(b′) passes with a `fields` map.** The CLI emits snake_case columns and the Kysely path looks up camelCase, so BetterAuth refuses to start until mapped. The columns stay

snake_case, so no hand-written SQL needs quoted identifiers. `db` derives the map from the Drizzle tables, so it cannot drift. The §3.1c grants hold as `poly_app`. dbmate would meet the same coupling, so this does not discriminate between the tools.

- **(e) passes with FORCE RLS in custom SQL.** Drizzle Kit has no way to declare FORCE, so the generator never sees it and stays clean. Nothing in the tool notices a missing FORCE; the catalog probe of §3.3 does.
- **(d)** is deferred to POLY-121 (Neon from a Vercel build). **(f)** concerns sign-up (§3.1), not the tool.

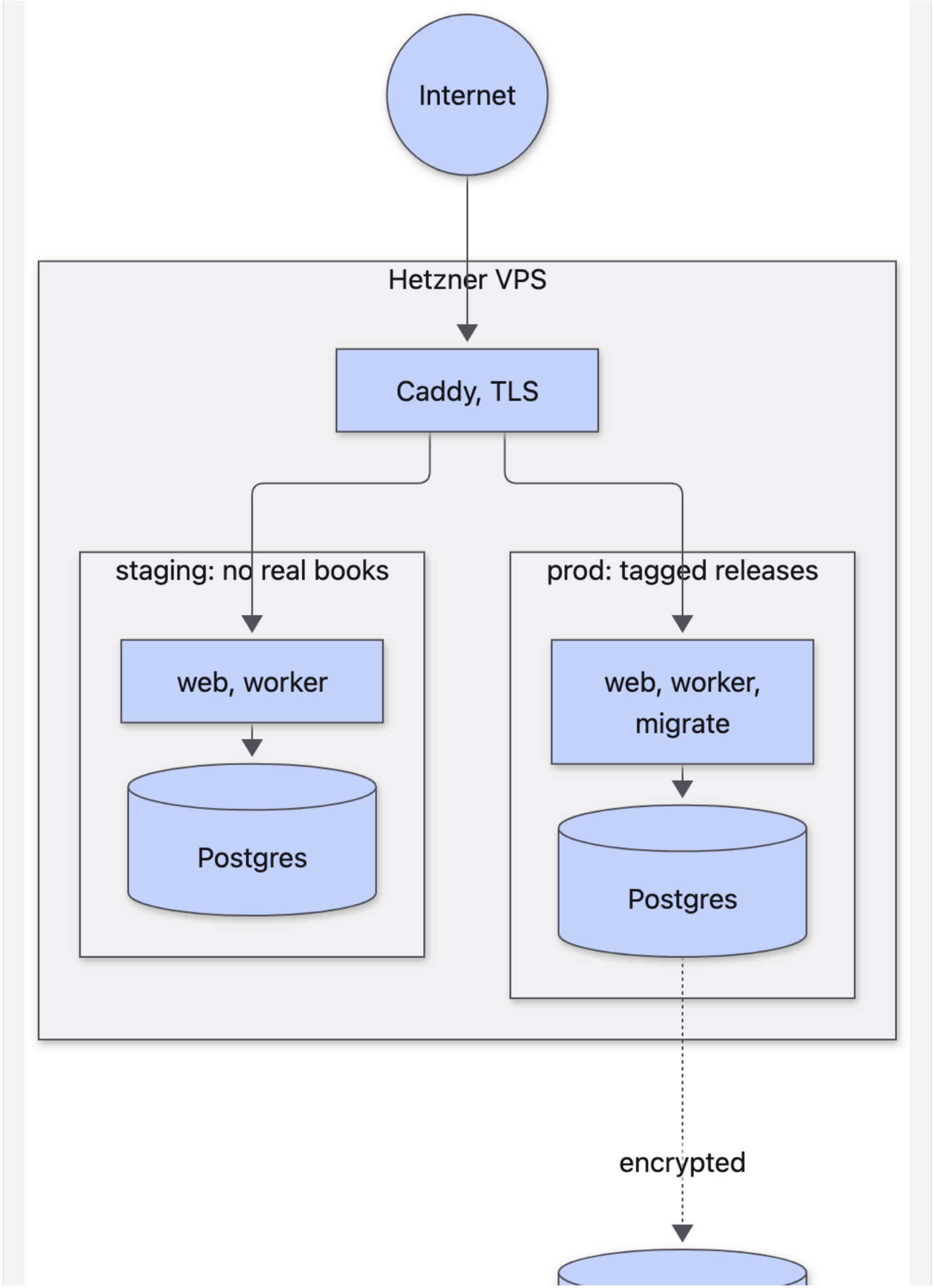
What the build carries (POLY-83, POLY-86, POLY-100):

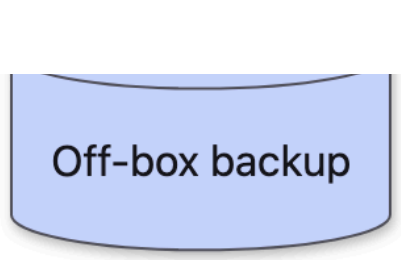
- **Ownership.** The migrate client logs in as `poly_migrator` and runs `SET ROLE poly_owner`, so audit keeps the login. A custom file that creates an object for another definer owner switches with `SET ROLE` and ends on `SET ROLE poly_owner`. A file check enforces that, and also refuses `SET SESSION AUTHORIZATION`, `RESET ROLE` and a `SET ROLE` to anything but `poly_owner` or a definer-owner role (the C-10 set). Drizzle's `migrate()` commits inside itself and offers no hook before the commit (read), so the end-of-file role is checked by privilege instead. Only `poly_owner` may insert into the tracking table, and no other role holds INSERT, UPDATE, DELETE or TRUNCATE on it, so a file that ends on another role fails its tracking insert and the whole run rolls back. That holds only if `poly_migrator` cannot use `poly_owner`'s privileges without switching, so its membership is granted `WITH INHERIT FALSE, SET TRUE` and without ADMIN OPTION (Postgres 16 and later; unmeasured, the spike used INHERIT). The same holds for its membership in the definer-owner roles. The runner also asserts `current_user = 'poly_owner'` when `migrate()` returns, as a second check. The runner uses one client, never a pool, and closes it in `finally`. `poly_owner` is NOLOGIN NOBYPASSRLS, and `poly_migrator` is its only member. `poly_migrator` owns no object, which a catalog test checks over `pg_class`, `pg_proc`, `pg_namespace` and `pg_type` (SECURITY C-85, T-91, A-40). Custom migrations are created with `generate --custom`.
- **Tracking.** `drizzle.__drizzle_migrations`, created by `poly_owner`. `poly_migrator` needs no database CREATE of its own (to be measured on POLY-83). The runner compares the tracking rows with the journal over their common prefix, hash for hash, and refuses on any difference; tracking rows beyond the journal mean the database is ahead of the image, so nothing is pending. After `migrate()` it checks that the rows now equal the journal, which catches the silent skip below. Exit codes, the lock key and the order of steps: `process/cairn/reviews/POLY-83/ruling.md` R6. `poly_app` and `poly_worker` get USAGE on the schema and SELECT on the table, for the start-up check that the schema is not behind the image.

- **What Drizzle's runner does not do** (read from drizzle-orm 0.45.4, not run). It applies every pending migration in one transaction, so a failing file changes nothing. It takes no lock. It counts a migration as pending only if its journal timestamp is later than the last applied one, and it never compares hashes. A migration merged with an older timestamp would therefore be skipped silently, and an edited applied file would go unnoticed. So our runner takes an advisory lock first, and refuses to run unless the tracking rows match the journal's start hash for hash. CI refuses a journal whose timestamps do not strictly increase. One transaction means no `CREATE INDEX CONCURRENTLY`.
- **BetterAuth's tables** are edited from the CLI output, not frozen. Every timestamp is `withTimezone` (`timestamptz`, unmeasured with BetterAuth, measured on POLY-86), and the grants follow §3.1c's column list exactly.
- **Versions.** drizzle-orm and drizzle-kit are pinned exactly to the stable line measured. The 1.0 line (release candidate at the spike) is a separate upgrade, gated on the §3.3 battery and a clean second `generate`. ARCH fixes the Postgres major at 17; the image and minor are INFRA's.

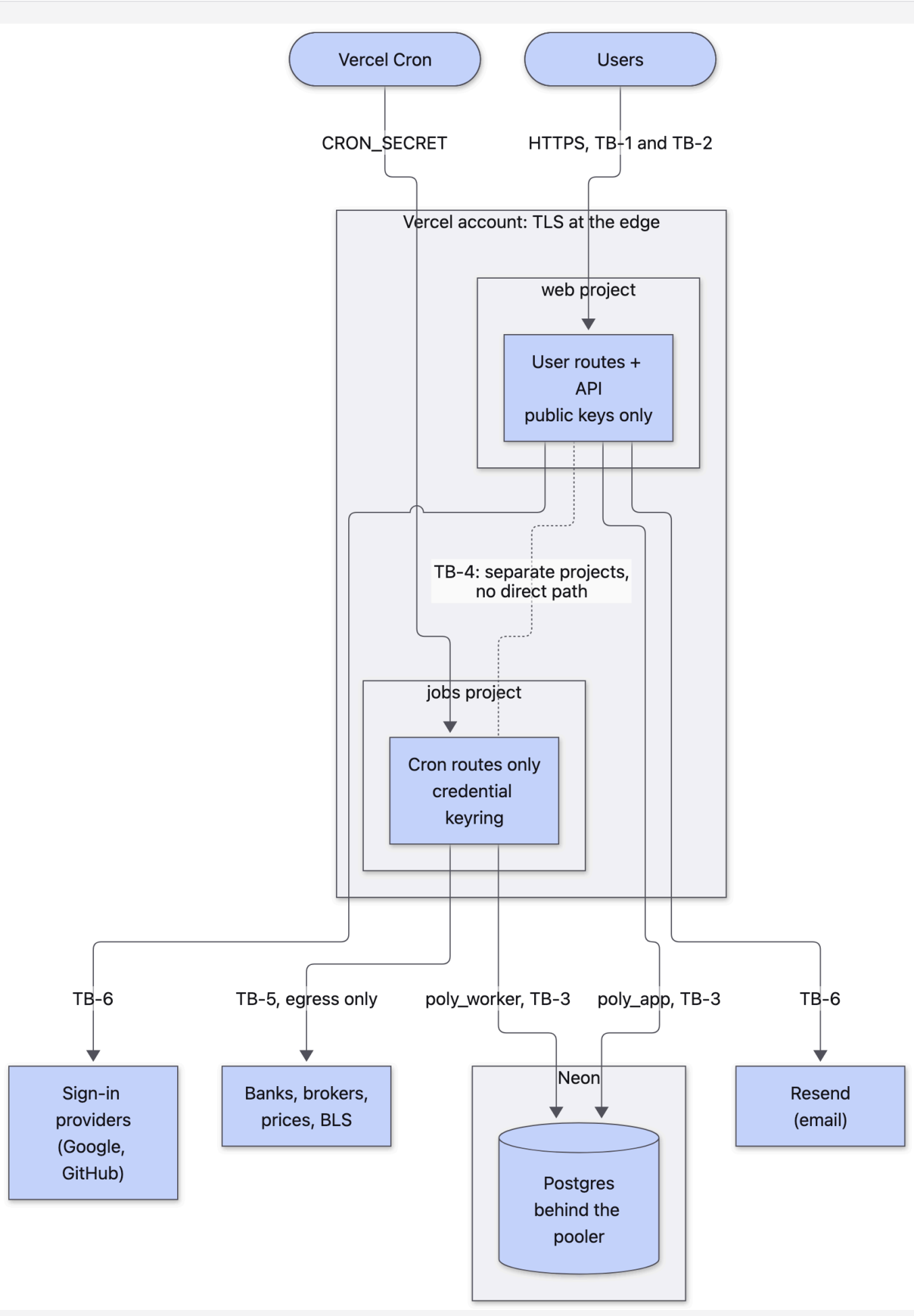
7. Deployment Topology

Environments, network paths, trust boundaries. DevOps owns the INFRA-level detail; cross-reference SECURITY.


Internet
Hetzner VPS
Caddy, TLS
staging: no real books
web, worker
Postgres
prod: tagged releases
web, worker, migrate
Postgres
encrypted



Co-located: the Docker Compose path on the Hetzner VPS. One box runs everything behind Caddy.


Vercel Cron
Users
CRON_SECRET
HTTPS, TB-1 and TB-2
Vercel account: TLS at the edge
web project
User routes +
API
public keys only
TB-4: separate projects,
no direct path
jobs project
Cron routes only
credential
keyring
TB-6
TB-5, egress only
poly_worker, TB-3
poly_app, TB-3
TB-6
Sign-in
providers
(Google,
GitHub)
Banks, brokers,
prices, BLS
Neon
Postgres
behind the
pooler
Resend
(email)

Split: the Vercel + Neon path (§7.6). What differs from Compose: there is no Caddy, because Vercel terminates TLS. There is no worker container: `jobs` runs the same job units when Vercel Cron calls them, capped at once daily on the Hobby plan. Postgres is Neon's, reached through its pooler. Secrets are Vercel env vars instead of mounted files. Migrations run as a build step. The `web` ↔ `jobs` line is the same TB-4 as `web` ↔ `worker` on the box, held by a project boundary instead of a container boundary.

- **Trust boundaries** (numbering and controls from SECURITY §3):
  - TB-1: internet ↔ Caddy.
  - TB-2: clients ↔ app.
  - TB-3: app / worker ↔ Postgres (limited roles, RLS).
  - TB-4: `web` (seals with public keys only) ↔ `worker` (holds the private keyring, mounted into worker only; plaintext exists only inside one `open()` call frame). Redefined by SECURITY §6.3.
  - TB-5: worker ↔ providers (egress only).
  - TB-6: app ↔ sign-in providers (Google, GitHub) and Resend.
  - TB-7: box ↔ backup store (encrypted before it leaves the box).
  - TB-8: operator SSH ↔ box (superuser; accepted risk R-1).
  - TB-9: ordinary write path ↔ purge path.
  - TB-10: prod ↔ staging compose projects on one host (shared kernel; accepted risk R-4).
  - TB-11: user upload (CSV / OFX, attachments) ↔ web parser.
  - TB-12: supply chain (GitHub CI / Actions / Renovate / registry) ↔ prod images. Whoever can publish an image that prod runs owns the box.
  - TB-13: operator SSH tunnel ↔ loopback-bound Admin listener, never routed by Caddy. Placeholder, defined under POLY-81 (OQ-15).
- **Postgres is never exposed publicly.** It lives on the compose network only; the operator reaches it through `docker compose exec` or an SSH tunnel.
- **Only Caddy publishes ports** (SECURITY C-70): every other service (`web`, `worker`, Postgres, migrate) has no `ports:` mapping at all, reachable only over the compose network. Prod and staging run as separate compose projects on separate compose networks — no shared network, so a staging container can't reach a prod one by DNS name even if TB-10's shared kernel is compromised. No Docker socket is ever mounted into a container. Every container runs non-root, `read_only`, with `cap_drop: [ALL]` and `no-new-privileges`; a compose-policy lint enforces this in CI (C-70).
- **The firewall is the Hetzner Cloud Firewall**, not host `ufw` (SECURITY C-71): it sits outside the host, so a Docker-published port can't bypass it the way it bypasses `ufw`. Only 22 (SSH,

key-only, no root login), 80 and 443 are open. Unattended security upgrades are enabled on the box.

- **Environments:** *dev* — local compose (Postgres + web + worker) with the `fake` provider; *CI* — fresh Postgres service per run; *staging* — same VPS, separate compose project and database, no real books, auto-deploys every merge to `main`; *prod* — tagged releases from `v1.0.0` only (kickoff §2.13). No real books exist before v1.0. Staging on the same box is a cost trade-off (§10).
- **Hosting order:** primary the Hetzner VPS; secondary a local-network box, the maximum-security option (§7.8); tertiary Vercel + Neon. These are deployment targets, not separate architectures.
- **Tertiary target:** the same app built with `adapter-vercel` against Neon, as **two Vercel projects from the same repo** (SECURITY C-61; §7.6). CI builds it and runs the smoke suite against a Neon branch (or a local Postgres standing in for one); no Vercel-specific APIs anywhere.
- **Sizing and location:** a 4 vCPU / 8 GB shared-x86 Hetzner Cloud server in Falkenstein (`fsn1`, EU) — see §7.3.

7.1 Environments

Environment	Data	Deploy trigger	Access
dev	`fake` provider only	local compose, manual	developer machine only
CI	fresh Postgres service per run, torn down after	every PR	GitHub-hosted runner only
staging	The deterministic `fake` source as the repeatable fixture, plus sandbox data through real adapters: Plaid Sandbox, and SimpleFIN's demo token. Never real books.	auto, every merge to `main`	same VPS, separate compose project + DB; reachable behind Caddy basic auth / IP allowlist — not indexed, not linked from prod
prod	real data, from v1.0	tagged releases only, via `/merge-pr`. A `0.x` tag cut before GA is a	public, TLS via Caddy

Environment	Data	Deploy trigger	Access
		staging release, and the prod poll ignores it.	

7.2 Backup & Disaster Recovery

- **Staging co-location:** staging stays on the prod VPS (confirms §10's trade-off). Staging holds no real books, so its compromise has no ledger-confidentiality blast radius; splitting to a second box roughly doubles the fixed monthly cost for a single-household deployment with no compliance driver forcing separation. Revisit only if the app is offered to other self-hosters at meaningfully larger scale, or staging starts touching anything beyond fake data.
- **Backup store (INFRA D-3, §6.1):** a Backblaze B2 bucket in B2's EU region with Object Lock in **compliance mode** and a 35-day default retention, one bucket per instance, written through B2's S3-compatible endpoint over HTTPS 443 by `rclone` in the upload step, which holds no database credential (below). The box holds a **put-only** key (write on this bucket only; no delete, no retention changes, no governance bypass); a separate read key lives on the operator's laptop for drills and restores; the account master key is never on a box. The store, not the box, enforces that nothing is deleted or overwritten for 35 days, which is also the bound on how long purged data can survive in backups (SECURITY R-5). One mechanism on every target, because 443 is the only outbound port the home box allows (C-76). A Hetzner Storage Box is a documented fallback on the Hetzner target only, once the C-73 cannot-delete test passes against it; Hetzner Object Storage is an alternative pending verification of its lock. The key capabilities are **(unmeasured)** until the first drill (INFRA OQ-INFRA-4).
- **Mechanism, revised per SECURITY §6.6 (asymmetric recipient, not restic's symmetric repository password):** a nightly set of three artifacts — a `pg_basebackup -F tar -X none` of the cluster piped to stdout (the physical base WAL replays onto; a `pg_dump` cannot be one; tar to stdout cannot stream WAL, so the segments the base needs come from `archive_command` like every other, INFRA R-10, unmeasured, OQ-INFRA-11 checks it), a `pg_dump --format=custom` of the database (the portable fallback, restorable onto the same or a newer major), and `pg_dumpall --globals-only --no-role-passwords` (the roles RLS depends on, which a per-database dump omits) — and continuously-shipped WAL segments (`archive_command`, `archive_timeout = 300`) are each encrypted with `age` (X25519, the same primitive family as the credential keyring's HPKE scheme) to a **recipient public key**, then uploaded. The box holds only the age *public* key — sufficient to encrypt, useless to

decrypt. `archive_command` runs inside the `postgres` container and encrypts each segment with `age` before it lands in the spool, so our Postgres image is the pinned upstream major plus `age`, and the spool never holds plaintext WAL (INFRA R-11). This is a deliberate change from an earlier `restic`-based draft: restic's repository password is symmetric, so a box holding it to *write* backups could also *read* them, which fails SECURITY §6.6's "box holds only the public half" requirement outright. Dedup/incremental snapshotting is given up for this; at single-household data volumes the cost is negligible.

- **Postgres major pinned (INFRA §6.1, §9.3):** a physical restore needs the same major, so the major lives in one committed file, `infra/postgres-major` (one line, e.g. `17`), is `17`, the major the POLY-76 spike measured (§6.2). It is read by the backup object names (`<instance>/<pgmajor>/…`) and the Neon project setup. The minor and digest are INFRA's and are written in one place: the `FROM postgres:17.<minor>@sha256:…` line of the Postgres derivative image. Compose, CI's Postgres service and the backup container all take their image from that line, never retyped (INFRA §6.1a). The two must agree: a CI check fails when that line's major differs from `infra/postgres-major`. A minor bump changes only that line, while a major upgrade changes both, as a planned step that takes a fresh base backup straight after.
- **The backup job's database role** (SECURITY C-79, A-36, T-89): `poly_backup` (§3.3) is the one login with `BYPASSRLS`. It needs `REPLICATION` for `pg_basebackup`, and `BYPASSRLS` with `pg_read_all_data` for `pg_dump`, which refuses to dump a FORCE-RLS table it cannot read whole. `BYPASSRLS` grants no read the role does not already have, since a physical base backup copies every row of the cluster. What is controlled is where the credential can be used and where its output can go:
  - *Attributes:* `LOGIN REPLICATION BYPASSRLS NOSUPERUSER NOCREATEDB NOCREATEROLE CONNECTION LIMIT 3`; a member of `pg_read_all_data` and nothing else; no other grant; owns nothing; `ALTER ROLE poly_backup SET default_transaction_read_only = on`.
  - *Where it connects from:* the `backup` container sits on its own compose network, shared with `postgres` only, `internal: true` (no internet route) with a fixed subnet. Compose addresses are not stable, so `pg_hba.conf` matches the subnet, not a container address. Its first lines allow `poly_backup` for `replication` and the `polycarpic` database from that subnet (`scram-sha-256`), then reject `poly_backup` from anywhere else and any replication connection from anywhere else. `db_password_backup` is mounted into `backup` alone.
  - *The credential and the internet never share a process:* `backup` holds `db_password_backup` and the age public key. It pipes each dump and base backup through `age` as it is taken, so the spool holds only `.age` files. The upload is a separate

step (a host script or an `uploader` container) that holds only the put-only key and the spool and is not on the backup network. (INFRA §2.2).

  - *Output:* `pg_dumpall --globals-only --no-role-passwords` keeps password hashes out of the globals file; restored logins get new passwords from the password manager. The data dump does carry BetterAuth's tables, which is why the age private key is on no box.
  - *Slots and WAL:* `max_replication_slots = 2`, `max_wal_senders = 3`, `max_slot_wal_keep_size` 2 GB to start ((unmeasured), tuned with OQ-INFRA-5), so a slot cannot fill the disk. If `pg_replication_slots` shows any non-temporary slot, the 5-minute health ping is skipped and an alert is raised (`pg_basebackup -X none` opens none; WAL comes from `archive_command`).
  - *Detection and one path:* `log_connections = on`, and `polycarpic-ops status` flags any `poly_backup` connection in the last 24 hours that no backup or pre-migration run accounts for. The pre-migration backup (§8.1) and the nightly set both run through this container; no other dump path, and no superuser dump, exists in the scripts.
  - *Tests (C-79):* pgTAP on the attributes and membership, and an INSERT, UPDATE or DDL as `poly_backup` fails. `provision-check`: a `poly_backup` login (SQL and replication) is rejected from `web` and from the uploader and succeeds from `backup`; `backup` cannot reach the internet; the uploader has no `db_password_backup`; the spool holds only `.age` files; the slot and WAL settings are in place. A compose-policy lint checks that only `backup` and `postgres` are on the backup network, that it is `internal`, and that only `backup` mounts the password. The restore drill counts a FORCE-RLS table's rows in the dump against the base-backup restore.

- **Key custody:** in plain terms, backups are locked with a key that never lives on the server, so stealing the server does not open them, and losing both offline copies of that key means the backups can never be opened. The age keypair is generated once at bootstrap, off the box. The public half is placed on the VPS (non-secret — it can only encrypt). The private half lives in the operator's password manager and a printed offline recovery copy per the runbook; it is **never** on the VPS, and is a wholly separate key from the credential keyring (§6.1/§6.4) — the two are never stored together, so a single compromised secret can't unwrap both live credentials and backup history. Rotation (new keypair) is a manual runbook step on suspected compromise; old backups stay readable under the old private key, which the runbook says to retain until its last covered backup expires.
- **RPO:** ≤ 15 minutes via continuous, individually-encrypted WAL segment shipping; the nightly full dump is the tested fallback restore point (RPO 24h) if WAL replay is unavailable.
- **RTO:** ≤ 4 hours for total loss of the VPS — provision a replacement box from the committed provisioning scripts (`hcloud` CLI and cloud-init, INFRA D-7), decrypt the latest base backup and WAL with the offline private key, restore, replay, redeploy the last tagged release image. ≤

60 minutes for a bad-migration rollback that doesn't need a box rebuild, from the pre-migration backup (§8.1). That backup is encrypted to the offline key, so the restore runs from the operator's laptop, never on the box, and no plaintext dump is kept there (INFRA R-9). Per-target figures, including the home box's 1 day and the Vercel target's, are in INFRA §6.4.

- **Restore drills:** monthly (SECURITY C-73, tightened from this section's earlier quarterly draft), scripted, and run **off the prod VPS** — the operator's own machine, or a throwaway scratch VPS spun up and destroyed for the drill — never a scratch compose project on the box itself: that would put the age private key on the VPS for the drill, which this section's own key-custody rule (§7.2 above) forbids outright, drill or not. Checked against a fixture query (trial balance sums to zero); logged as a runbook checklist item, not assumed to work from the backup job succeeding alone. The drill script also asserts the store credential cannot delete an existing object (C-73's second test, and the measurement the store bullet above is waiting on), and that the restored cluster carries the RLS roles from the globals artifact. The drill is a GA gate (INFRA §6.7).

7.3 Sizing

A 4 vCPU / 8 GB RAM shared-x86 plan rather than 2 vCPU / 4 GB, named by specification because Hetzner renames its plans (INFRA §2.1 records the plan name at order time): Postgres (prod + staging) + web + worker + Caddy fit comfortably inside 8 GB idle, and the extra headroom absorbs backup/WAL-shipping runs, migration bursts, and the staging Playwright smoke without contending with prod. **Unmeasured** — confirmed against real resource metrics after POLY-M1 lands; downgrading to 2 vCPU / 4 GB is a one-line change in the provisioning file if it proves oversized. Data lives on a separate 20 GB Hetzner Cloud Volume, LUKS2-encrypted and mounted at `/srv/polycarpic` (SECURITY C-72), so the root disk is replaceable. Swap is off on both Compose targets, so a memory spike hits a container cap, not a page of decrypted data on disk; per-container memory caps are INFRA §2.2.

7.4 Observability & Operations

- **Logs:** web / worker emit structured JSON to stdout (one schema, committed at `packages/config/log.schema.json`: `ts, level, service, event` required, a closed set of declared optional fields, no free-text message). The logger, `@poly/config/log`, is an allowlist: undeclared keys are dropped, errors log as `{type, code}` only, and pino `redact` is the backstop (SECURITY C-33). Docker's `json-file` driver with `max-size` / `max-file` rotation captures them; `docker compose logs` / `journalctl` is the query surface at this scale. No log-aggregation service until a second box or third-party hosting is in play.

- **Health, split in two (INFRA R-6, D-10):** public `/api/v1/health` answers `ok` or not and nothing else, so a stranger learns nothing about schema version or backup cadence. The detail goes on a **status listener** inside the `web` process, bound to `127.0.0.1` within the container, never given a `ports:` mapping and never routed by Caddy (TB-13): `GET /status` returns DB connectivity, the applied schema version against the image's expected one, and the time of the last successful job runs. The host scripts read it with `docker compose exec web`. Backup age is not app state: the backup job writes its last success to a host state file that `polycarpic-ops status` and the alerting read. The full Admin view (OQ-15, POLY-81) may move this listener into its own `admin` service; the seam (loopback only, read-only, no tenant data) stays. Payloads: health is exactly `{"ok":true}` (200) or `{"ok":false}` (503), true when the database answers and the schema is current or ahead; `/status` on `STATUS_PORT` (default 8081) returns a versioned object of service, image commit, start time, db reachability, schema state with expected and applied tags, and last job runs.
- **Two alert paths, independent (INFRA §6.5):** path A is app and host-script email through Resend; path B is a dead-man's switch at Healthchecks.io (free tier) that each box pings after a passing self-check, a backup upload and a deploy poll, and that emails the operator when a ping stops. It holds no data and never probes the box, so it works for the outbound-only home box as well. One failure (a dead box, a dead mail relay) never silences both.
- **Metrics:** container resource metrics via `docker stats` on demand; no Prometheus/Grafana until scale or a second operator justifies the extra service.
- **Alerts** (all by email — no on-call rotation for a household of one): backup job failure or a backup older than 26h; a missed heartbeat (path B); a failed or rolled-back deploy; **a ledger hash-chain break**, SEV1, from the weekly per-entity verifier (§3.2.9), which also emails the entity's owners; a verifier run that has not completed in 8 days alerts as well, so a stopped verifier is not silence.
- **Dashboard:** the Healthchecks.io check list (green or red per box) plus `polycarpic-ops status` (running digest, last deploy, last backup, certificate days left, disk) is the v1 dashboard — the thing the team-lead can point to at launch. A Grafana-class dashboard is a documented future initiative once observability volume justifies the extra container.

7.5 Secrets Delivery & Postgres Config

DevOps's INFRA-level implementation of SECURITY §6.2 and §12.3's remaining asks.

- **Secrets as files, not env vars (SECURITY §6.2, C-74):** every operator secret lives under `/etc/polycarpic/secrets/` on the host (root:root, files `0400`, directory `0700`), mounted into containers as Docker Compose `secrets:` at `/run/secrets/<name>`. Apps read them through a `*_FILE` convention (e.g.

`DATABASE_PASSWORD_FILE=/run/secrets/db_password`), never a bare env var — `docker inspect`, `/proc/<pid>/environ` and an error reporter dumping `process.env` all leak plain env vars, never a file only the right container can read. Per-container scoping: `web` never gets the credential private keyring or the backup private key; only `worker` mounts the keyring secret. Staging gets its own distinct secret values and its own keyring, which seals sandbox tokens only; it never holds a real provider credential. No secret is ever baked into an image or passed as a build arg; the app refuses to start outside development if a required secret file is missing. The loader is `@poly/config`: one registry lists every `*_FILE` variable with the services allowed to mount it; a required one unset, a missing, unreadable or empty file, a registered file set in a service not listed for it, or an unregistered `*_FILE` refuses start, naming the variable and never the content. Every entrypoint (`web`, `worker`, `migrate`) takes `check-config`, because each container sees only its own secrets. `packages/secrets` stays the user-credential vault; it reads no environment and no file.

- **The auth secret** signs session cookies and also encrypts TOTP seeds, backup codes and OAuth access and refresh tokens (measured, 1.7.18). It is a secret file like the rest, mounted into `web` and `worker` only. Rotation goes through BetterAuth's versioned `secrets` option: the new version encrypts, older versions still decrypt until each row is rewritten (code-read; SECURITY §6). Replacing the secret outright ends every session and makes every enrolled factor undecryptable, so it is the incident path only.
- **Postgres logging (SECURITY C-75):** `log_parameter_max_length = 0` and `log_parameter_max_length_on_error = 0` (bound parameter VALUES never reach the log, even on error — a sealed credential's ciphertext is one thing, but a query parameter carrying a raw token before it's sealed is exactly what this guards), `log_statement = 'ddl'` (schema changes only, never row data). Set in the Postgres compose service's config, asserted by a config-check in the DB battery (§8's DB battery row).
- **Connection pooling per target (INFRA D-6):** the Compose targets run **no pooler** in v1; the app's own `node-postgres` pool holds the connections, and `set_config(…, true)` (§3.3) is transaction-local, so reuse is safe. A pooler service added to compose later is refused by the compose-policy lint unless it declares transaction mode. On Neon the runtime uses the pooled endpoint (PgBouncer, transaction mode) and migrations use the direct endpoint. A CI test connects through each target's endpoint (a local PgBouncer in transaction mode stands in for Neon on every run, a real Neon branch on the scheduled run), sets the setting inside a transaction, commits, and asserts that a later transaction on the same backend sees it empty — the pool-reuse property the pgTAP battery checks at the RLS layer, asserted here at the endpoint.

7.6 Vercel Tertiary Target: Two-Project Split

SECURITY C-61, R-3. DevOps builds the CI check.

- **Topology:** the Vercel target is two Vercel projects deployed from the same repo, not one. `web` exposes only user-facing routes, holds only public keys and the `poly_app` DB credentials. `jobs` exposes only cron routes (each checking `CRON_SECRET` with `timingSafeEqual`), holds the credential keyring and the `poly_worker` credentials. This is the same TB-4 (`web` seals / `worker` opens) split the VPS target already has via separate compose services — Vercel just can't isolate them inside one deployment, so it takes two deployments instead.
- **CI check, three legs (SECURITY C-61's own "build tests"):**
  - *Route manifest vs. allowlist.* Each project's build output (SvelteKit's route manifest under `.vercel/output/functions/`) is diffed against a checked-in allowlist per project — an exact-set match, not a subset check, so a route silently added to the wrong project's allowlist is caught too, not just one silently missing. `jobs`'s allowlist is exactly the cron routes; `web`'s is everything else.
  - *No keyring reference in the `web` build.* A bundle-scan (grep the built output for the keyring module's resolved import path, same mechanism the existing `packages/secrets` import-boundary lint uses source-side) asserts `open()` and the keyring module never got bundled into `web`'s build artifact — catches a route misclassified into `web`'s allowlist that would otherwise pass the manifest check by having the right PATH but the wrong CODE behind it.
  - *Node build 404s job routes.* The VPS's `adapter-node` build (one image, `web` / `worker` / `migrate` entrypoints, §8.1) is hit on its `web` entrypoint at each cron route path and must 404 — job routes exist only behind the `worker` entrypoint there, and this is the cross-check that the VPS and Vercel targets enforce the identical TB-4 boundary by two different mechanisms (separate compose service vs. separate Vercel project) without drifting apart.
- **Deploy (INFRA D-11, §4.7):** the operator runs `infra/vercel/deploy.sh` from the laptop on a signature-verified tag; Vercel's own Git deployment points at a branch nobody pushes to. No Vercel deployment credential lives in GitHub, consistent with C-83: nothing GitHub holds can move a deployment. A failed migration step fails the build and Vercel keeps serving the previous deployment.
- **Backups (INFRA §6.6):** a cron function cannot run `pg_dump` or ship WAL, so the target relies on Neon's point-in-time restore for the plan in use, plus a weekly `polycarpic-ops neon-export` run from the operator's laptop through a read-only role, encrypted to the same age recipient and put into the same bucket. RPO is up to 7 days beyond Neon's window. No database credential is held in GitHub; revisited with SecEng only if real data lands on this target.

- **Env check in CI (C-61):** a Vercel API call, made with a **read-scoped** Vercel token held as a secret of a GitHub environment with required reviewers, lists each project's configured env var *names* (never values) and asserts the keyring variable name is present in `jobs` and absent from `web` — catches an operator (or a future IaC change) setting it on the wrong project directly in the Vercel dashboard, which the build-time checks above can't see.
- **Cron frequency — measured:** Vercel's Hobby plan caps cron jobs at once per day, with timing guaranteed only within the hour (Pro lifts this to per-minute). Since Hobby is already the ceiling this self-hoster fallback target targets, daily is not a choice DevOps is making for cost reasons — it is the only cadence Hobby allows at all. Import polling, EOD prices and period-end jobs on the Vercel target therefore run once daily, a real (accepted) data-freshness gap against the VPS target's more frequent worker loop; SecEng's framing ("daily polling is acceptable for this fallback target") holds given Vercel is the tertiary target, the last resort of the hosting order.
- **Cost:** two Hobby-tier Vercel projects, same as one — Hobby is free per project, so the split costs nothing beyond the CI check's own build time (one extra `adapter-vercel` build, ~same cost as the existing one). If POLY-77 ever needs Pro-tier cron frequency, that's a per-project upgrade, not a re-architecture.

7.7 Sync Rate Floors

The INFRA minimum interval SECURITY §4.2.1 and §3.1 defer to, for "Sync now" and the per-entity import schedule. Rejected below the floor (SECURITY §4.2.1).

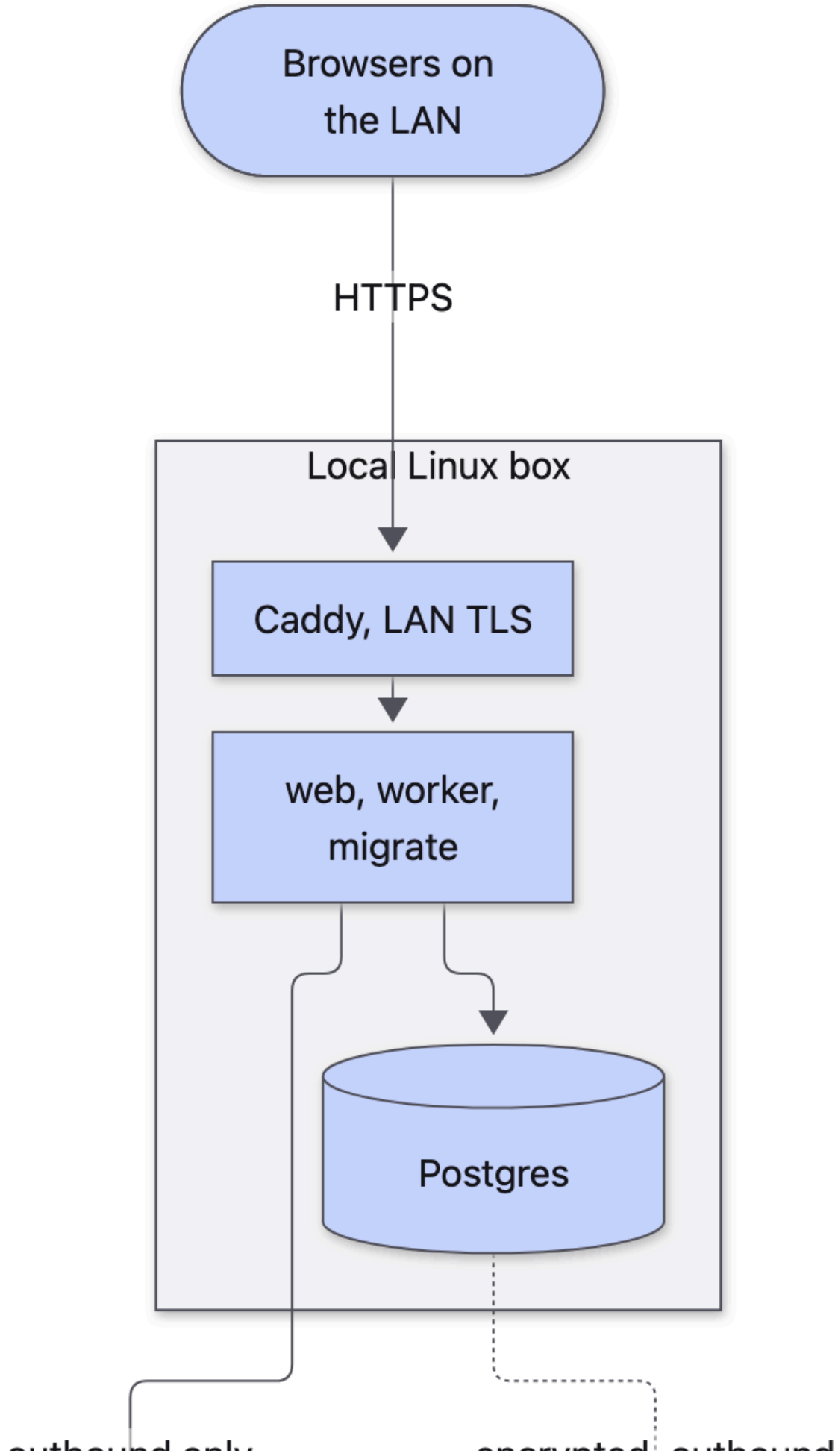
- **Aggregators (SimpleFIN, Plaid) — floor 2 hours, default 6 hours.** Measured (provider docs, 2026-09-28): SimpleFIN Bridge caps a connection at 24 requests/day, with warnings then access-token disablement beyond that — the binding constraint, since it has no per-call cost at all. Plaid's own `/transactions/sync` limit (50 req/min/Item in production) is far looser, and Transactions is billed per active Item monthly, not per call, so Plaid imposes no comparable floor of its own. One shared floor governs both providers rather than branching per-provider in the schedule UI: **2 hours** (≤ 12 scheduled requests/day) leaves ≥ 12/day of SimpleFIN's budget as headroom for manual "Sync now" clicks and backoff retries without risking disablement, even if every connection runs at the floor. The out-of-the-box **default is looser than the floor, 6 hours** (4 syncs/day) — ample freshness for a personal dashboard that isn't real-time trading, and it leaves most of the SimpleFIN budget unused for the common case.
- **Broker APIs (Schwab, IBKR) — floor 1 hour, default 24 hours (EOD), both provisional (unmeasured against a registered app).** Schwab's Trader API publishes generous, per-app-configurable limits (on the order of 120 req/min for market data) with no stated per-call cost; IBKR's are comparably loose for read-only position/lot queries. Neither is the binding constraint for a buy-and-hold personal-investor persona — product reasoning is: lots and

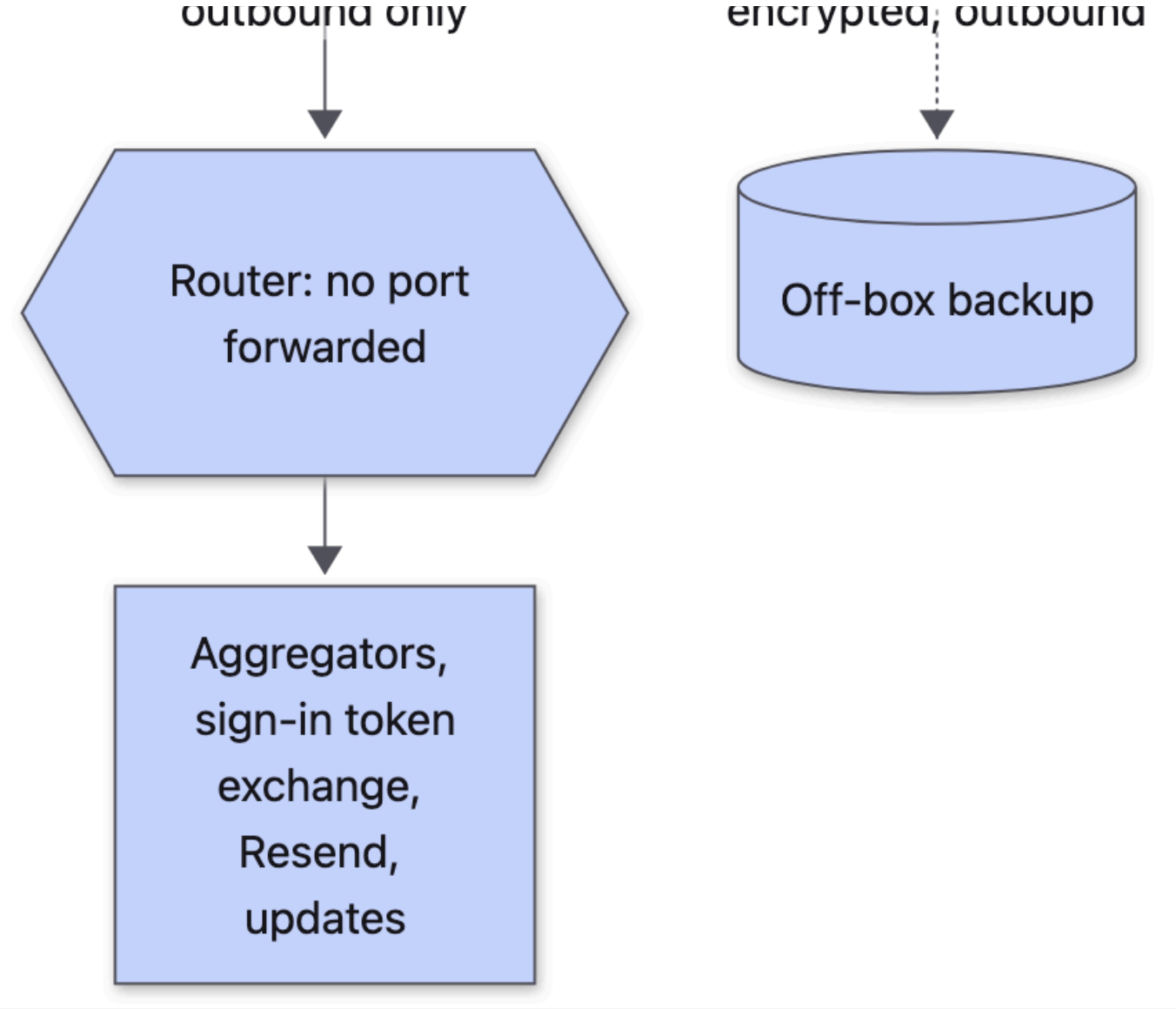
positions settle end-of-day, so there is little value polling a brokerage intraday the way a checking account's pending transactions reward polling. Floor is set looser than the aggregator floor (1 hour) because the provider headroom is real, not because the data changes faster; default matches the existing EOD price-provider cadence (§4.4) at once daily. Reopen when POLY-67/69 registers a real developer app and its actual configured limits are known — "provisional" here means the number, not the reasoning, may move.

- **Enforcement:** both floors are a single config value each (`SYNC_FLOOR_AGGREGATOR_SECONDS` , `SYNC_FLOOR_BROKER_SECONDS`), read by the same validation SECURITY §4.2.1 already specifies for "Sync now" and the per-entity schedule UPDATE policy — one seam, not a per-provider branch in application code. A provider added later slots into whichever floor its own rate limit and pricing most resembles; a provider that doesn't fit either (a hard per-call cost, for instance) gets its own floor then, not before.

7.8 Local-Network Secondary Target

The same Compose project on a user-owned Linux box, outbound-only. SECURITY specifies the posture; this section states what changes against §7.


Browsers on the LAN
HTTPS
Local Linux box
Caddy, LAN TLS
web, worker, migrate
Postgres



Concern	On the local box
Ingress	LAN only. The router forwards no port, and the Cloud Firewall (C-71) has no equivalent, so the host firewall does that job. Docker can bypass `ufw`; SecEng specifies the control.
TLS (SECURITY C-76)	HTTP-01 cannot work, since nothing reaches in, so Caddy runs in one of two modes, set by explicit configuration (`TLS_MODE`). **Default, real domain:** a subdomain of a domain the operator owns, resolved only on the LAN (split horizon, no public A or AAAA record), with a DNS-01 wildcard certificate so the box's own name stays out of CT logs; the domain's DNS is Cloudflare's free DNS in DNS-only mode, and its API token is scoped to that one zone and mounted into Caddy only (R-18). **Supported fallback, `.local` with a private CA:** an operator-run CA whose root, name-constrained to the box's name, is generated off the box and kept offline; the box holds only an intermediate issuing short-lived leaves;

Concern	On the local box
	the root is installed on each device (R-20, browser enforcement of the constraint measured at the first home-box deploy). Plain-http sign-in is refused in both modes.
OAuth sign-in	The callback is a redirect in the user's own browser and the token exchange is outbound, so neither needs an inbound path; the hostname is the constraint (Google requires a public-suffix TLD, measured 2026-11-06; GitHub accepts any registered callback URL, unmeasured). **Real-domain mode:** Google and GitHub work (SECURITY A-27). Sign in with Apple is not offered on the web in v1; it arrives with the native iOS milestone (POLY-122), which also needs a real domain. GitHub asks for `read:user` and `user:email` only, and its verified email comes only from `/user/emails`, the entry that is both primary and verified, never the profile's public email (C-05). **Private-CA mode:** both OAuth providers are disabled server-side: the buttons are not rendered and every OAuth route returns 404, so a stale link or a crafted request finds nothing. Sign-in is email, password and the second step; TOTP is always offered, since passkeys on a `.local` RP ID are unmeasured.
Mode cross-check at startup	The app derives the mode it needs from its public origin and refuses to start on a mismatch (SECURITY A-26, C-76). **Suffix test, offline and decisive:** a name ending in `.local`, or with no public suffix, requires `TLS_MODE=private-ca`. **Zone test, otherwise:** an SOA query for the registrable domain from a public resolver; an answer means real-domain mode is required; an NXDOMAIN refuses start; a DNS timeout starts anyway, logs and retries, so a flaky upstream never keeps the box down. The test is on the zone, not the host: in real-domain mode the host deliberately has no public record, so "does the name resolve publicly" would misfire. A public record for the box's own name is a warning, not a refusal. The served leaf's issuer is checked against the mode by the provisioning step and the health check. Startup needs outbound DNS, which C-76 already allows.
Moving a `.local` box to a real domain	The origin changes, so every session ends and passkeys, bound to the old RP ID, do not carry over. Before the move every owner and manager must hold an authenticator app or backup codes, and the runbook's move step checks that first. After it, Google sign-in is linked from Settings while signed in; never auto-linked by matching email. The INFRA runbook covers both modes and the move.
Email	Resend sends outbound as usual, but a verification or magic link only opens from the LAN.

Concern	On the local box
links	
Aggregators	Unchanged. Polling is outbound and the design uses no provider webhooks, so nothing needs to reach in.
Deploy	Unchanged. Pull-based deploy is already outbound-only.
Backups	Same `age` encryption and the same nightly set and WAL stream, uploaded over HTTPS 443 to the same kind of Object-Lock bucket (§7.2) on a put-only key (C-73); SFTP is not available under C-76's outbound allowlist. RPO 15 minutes; RTO 1 day, with a Hetzner VPS restored from the same bucket as the break-glass at about 4 hours, then moved back; no spare hardware is kept (INFRA D-4, §6.4). A rotated drive is optional and never kept beside the box. They must leave the box, because a fire or theft takes the box and anything beside it. Disk unlock is TPM-sealed (C-72; INFRA).
Monitoring	An outside monitor cannot reach in, so the box pings Healthchecks.io outbound (§7.4) after it has fetched its own LAN name successfully, and an absent ping raises the alert.
Access away from home	None. Access from outside the LAN, and from later native clients off Wi-Fi, needs a VPN into the LAN, which is outside v1.
Staging	Not required; the box runs tagged releases that staging already proved.

8. CI/CD Pipeline

Owned by `devops-engineer`; this is the shape, INFRA holds the detail.

Stage	Tool	Gate
Change detection	Turborepo affected graph; docs-only changes skip test jobs, except that a change to `docs/ARCH/index.html` or `layers.json` runs the layer-map checks (§2.1)	—
Lint + types	ESLint with `eslint-plugin-boundaries`, generated from `layers.json`; the layer-map checks (§2.1); `svelte-check`, `tsc` resolving through `exports`	Must pass
Unit / package tests	Vitest, affected packages only	Must pass
DB battery	Apply all migrations to a fresh Postgres as `poly_migrator`; pgTAP RLS + trigger battery (incl. inversion self-test, pool-reuse test)	Must pass when `packages/db` or any schema changes
BetterAuth routing-pool test	Vitest against a real Postgres, as `poly_app`, on the pinned `better-auth` and `@better-auth/passkey`: the trigger sees `app.session_id` on our backend and txid; rollback removes BetterAuth's rows; nested transactions become savepoints; a factor write outside `withTenant` is refused; the first TOTP verify runs in one transaction in the measured order (§3.1c; SECURITY A-31)	Must pass when `packages/auth`, `packages/tenancy` or either BetterAuth version changes; a dependency bot's BetterAuth bump cannot merge without it
BetterAuth route-policy test	Vitest, on the same pinned versions, BetterAuth built from the exported app configuration: every HTTP route of the instance is in `AUTH_ROUTES` and every entry is a live route; the per-class checks of §3.1c (SECURITY A-34, A-38), run in real-domain and private-CA mode. Lint refuses `drizzleAdapter` in `packages/auth`.	Must pass when `packages/auth` or a `better-auth` version changes
OpenAPI	Regenerate and diff against the committed spec	No diff

Stage	Tool	Gate
freshness		
Build	`adapter-node` image; `adapter-vercel` build (both Vercel projects, §7.6)	Both must build
Vercel topology check	Route manifest vs. allowlist per project; no keyring reference in the `web` build; node build 404s job routes; env-name check via the Vercel API with a read-scoped token in a reviewer-gated GitHub environment (§7.6)	Must pass when `apps/web` routes or Vercel project config change
Smoke (integration milestones)	Playwright: login, import a batch, confirm a draft, view a statement	Must pass
Security scan	Dependency audit, secret scan	No high-severity findings
Deploy → staging	Pull-based: the box polls and verifies attestation (§8.1)	Auto on merge to `main`
Deploy → prod	Same mechanism, triggered by a release tag cut by `/merge-pr`	Tag only
Deploy → Vercel (tertiary)	The operator's `infra/vercel/deploy.sh` on a verified signed tag; migrations in the build step on Neon's direct endpoint (§7.6)	Operator only; no deploy credential in GitHub

8.1 Deploy mechanism

- **Decision, revised per SECURITY C-83: pull-based, not push-over-SSH.** An earlier draft of this section chose GitHub Actions → SSH push; SecEng's explicit preference is pull-based specifically because it means **no GitHub-held credential can ever reach prod** — the box, not a GitHub Actions runner, decides when and what to deploy. A `systemd` timer on the box (every 5 minutes) runs a small poll script: query the GitHub Releases API for the newest `v*` tag (prod) or the registry for the newest `main`-branch image digest (staging); if it differs from what's running, pull the image **by digest** (never a mutable tag), then verify. A failed check at either verification step aborts the poll cycle and alerts (§7.4) — the box never runs an unverified image.
- **Attestation verification, pinned per SecEng's delta (SECURITY C-73/C-83 @ 61b4d2a):** `gh attestation verify <digest> --repo richmosko/polycarpic --signer-workflow .github/workflows/build.yml` (the workflow that builds and attests; `release.yml` only looks the digest up and never rebuilds, INFRA §4.4) — pinning the repo and the exact signer workflow, not just "an attestation exists" (an unpinned check would accept a provenance attestation from any workflow in any repo the same GitHub identity can sign for). **Prod adds a second, independent check:** the `v*` tag itself must carry an SSH signature verifiable against `/etc/polycarpic/allowed_signers`, installed on the box at bootstrap from the operator's own machine and changed only by an operator step — never fetched from the repository, where anyone who can push could forge it (INFRA D-5) — and the tag's commit must be the one the digest was built from, read from an image label the attestation covers (`org.opencontainers.image.revision`, the full 40-hex commit, set from a required `GIT_COMMIT` build argument in the root `Dockerfile`) (the same signing `/merge-pr` already does before cutting a release tag, SECURITY §8 "Signing") — so a compromised registry or attestation service alone still can't move prod without also forging a tag signature staging never checks.
- **The box's GitHub credential:** a fine-grained PAT, scoped to this one repo only, `contents:read` + `packages:read`, with an expiry (rotated per the runbook, same cadence discipline as §6.4's other operator secrets) — never a classic PAT, never org-wide. If the GHCR package and the repo are ever made public, the PAT is dropped entirely: an unauthenticated `gh api` / registry pull needs no credential on the box at all, which is strictly less attack surface than a scoped-but-present one.
- **Build once, promote:** CI builds a single `adapter-node` image per commit, pushes it to GHCR tagged with the git sha, with an SBOM (CycloneDX) and a provenance attestation attached (SECURITY §8); that same image digest is promoted from staging to prod on tag, never rebuilt. This rules out a staging/prod code-drift class of bug. `adapter-vercel` builds separately in CI as the tertiary-target check (§7) but is not deployed by this pipeline. The one image serves three compose services through different entrypoints: `web`, `worker` and `migrate`. Each

gets only its own secrets; the credential keyring is mounted into `worker` alone (TB-4), and the layer map keeps `open()` out of the web bundle (§2.1).

- **Deploy script** (idempotent, same for staging/prod, under `flock`, invoked by the poll timer once it has a verified digest; INFRA §4.4–§4.5): extract the **ops bundle** (compose files, Caddyfile, scripts) from the verified image into `/opt/polycarpic/releases/<digest>/`, so the compose definitions carry the same attestation as the code → `migrate check-config` → `migrate --plan` → if anything is pending, a **pre-migration backup** (an encrypted `pg_dump`, uploaded and kept in the local spool for 7 days) → `migrate` under an advisory lock → `docker compose up -d` → poll the health check for up to 120 seconds. Healthy: record the new current digest and ping the monitor. The deploy script and the timer never change themselves during a deploy; `polycarpic-ops self-update` replaces them through the same verification, by an operator step.
- **The two migrate subcommands the script relies on** (backend-lead implements; same image, `migrate` entrypoint, `poly_migrator` password only): `migrate check-config` reads the connection settings `DATABASE_HOST`, `DATABASE_PORT`, `DATABASE_NAME`, `DATABASE_USER`, `DATABASE_SSLMODE` and `DATABASE_PASSWORD_FILE` (POLY-83), and the mounted secret files and `host.env` (POLY-85), without connecting anywhere and exits non-zero, naming what is wrong, if a required secret file is missing or empty, `TLS_MODE` fails the §7.8 cross-check, or `PLAID_ENV` is not `sandbox` on staging. `migrate --plan` connects read-only, lists the pending migration files in order and exits 0 if none are pending, 3 if some are, and any other code on error; it never takes the advisory lock and never writes. `web` and `worker` refuse to start when the applied schema is *behind* the image, and start when it is ahead, which is what lets the previous release run on the new schema. The image knows its version from the migration journal bundled with it: the guard (`db/startup`) starts only if the journal's timestamps are a prefix of the tracking rows'. `--plan` also exits 2 when the chain has diverged, and the deploy aborts on it.
- **Interruption during a deploy (INFRA §4.6, R-4):** `up -d` recreates `web`, so for a few seconds Caddy has no healthy upstream; Caddy retries for 10 seconds (`lb_try_duration`), which hides it for most requests. There is no blue-green swap on one box; v1 accepts those seconds (§10).
- **Automatic rollback:** if health does not pass within 120 seconds of `up -d`, the script points the desired digest back at the current one, restarts that release's containers from its own ops bundle, writes the failure to the deploy log and alerts. The failed digest goes into `/opt/polycarpic/quarantine` and the poll skips it until a newer one appears. A failed migration file rolls back in its own transaction, so the old containers keep running on the old digest.

- **Manual rollback (one command):** `polycarpic-ops rollback` points the poll script's "desired digest" file at the previous release's digest and re-runs the deploy script directly (no need to wait for the next timer tick, and no dependency on GitHub's Releases API being reachable at rollback time — the operator already knows the prior digest from the deploy log), and quarantines the newer release so the poll does not re-deploy it. Migrations are additive / backward-compatible by convention (expand-contract — no destructive migration ships in the same release as the code that depends on the drop), so a code rollback never needs a schema rollback. A schema-level rollback is rare and falls back to the DR restore path (§7.2), starting from the pre-migration backup.
- **Operator secrets** on the box live as files under `/etc/polycarpic/secrets/`, mounted at `/run/secrets/` into each container that needs it (§7.5) — placed once during VPS bootstrap and never touched by CI or the poll script, which only ever reads a public digest and a public attestation. Neither transmits a secret value onto, or reads one off, the box.

9. Integration Points

Limits marked (unmeasured) are to be verified against provider docs before each adapter's milestone.

Service	Purpose	Auth	Limits	Failure mode
Fake source	Deterministic transactions for three personas; fake EOD walk	none	none	n/a — used in dev, CI and staging
CSV / OFX file import	Credential-free import path (SecEng's preferred default for self-hosters)	none (user upload)	upload size cap	Parse errors surface per row; nothing is posted without confirmation
SimpleFIN Bridge	Fallback aggregator, for institutions Plaid does not cover (e.g. Synchrony Bank). Not used or paid for unless a custodial needs it. A demo setup token gives sample data without an account (measured, developer guide, 2026-11-06).	Setup token → access URL (bearer-equivalent; sealed)	24 requests/day/connection, measured (§7.7) — binding constraint on the shared aggregator sync floor	Job retries with backoff; the dashboard shows "last synced"; no data loss because de-dup is by hash
Plaid	Aggregator for custodials with no file or free direct API. Plaid Sandbox is the pre-GA test source (§7.1).	client id + secret (operator secret) + per-item access token (sealed)	Per-item monthly pricing; 50 req/min/Item on `/transactions/sync`, measured (§7.7) — looser than SimpleFIN's, not the binding constraint	As above; item-login-required surfaces a relink prompt; `/item/remove` on unlink
Schwab / IBKR APIs	Positions, trades, lots. Preferred over Plaid where free, so a trial-tier Item is not spent	OAuth 2.0 / broker-specific	Provisional floor set (§7.7); exact per-app limits unmeasured until POLY-67/69	As above

Service	Purpose	Auth	Limits	Failure mode
			registers a real developer app	
EOD price provider	Daily closes for held securities	API key (operator secret)	Provider-dependent (OQ-5)	Fall back to the last observation, flagged stale; source priority lets a better source replace it later
BLS API v2	CPI-U (CUUR0000SA0), monthly	Free per-operator key (low-tier secret); v1 without a key	500 queries/day with a key (PRD §8)	Use the latest stored month; manual CSV import fallback
Resend	Transactional email (verification, magic link, MFA recovery, the first-enrolment and unenrolled-deletion codes and the factor-enrolled notice of §3.1c and §3.4a)	API key (operator secret)	Free-tier volume cap (unmeasured)	Auth email fails loudly; OAuth login still works
Google / GitHub (Apple with native iOS, POLY-122)	OAuth identity providers	OAuth 2.0 client credentials per operator	n/a at this scale	That login method is unavailable; others still work
Backblaze B2	Encrypted backups in an Object-Lock bucket, EU	Put-only key on the box;	Pay-as-you-go storage, cents a month at	Files wait in the encrypted spool on the LUKS

Service	Purpose	Auth	Limits	Failure mode
(backup store)	region, S3-compatible endpoint over 443 (§7.2)	read key on the operator's laptop only	household volume (unmeasured)	volume; a spool older than 2 hours or a backup older than 26 hours alerts; the RPO clock starts
Healthchecks.io	Dead-man's-switch heartbeat, alert path B (§7.4)	A ping URL per check (operator secret); carries no data	Free-tier check count and grace limits (unmeasured, INFRA OQ-INFRA-9)	Silence raises the alert, so an outage there produces false alarms, never missed ones; path A still works
Cloudflare DNS (home box, real-domain mode)	DNS-01 challenge for the LAN wildcard certificate (§7.8)	API token scoped to one zone, mounted into Caddy only	n/a at this scale	Renewal fails and alerts at 14 days left; the current certificate keeps serving until expiry

Data-source priority per custodial: 1 files, 2 a free direct API (e.g. Schwab), 3 Plaid, 4 SimpleFIN. Each custodial account uses the highest-priority source available for it, through the same adapter interface. **Implementation order follows:** fake → files → direct API (Schwab) → Plaid → SimpleFIN. GA needs the files path working; direct and Plaid follow for the custodials the Principal actually holds.

Operator secrets (provider client secrets, API keys, the credential keyring, the DB passwords) live as files on the VPS, each mounted only into the container that needs it (the keyring into `worker` only; SECURITY §6.2). The local box handles them the same way; on the Vercel target they are env vars. They never live in the repo or the database. **User credentials** (access URLs, access tokens) live sealed in the database (§3.1, SECURITY).

10. Trade-offs & Alternatives

- **Home-grown GL over pgledger.** We own correctness, and in exchange we get a model that fits (multi-leg, tenancy, multi-currency-ready lines). Forking pgledger, a TS library such as Medici, and the external services of kickoff §2.5 were the alternatives. §6.1.
- **Drizzle over dbmate.** One TS schema for queries and migrations, plus BetterAuth in the same chain, at the cost of hand-written files for triggers, functions and FORCE RLS, and a runner whose lock and ordering checks we add. The policy differ is measured clean (§6.2).
- **BetterAuth inside our transaction (routing pool) over its own pool.** The enrolment guards can read the session, and a refusal rolls back BetterAuth's earlier writes in the same request, so the first TOTP verify cannot be half-applied (measured). The cost is a ~30-line shim that depends on BetterAuth's internal pool use, hence an exact version pin and a CI test, and using BetterAuth's Kysely path rather than its Drizzle adapter. Keeping its own pool was rejected: the guard could only match "one live grant per user", and a refusal at the last write left `twoFactorEnabled` true with TOTP unverified (SECURITY A-31, A-32).
- **Transaction-local RLS context over per-user DB roles or session `SET`.** Per-user roles don't scale with memberships and complicate pooling; session `SET` leaks across pooled connections. `set_config(…, true)` is pool-safe on both targets.
- **Import split into raw (L2) and posting (L4).** This honours "import is the lowest layer" for raw data while keeping dependencies strictly downward. It costs one extra package.
- **One entry per entity for cross-entity transfers.** Each entity's books balance on their own; there are no cross-entity entries. The cost is that matching must pair across entities and post twice (§4.3).
- **Signed single amount per line over debit/credit columns.** The balance check becomes a single `SUM = 0`; presentation derives Dr/Cr from sign and the account's normal side.
- **Balances computed, not stored.** No running-balance column until a measured need. That removes a whole class of lock contention and drift bugs, at the cost of a query that grows with history (indexed).
- **pg-boss (Postgres) over Redis for jobs.** No extra service. The cost is job throughput we will never need.
- **Web pages call packages directly, not through `/api/v1`.** This skips a serialization hop and keeps SSR fast. The two paths share DTO types, so they can't drift; the cost is that the web UI doesn't dogfood the HTTP API (the Playwright smoke and the native client will).
- **Staging on the prod VPS.** Saves a second box. Staging holds no real books, so a staging compromise exposes none, but the two share a kernel. INFRA may split them later.

- **A recreate with a Caddy retry over blue-green on one box.** A deploy costs a few seconds without a healthy upstream, mostly hidden by Caddy's 10-second retry, in exchange for one copy of each service and no swap logic to get wrong. INFRA §4.6.
- **An Object-Lock bucket over a Hetzner Storage Box for backups.** The store enforces that nothing is deleted for 35 days and one mechanism works on every target over 443, at the cost of a second vendor account. The Storage Box has no measured immutable mode and its SFTP port is closed on the home box (C-76). INFRA D-3.
- **No connection pooler on the Compose targets.** One process pool per container is enough for a household and leaves one fewer service, at the cost of a pooler-mode difference between the Compose and Neon targets that a CI test watches. INFRA D-6.
- **Committed `hcloud` scripts over OpenTofu.** One server, one firewall and one volume need no planner and no state file for a novice operator to lose. Revisit with a second server. INFRA D-7.
- **Pull-based deploy over push-over-SSH (SECURITY C-83).** No credential GitHub holds can ever reach prod directly, at the cost of a systemd timer on the box and up to a five-minute delay between a tag landing and the box noticing it (acceptable for a single-operator deployment with no SLA).
- **Age-recipient backup encryption over restic's symmetric password (SECURITY §6.6).** A compromised box can encrypt new backups but never decrypt any backup, past or future — the cost is giving up restic's deduplication, negligible at this data volume.
- **Period closes over per-entry or year-end accrual.** Liabilities stay current without correcting earlier entries, at the cost of a user step each period and one computed line between closes. §4.5.
- **Chart by nature with custodian sub-accounts .** Allocation's top level and the ledger agree by construction, and a custodial keeps its own view. The cost is more accounts, plus an invariant tying lots to the ledger that every securities write must keep. §3.6.
- **An effective-dated, append-only cap table over a mutable share table.** Any past allocation or economic figure is reproducible, and a mid-year change splits the year correctly, at the cost of a "latest row per holder on a date" query on every allocation read. Equity sits apart from access, so a co-owner needs no purge or export rights to hold units. §3.1a.
- **Refusing entity chains deeper than one link over composing them or treating them as outside.** Tax never silently drops income and the composition stays one join, at the cost of not modelling a holding company inside a trust in v1. §3.1a.
- **A sibling economic function over a weighted mode inside `app.rollup_totals`.** The gross function's shape and tests stay untouched and the economic one is reviewed on its own, at the cost of two definers with one guard to keep in step. §3.3c.

- **Recursive CTE over a closure table for nesting.** One copy of the structure and no path counting on delete, at the cost of walking the graph on each roll-up read. §3.3b.
- **Locked counter row over a sequence for the Journal ID.** Gapless numbers, as accountants expect, at the cost of serializing postings per entity. §3.2.7.
- **Collect deletion choices, then apply.** Abort stays a true abort, at the cost of one more screen and a run that can stop midway on a failed revocation. §3.4a.
- **Reports as data (declarative) over code.** Users can clone and edit, and upgrades don't clobber clones. The cost is building a small formula grammar. Scripting stays deferred (PRD §3).

11. Milestone Map

Development milestones under `POLY-V1`, ordered by the layer model. The files in `process/cairn/milestones/` are authoritative; this is the rationale.

Milestone	Layers	Stories	Integration check at close
POLY-M1 — Foundations	L0, L1, L6 shell	POLY-76 (spike), POLY-62; enablers POLY-82 to POLY-86, POLY-90, POLY-128 (app shell); stories POLY-87 to POLY-89, POLY-91 to POLY-94, POLY-129 (Entities screen); operations POLY-95 to POLY-105; closed by POLY-106	A user signs in, creates an entity, grants a viewer; the RLS battery is green; the multi-owner gate and the founder trigger refuse as §3.1b says; migrate runs in CI and on staging
POLY-M2 — Ledger, Import & Dashboard	L2, L3, L4, first L6 screens	POLY-107 (ledger core), POLY-108 (custodial accounts), POLY-64, POLY-65, POLY-109 (hash chain and verifier, C-24, §3.2.9), POLY-110 (export with the chain head), POLY-63	The fake persona imports, drafts are confirmed into balanced, hash-chained immutable entries, and the verifier passes, categories / tax tags apply, and the per-entity dashboard shows them in under 2 s
POLY-M3 — Transfers & Reconciliation	L4, L6	POLY-71, POLY-111, POLY-72	The couple persona's transfers net CIT to zero; the custodial reconciliation matches
POLY-M4 — Securities & Market Data	L2 refdata, L3 securities, L5	POLY-67, POLY-69, POLY-75	Persona 3's lots, EOD valuation and allocation are correct; the real / nominal toggle works

Milestone	Layers	Stories	Integration check at close
	portfolio		
POLY-M5 — Statements	L5 reports	POLY-73, POLY-68	Seeded templates render; clones survive a template upgrade
POLY-1.0 — Tax, Sharing & Real Data (GA)	L5 tax, L6, L2 real adapter	POLY-66, POLY-113 (period close), POLY-70, POLY-112 (co-owner approval and the founding owner, §3.1b), POLY-114 (purge, including a cap-table holder, §3.4b), POLY-115 (account deletion), POLY-77 (file import as the real-data path; direct and Plaid follow per §9); gates POLY-116, POLY-117	Tags `v1.0.0`: a backup restore drill passes (GA gate, in place of early real data), then the Principal runs one entity end to end on real data through the files path (PRD goal 1)
POLY-1.1 — Consolidated Roll-up	L5 reports, L6	POLY-118 (`app.rollup_totals`, the consolidated view, the parent holdings popover), POLY-119 (`app.rollup_economic` and the gross / economic toggle)	Tags `v1.1.0`: a parent viewer sees totals equal to the parent's own accounts plus the entities beneath it, each counted once at any depth, and a drill stops at the boundary. In the economic view, every holder's figures plus the outside owners' share sum to the gross figure, one spouse's separate trust adds nothing to the other's economic net worth, and the economic net worth sums the parent's roll-up set only

First slice: story 1 (invite sign-in), story 2 (per-entity dashboard) and transaction import through the fake source. It runs from POLY-M1 through POLY-M2, which is why the dashboard moves up from POLY-M3: it needs posted entries to show, so it follows the ledger and import rather than preceding them. The order is POLY-62, the ledger and POLY-64, then POLY-63. The rows above state the intended shape; the milestone files carry it once the tracker is updated.

Roll-up follow-up: the parent link and membership ship in POLY-M1. POLY-1.1 comes immediately after GA and adds the view.

12. Open Questions

Each has an owner and a target resolution point.

#	Question	Owner	Resolve by
OQ-5	Which EOD price provider (free tier, coverage of mutual funds / ETFs)? mosko-fintech used provider-implied prices plus FMP.	architect	Start of POLY-M4 (POLY-67)
OQ-11	Field-level DTO shapes for the six DESIGN screens (mapped to package DTOs in §2.4) are pinned in each package's design file at the start of its milestone.	ux-designer + architect	Per milestone
OQ-15	Operator-only Admin view (POLY-81). It shows basic box and stack status (container health, last migration, backup age, disk, link-health counts) plus admin links still to be defined, and is reachable only through an SSH tunnel to a loopback-bound listener, never through Caddy. Topology options: (a) a route group inside the web process on a second listener bound to `127.0.0.1`, which is cheap but shares the web process and its attack surface; (b) a separate `admin` compose service bound to loopback with its own minimal DB role (read-only status views, no tenant data), which costs one service but isolates it. What it reads, and whether it may act (trigger a backup, rotate keys), is open. Leaning (b), pending SecEng.	architect + seceng + devops-engineer	Under POLY-81
OQ-36	The GL table is named `account` (§3.2), but BetterAuth already owns `public.account` (§3.1c). Rule the ledger tables' schema or the GL table's name before it is created. The §4.2.1 matrix data (`packages/tenancy/permission-matrix.json`, rows `read-books` and `chart-rules-edit`) names `public.account` and is corrected when this is settled.	architect	Before POLY-107

Generated by the `architect` agent via `/generate-archdoc`. Update when ARCH decisions change; log decisions in process/DECISIONS.md.